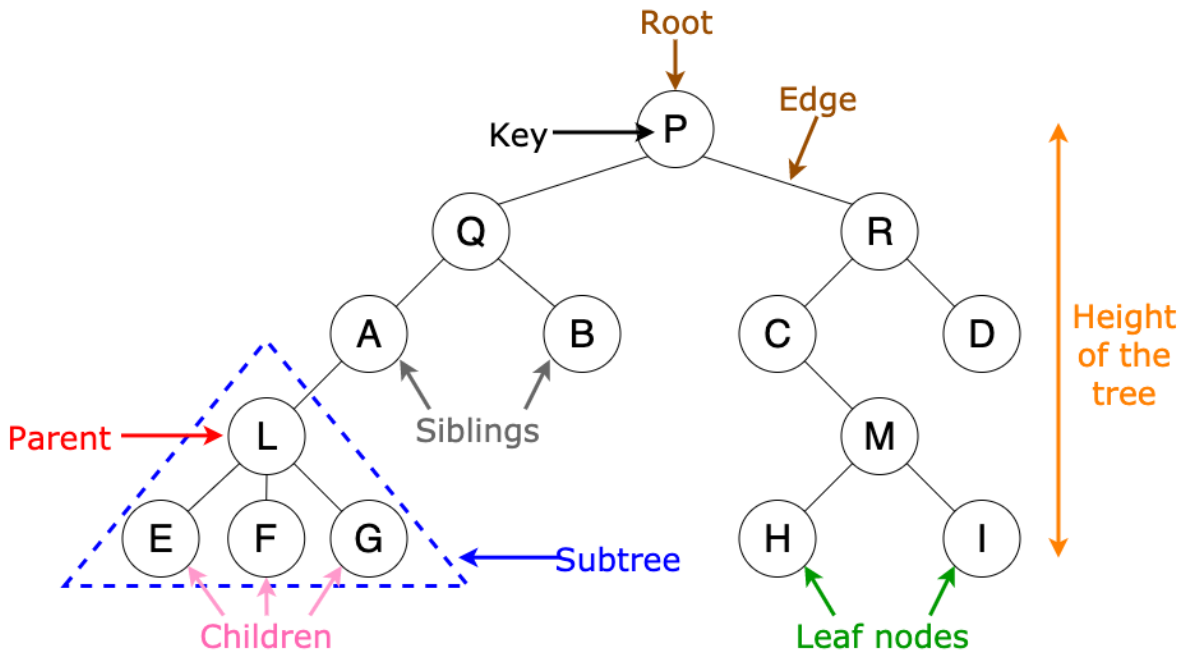
AI VIET NAM
@aivietnam.edu.vn

Data Structure

Tree and Its Applications



Vinh Dinh Nguyen
PhD in Computer Science

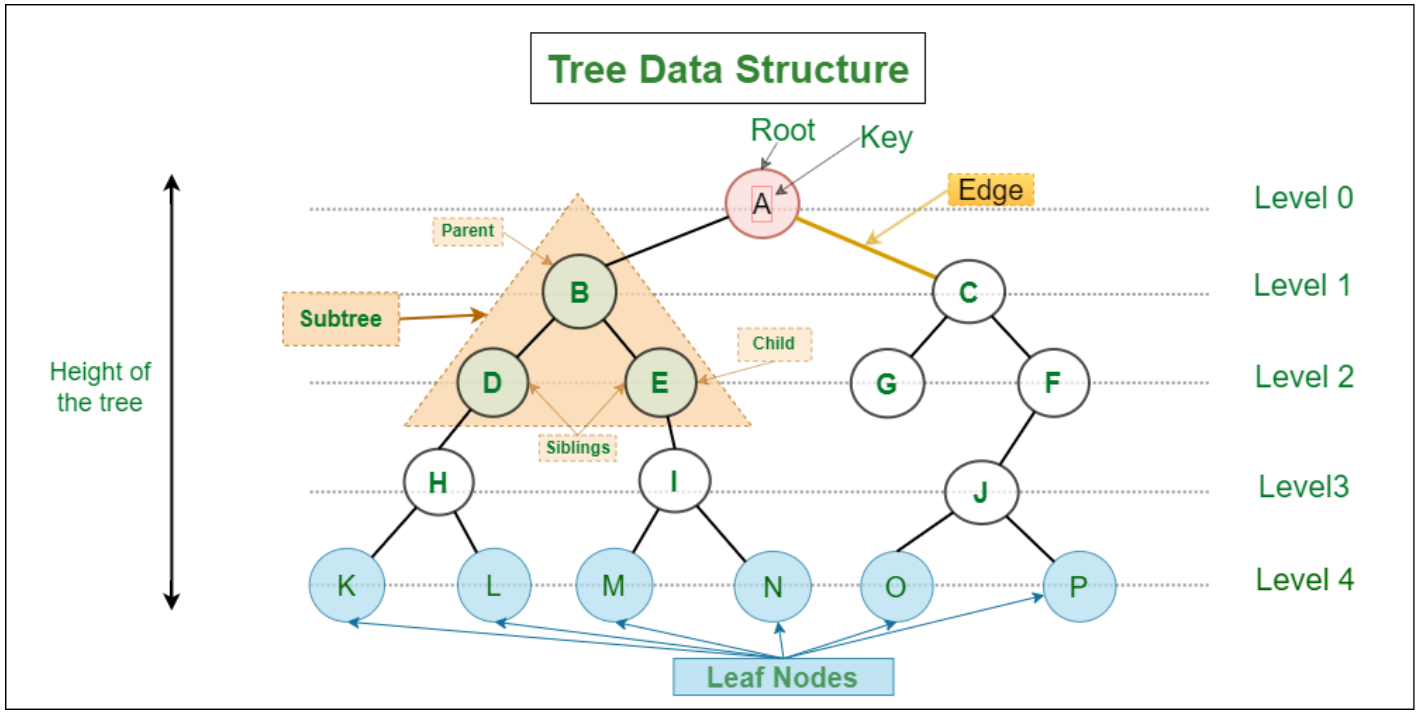
Outline

- What is a Tree
- What is a Binary Tree
- Algorithms on Trees
- What is a Binary Search Tree
- What is a K-D Tree
- Summary

AI VIET NAM
@aivietnam.edu.vn

What is a Tree?

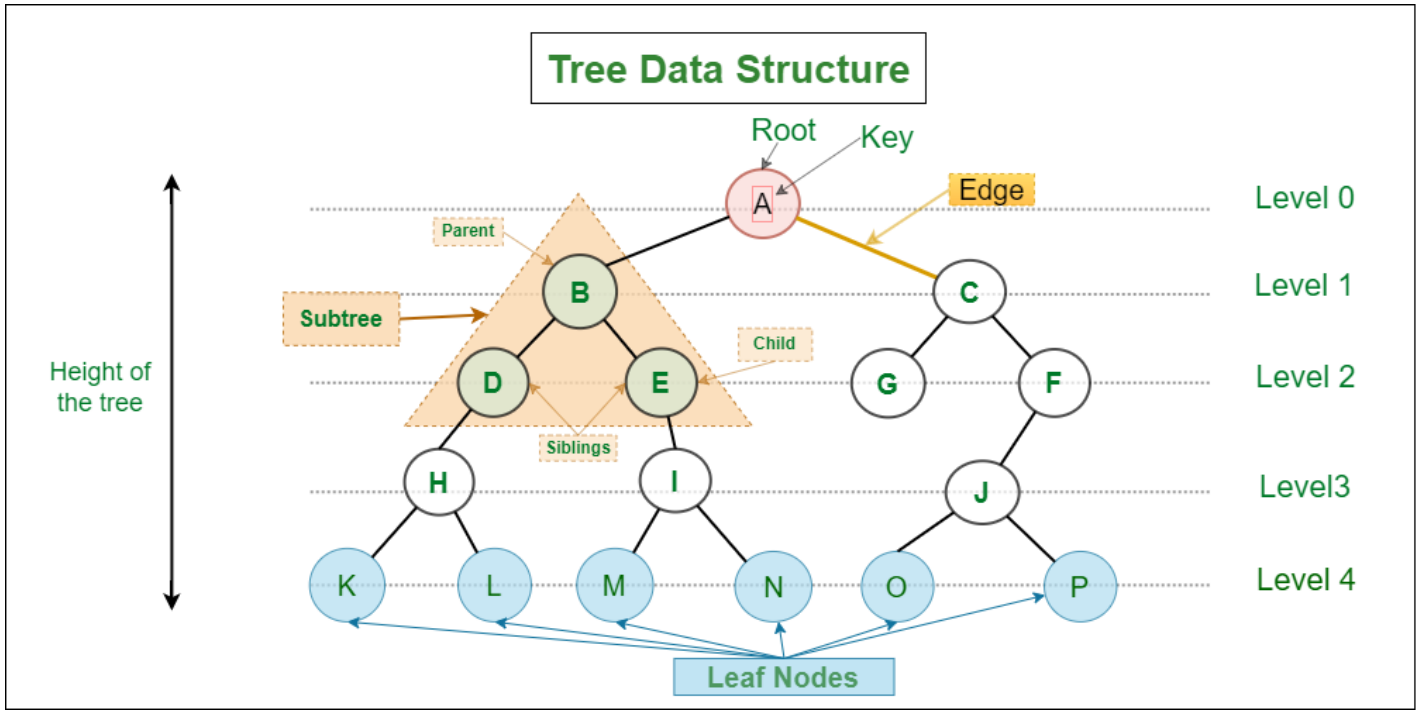
A non-linear data structure where nodes are organized in a hierarchy



In the above figure, where do you think the root of the tree is?

AI VIET NAM
@aivietnam.edu.vn

Basic Terminologies



Parent Node: {B} is the parent node of **{D, E}**.

Child Node: {D, E} are the child nodes of **{B}.**

Root Node: {A} is the root node of the tree

Leaf Node: {K, L, M, N, O, P, G} are the leaf nodes of the tree

Ancestor of a Node: {A,B} are the ancestor nodes of the node **{E}**

Sibling: {D,E} are called siblings.

Level of a node: The count of edges on the path from the root node to that node.

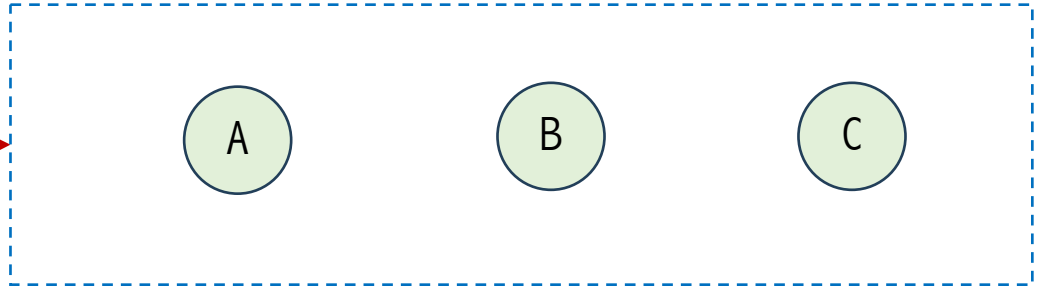
Subtree: Any node of the tree along with its descendant.

Tree Implementation

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []
        self.parent = None
```

Node of a Tree

```
a_node = Node("A")
b_node = Node("B")
c_node = Node("C")
```



Create nodes, A, B, and C

AI VIET NAM
@aivietnam.edu.vn

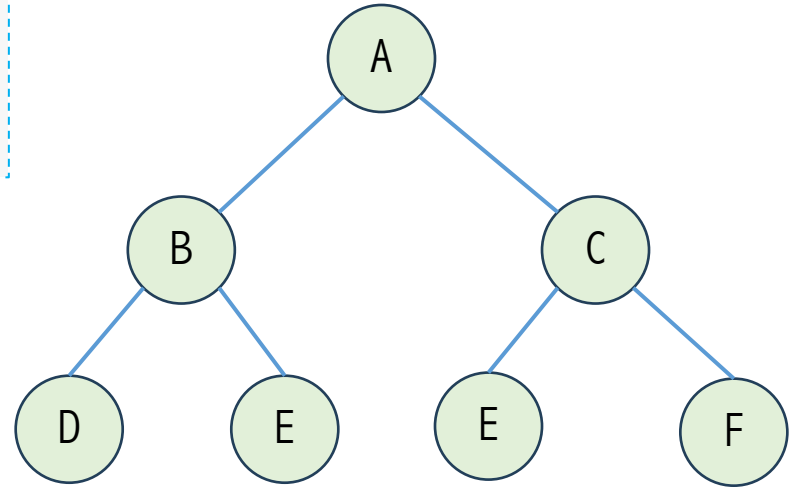
Tree Implementation

```python
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []
        self.parent = None

    def add_child(self, child):
        child.parent = self
        self.children.append(child)

    def get_level(self): #to get level of each node
        level =0
        p = self.parent
        while p:
            level += 1
            p = p.parent

        return level
    def print_tree(self):
        space = ' ' * self.get_level() * 3
        prefix = space + '|__' if self.parent else ''
        print(prefix + self.data)#add prefix
        if self.children:
            for child in self.children:
                child.print_tree()
```

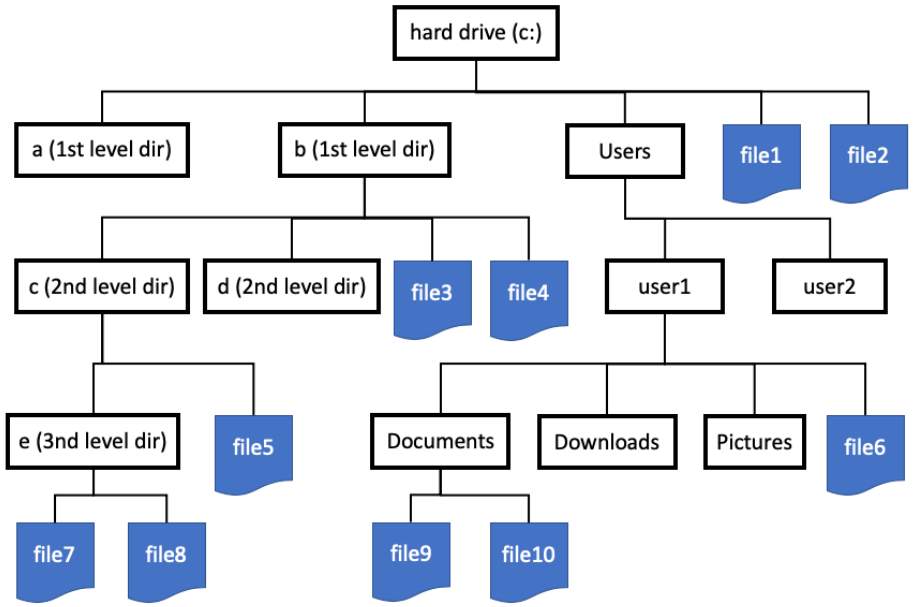
```python
def create_tree():
  a_node = TreeNode("A")
  b_node = TreeNode("B")
  c_node = TreeNode("C")
  d_node = TreeNode("D")
  e_node = TreeNode("E")
  f_node = TreeNode("F")
  g_node = TreeNode("G")

  a_node.add_child(b_node)
  a_node.add_child(c_node)

  b_node.add_child(d_node)
  b_node.add_child(e_node)

  c_node.add_child(f_node)
  c_node.add_child(g_node)
  return a_node
```

```python
tree = create_tree()
tree.print_tree()
```

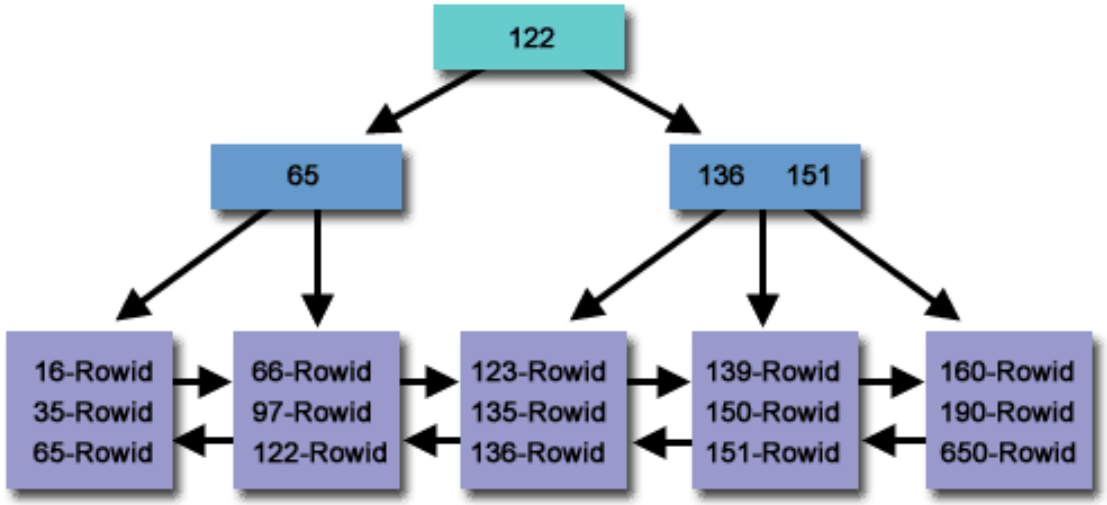
```
A
   |__B
      |__D
      |__E
   |__C
      |__F
      |__G
```



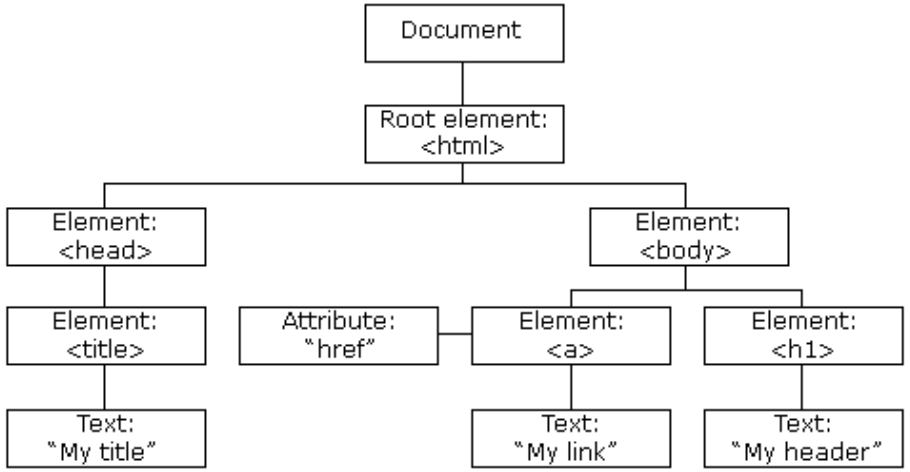
Tree Applications



File explorer



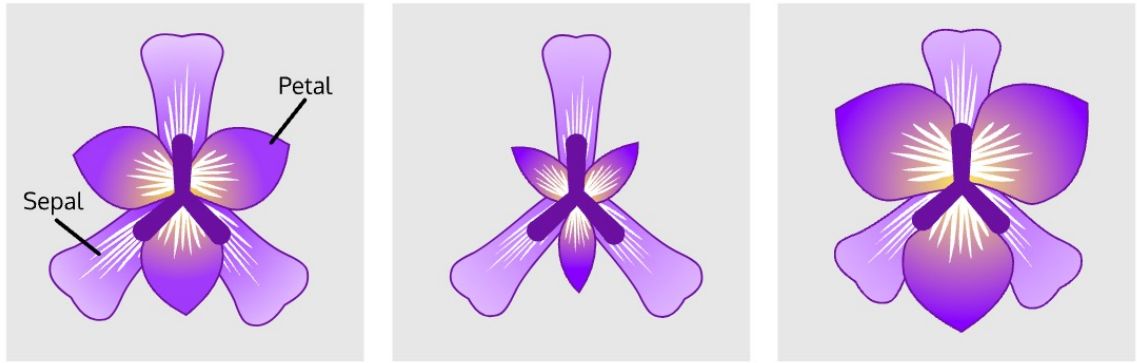
Database



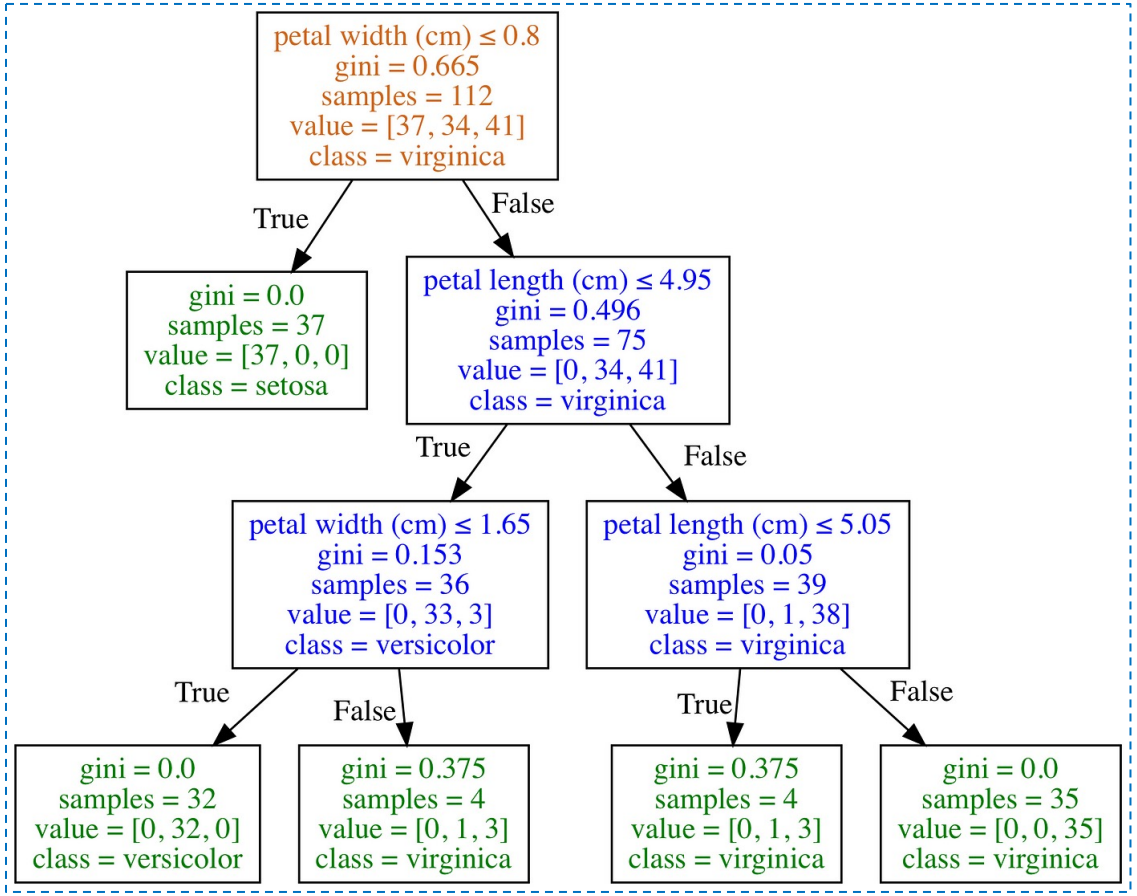
HTML DOM

Tree Applications

ML & AI : Decision Tree



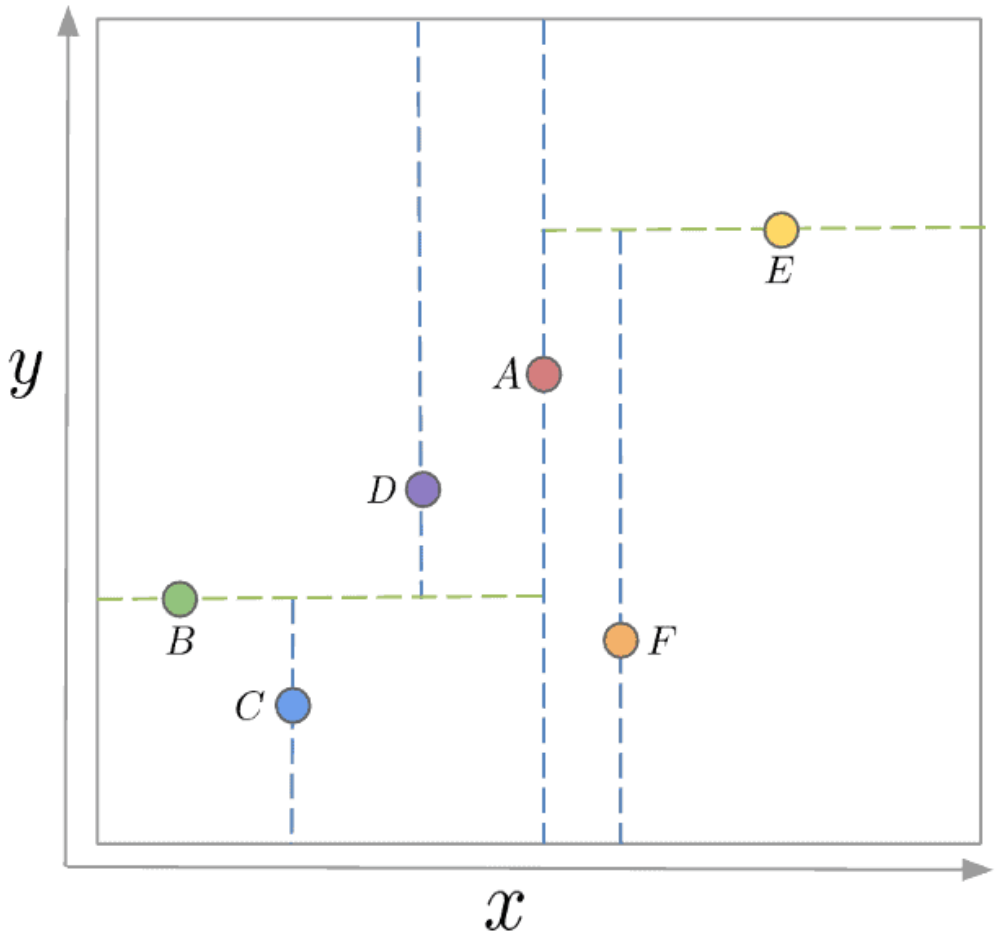
	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

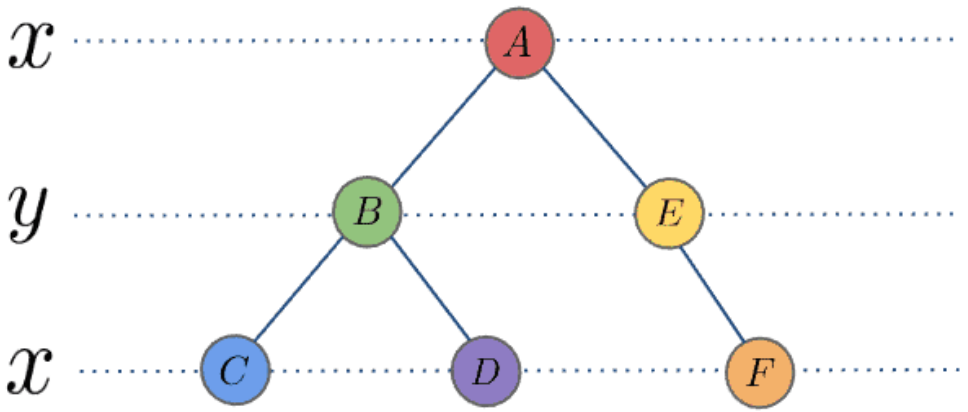


AI VIET NAM
@aivietnam.edu.vn

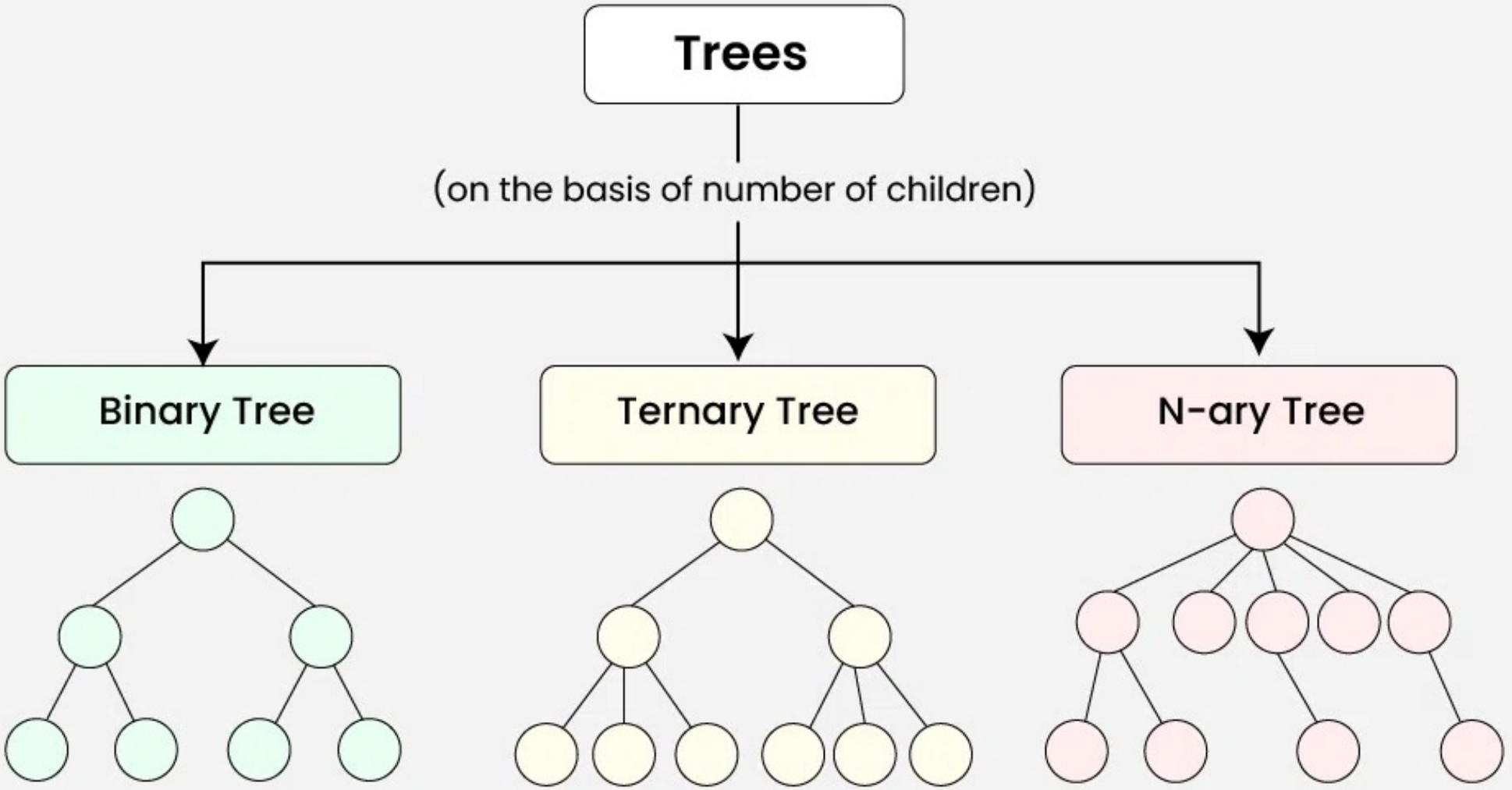
Tree Applications

ML & AI : K-D Tree (K-Dimensional Tree)





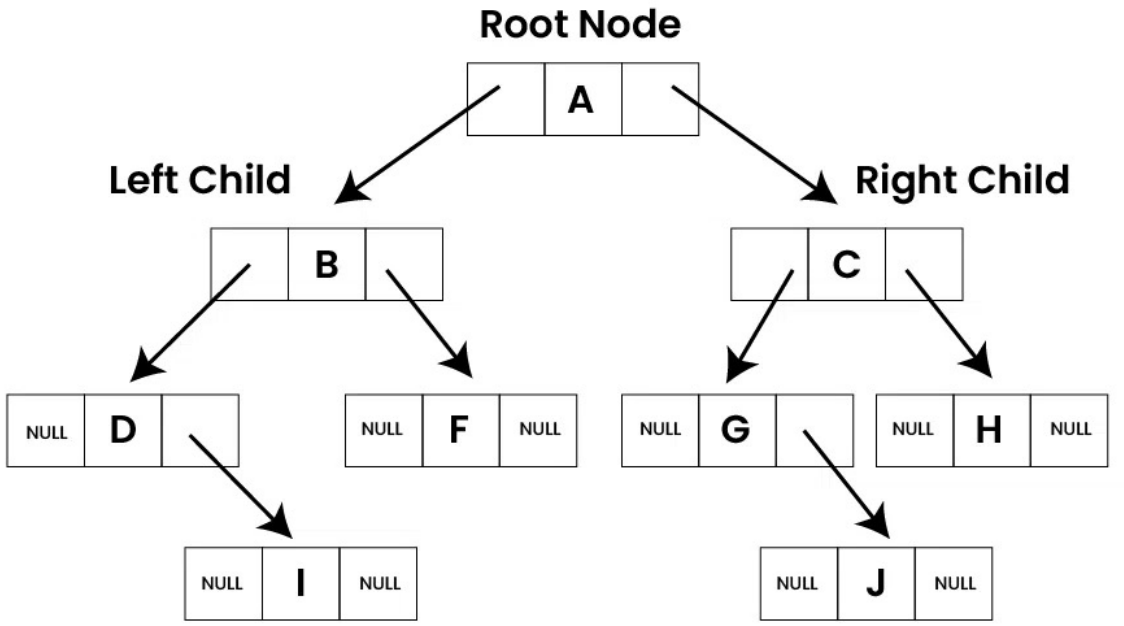
Types of Trees



Outline

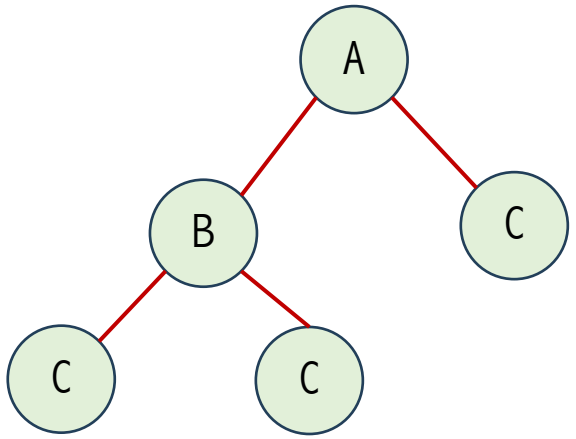
- What is a Tree
- What is a Binary Tree
- Algorithms on Trees
- What is a Binary Search Tree
- What is a K-D Tree
- Summary

Binary Tree

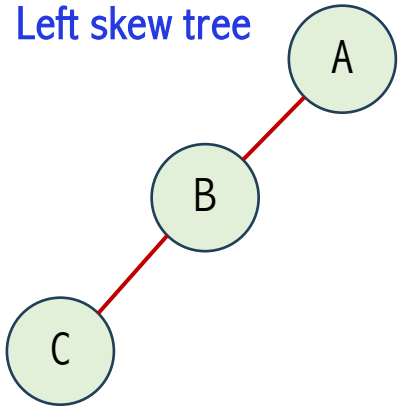


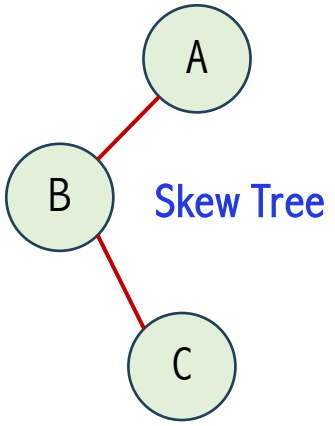
Binary tree is a tree data structure(non-linear) in which each node can have at most two children which are referred to as the left child and the right child

Full binary tree

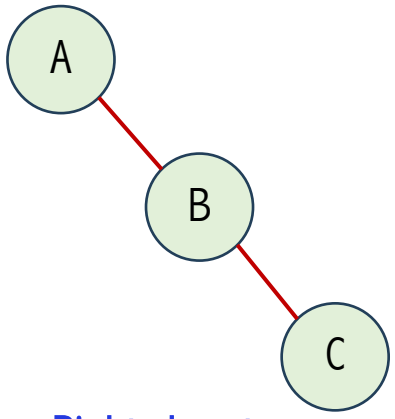


Left skew tree





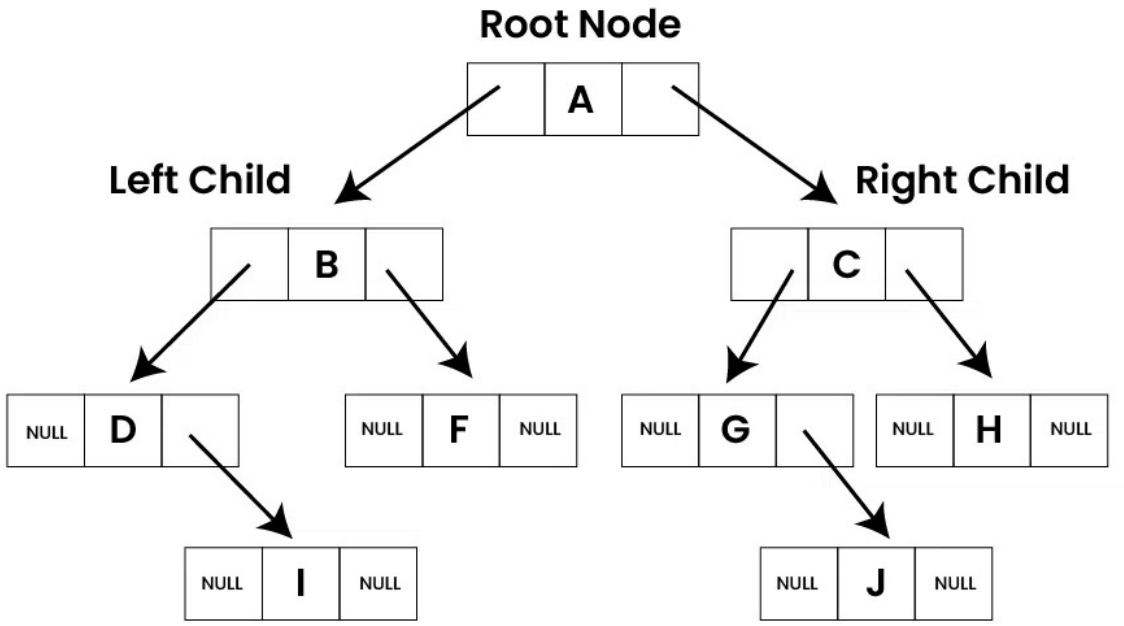
Skew Tree



Right skew tree

AI VIET NAM
@aivietnam.edu.vn

Binary Tree

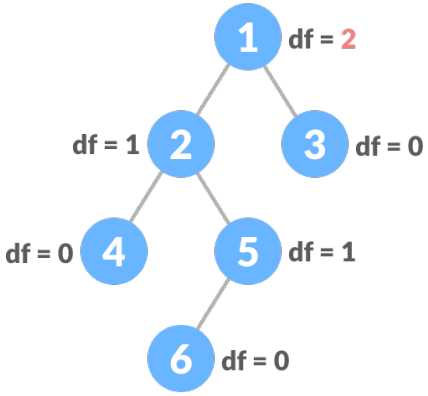


- The difference between the heights of the left and the right subtree for any node is not more than one.
- The left subtree is balanced.
- The right subtree is balanced.

height balance of node = height of right subtree – height of left subtree

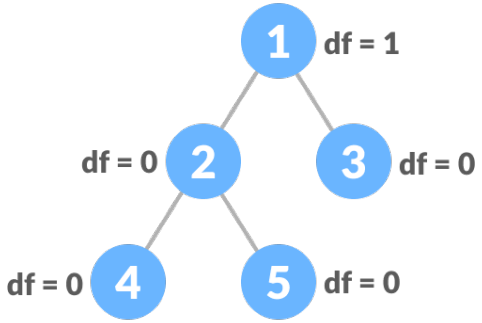
The height of a node in a tree is the length of the longest path from that node downward to a leaf

Unbalanced Binary Tree



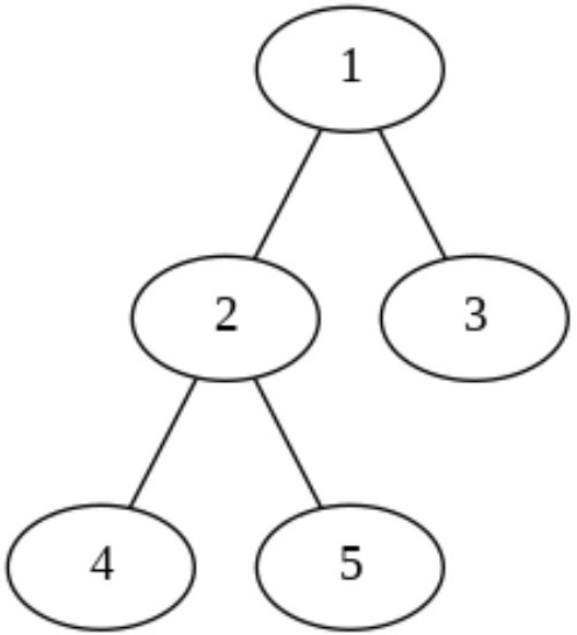
df = |height of left child - height of right child|

Balanced Binary Tree



Operations On Binary Tree

```
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
```




```
# Create the root of the tree
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)

dot = draw_tree(root)
dot.render('orignial_tree', format='png', view=True)
```

Tree visualization

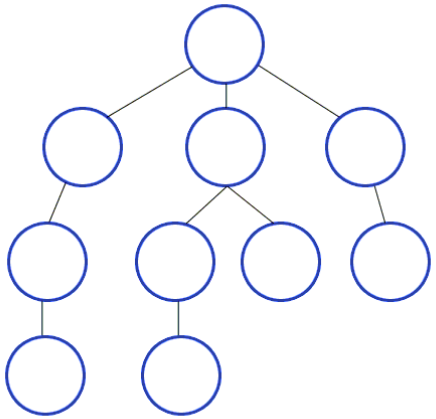
```
def add_edges(dot, node):
    if node is None:
        return
    if node.left:
        dot.edge(str(node.val), str(node.left.val))
        add_edges(dot, node.left)
    if node.right:
        dot.edge(str(node.val), str(node.right.val))
        add_edges(dot, node.right)

def draw_tree(root):
    dot = Graph()
    dot.node(str(root.val))
    add_edges(dot, root)
    return dot
```

Outline

- What is a Tree
- What is a Binary Tree
- Algorithms on Trees
- What is a Binary Search Tree
- What is a K-D Tree
- Summary

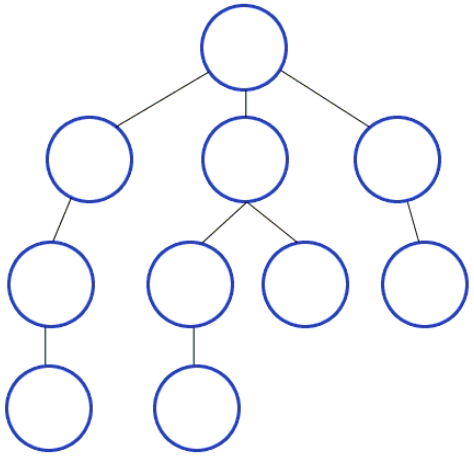
Traversal of Binary Tree

Traversal on a tree refers to the process of visiting (checking and/or updating) each node in a tree data structure, exactly once, in a systematic way



Breadth First Search

Explores all the nodes in a tree at the current depth before moving on to the vertices at the next depth level.



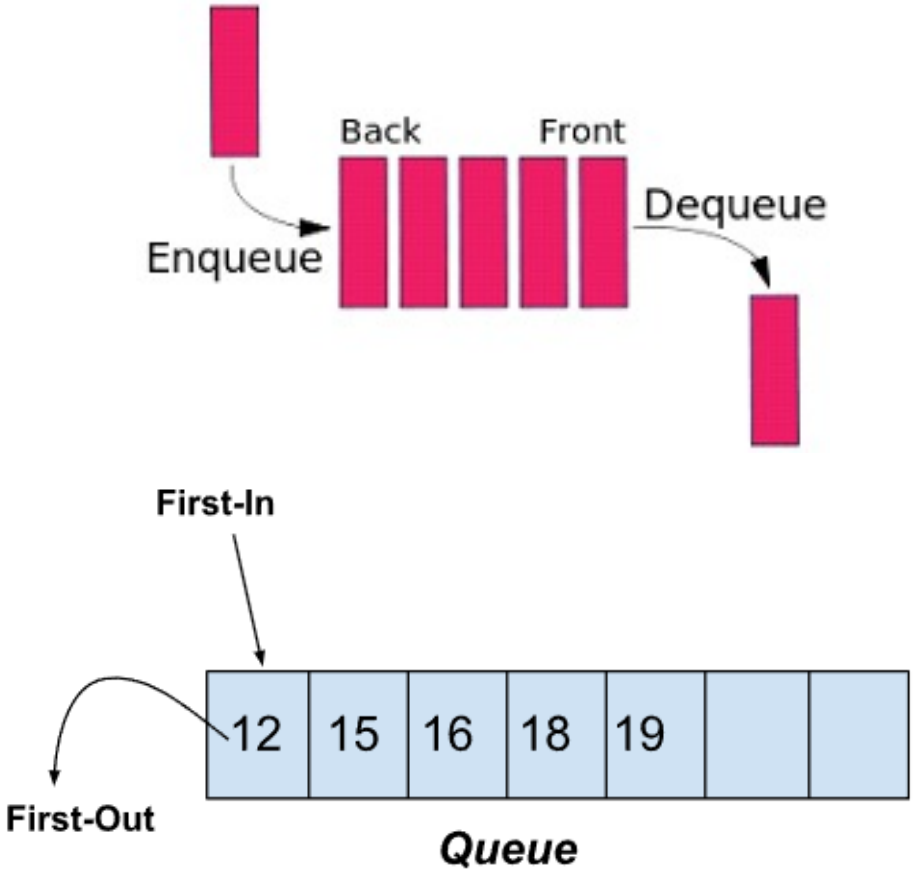
Depth First Search

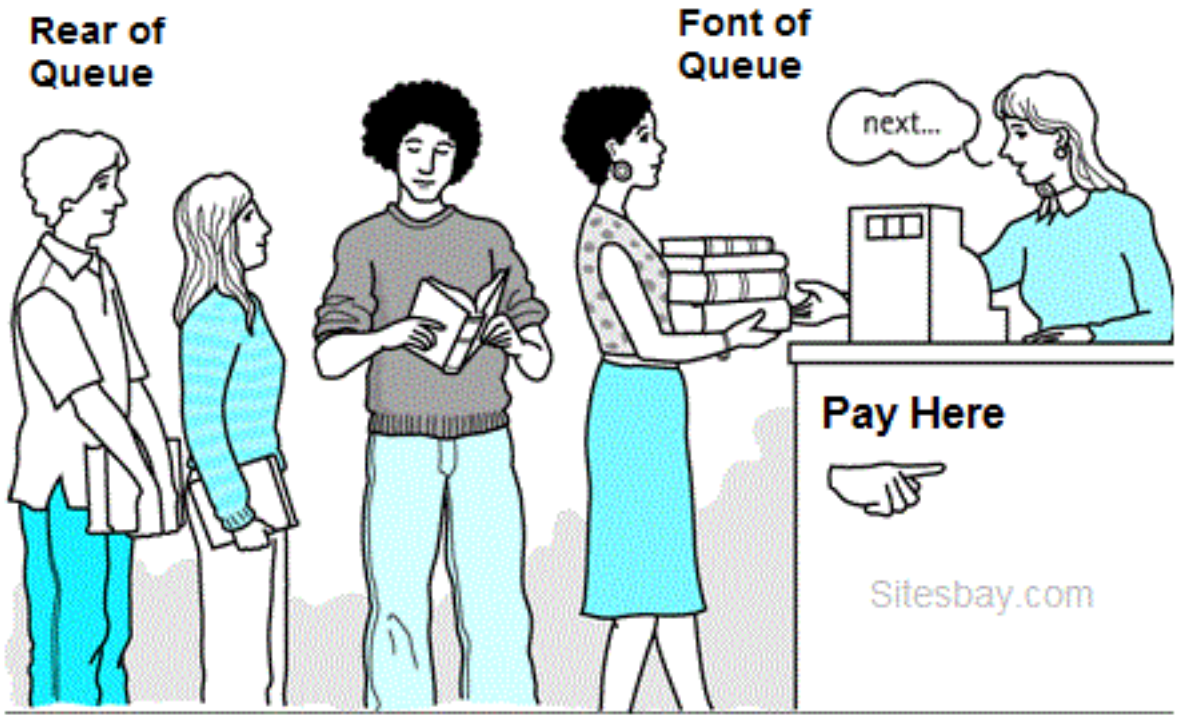
The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking

AI VIET NAM
@aivietnam.edu.vn

Queue

A queue is a linear data structure that follows the **First-In-First-Out (FIFO)** principle. It operates like a line where elements are added at one end (**rear**) and removed from the other end (**front**).





Real Life Example of Queue : Library Counter

Queue Implementation

❖ Using List

```
queue = []
queue.append('a')
queue.append('b')
queue.append('c')
print("Initial queue")
print(queue)
print("\nElements dequeued from queue")
print(queue.pop(0))
print(queue.pop(0))
print(queue.pop(0))
print("\nQueue after removing elements")
print(queue)
```

❖ Using deque

```
from collections import deque
q = deque()
q.append('a')
q.append('b')
q.append('c')
print("Initial queue")
print(q)
print("\nElements dequeued from the queue")
print(q.popleft())
print(q.popleft())
print(q.popleft())

print("\nQueue after removing elements")
print(q)
```

❖ Using Queue

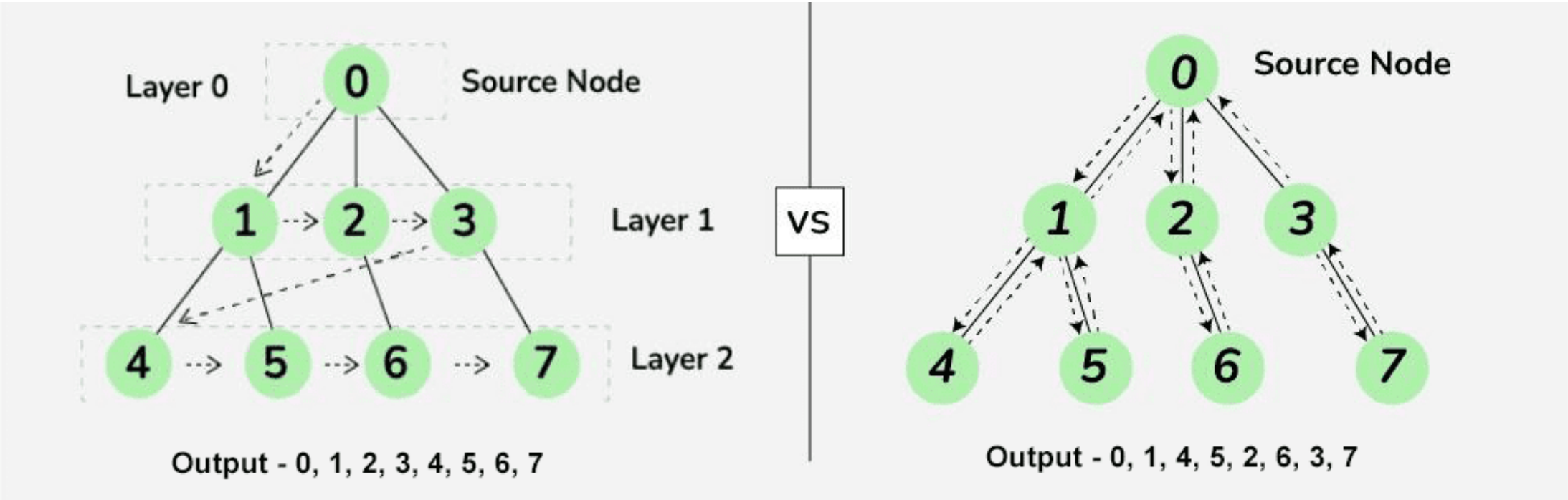
```
from queue import Queue
q = Queue(maxsize = 3)
print(q.qsize())
q.put('a')
q.put('b')
q.put('c')
print("\nFull: ", q.full())
print("\nElements dequeued from the queue")
print(q.get())
print(q.get())
print(q.get())
print("\nEmpty: ", q.empty())
q.put(1)
print("\nEmpty: ", q.empty())
print("Full: ", q.full())
```

Results

```
Initial queue
['a', 'b', 'c']

Elements dequeued from queue
a
b
c

Queue after removing elements
[]
```

Traversal of Binary Tree

Traversal of Binary Tree involves visiting all the nodes of the binary tree. Tree Traversal algorithms can be classified broadly into two categories:



Breadth First Search

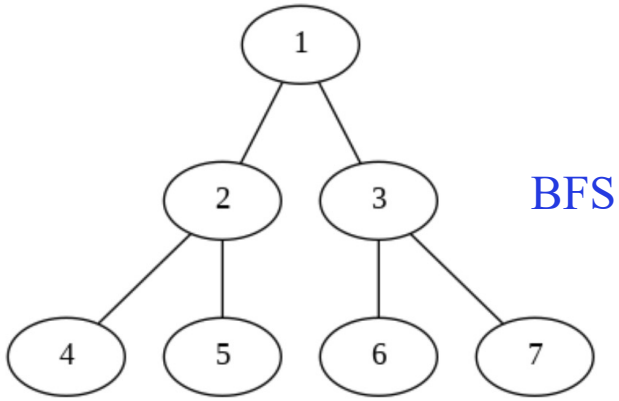
Depth First Search

AI VIET NAM
@aivietnam.edu.vn

Breadth First Search

```python
from collections import deque

def bfs(root):
    if root is None:
        return []

    result = []
    queue = deque([root])

    while queue:
        node = queue.popleft()
        result.append(node.val)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    return result
```

```python
if __name__ == '__main__':
    # Create the root of the tree
    root = TreeNode(1)
    root.left = TreeNode(2)
    root.right = TreeNode(3)
    root.left.left = TreeNode(4)
    root.left.right = TreeNode(5)
    root.right.left = TreeNode(6)
    root.right.right = TreeNode(7)

    # Perform BFS
    bfs_result = bfs(root)
    print("BFS traversal result:", bfs_result)
```

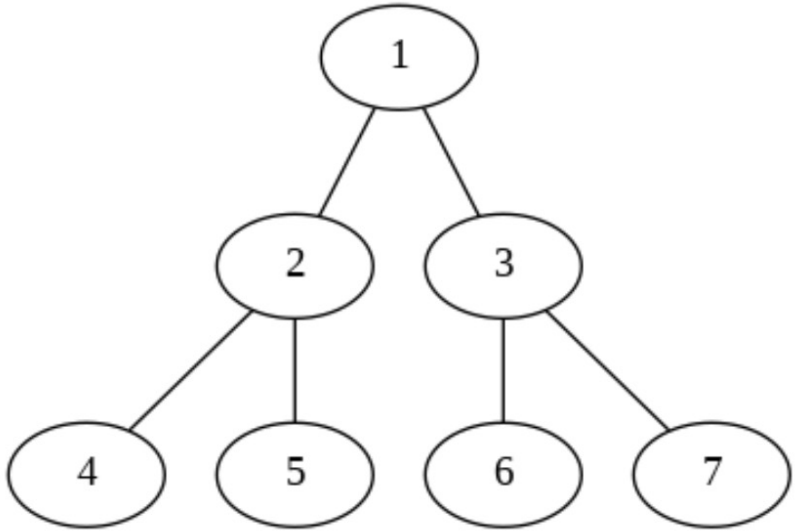


BFS traversal result: [1, 2, 3, 4, 5, 6, 7]

Depth First Search

```python
def dfs_recursive(node):
    if node is None:
        return []

    result = []
    result.append(node.val)
    result.extend(dfs_recursive(node.left))
    result.extend(dfs_recursive(node.right))

    return result
```

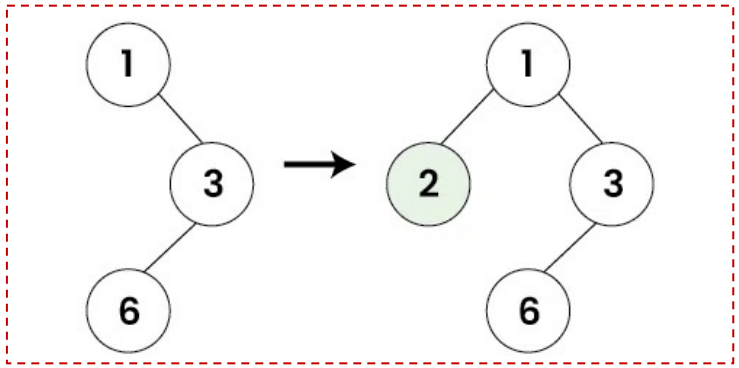
```python
if __name__ == '__main__':
    # Create the root of the tree
    root = TreeNode(1)
    root.left = TreeNode(2)
    root.right = TreeNode(3)
    root.left.left = TreeNode(4)
    root.left.right = TreeNode(5)
    root.right.left = TreeNode(6)
    root.right.right = TreeNode(7)

    # Perform DFS using recursion
    dfs_recursive_result = dfs_recursive(root)
    print("DFS (recursive) traversal result:", dfs_recursive_result)
```



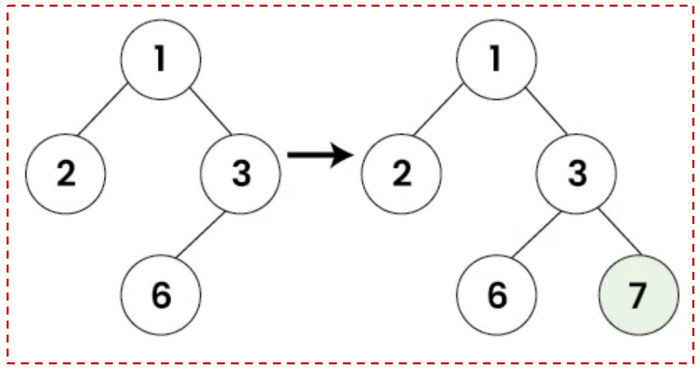
DFS (recursive) traversal result:
[1, 2, 4, 5, 3, 6, 7]

Operations On Binary Tree

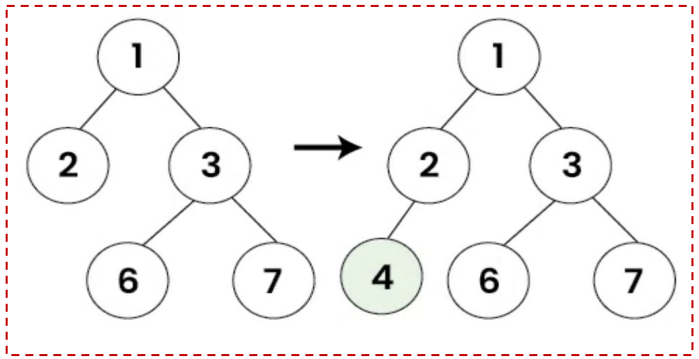
Insert a node in a Binary Tree



Node 2 is inserted to the left of Node 1 as it has a missing left child



Node 7 is inserted to the right of Node 3 as it has a missing left child



Node 4 is inserted to the right of Node 2 as it has a missing left child

Insert a Node

```python
def insert_node(root, key):
    # Create a new node
    new_node = TreeNode(key)

    # If the tree is empty, the new node becomes the root
    if root is None:
        return new_node

    # Use a queue to perform level-order traversal
    queue = []
    queue.append(root)

    # Traverse the tree
    while queue:
        # Dequeue a node
        temp = queue.pop(0)

        # Check if the left child is empty
        if temp.left is None:
            temp.left = new_node
            return root
        else:
            queue.append(temp.left)

        # Check if the right child is empty
        if temp.right is None:
            temp.right = new_node
            return root
        else:
            queue.append(temp.right)

    return root
```

Algorithm

1. Initialize:

- Create a new node with the given value.
- If the tree is empty, the new node becomes the root of the tree

.

2. Level-Order Traversal:

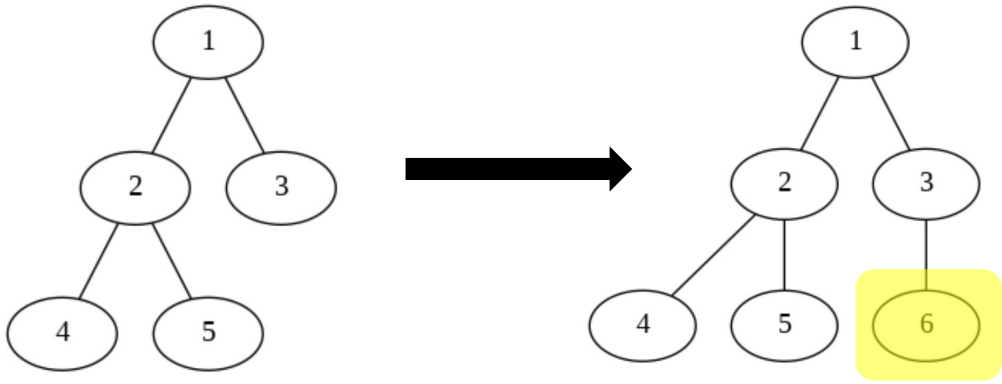
- Use a queue to perform a level-order traversal (BFS).
- Start by enqueueing the root node.

3. Traversal Loop:

- While the queue is not empty:
  - Dequeue a node from the front of the queue.
  - Check if the dequeued node has a left child:
    - If not, set the left child to the new node and return.
    - If yes, enqueue the left child to the queue.
  - Check if the dequeued node has a right child:
    - If not, set the right child to the new node and return.
    - If yes, enqueue the right child to the queue.

Insert a Node

```python
# Driver code
if __name__ == '__main__':
    # Create the root of the tree
    root = TreeNode(1)
    root.left = TreeNode(2)
    root.right = TreeNode(3)
    root.left.left = TreeNode(4)
    root.left.right = TreeNode(5)

    dot = draw_tree(root)
    dot.render('orignial_tree', format='png', view=True)

    key = 6
    print(f"\nInserting {key} into the binary tree.")
    root = insert_node(root, key)

    dot = draw_tree(root)
    dot.render('after_tree', format='png', view=True)
```

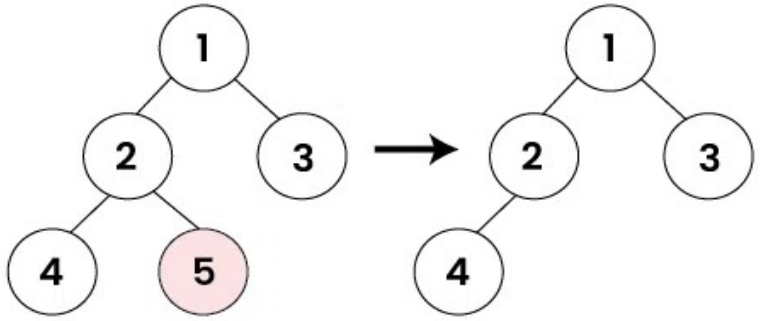


Visualization using `graphviz`

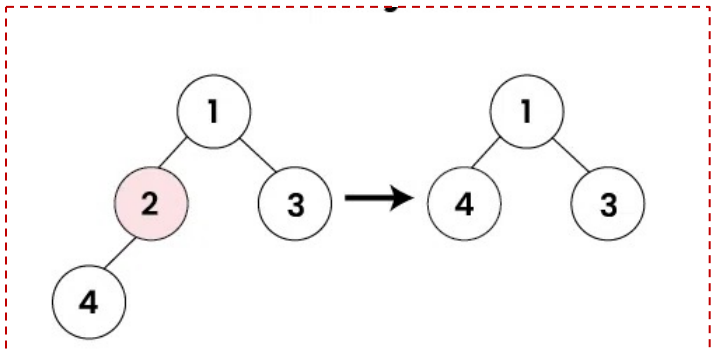
```python
def add_edges(dot, node):
    if node is None:
        return
    if node.left:
        dot.edge(str(node.val), str(node.left.val))
        add_edges(dot, node.left)
    if node.right:
        dot.edge(str(node.val), str(node.right.val))
        add_edges(dot, node.right)

def draw_tree(root):
    dot = Graph()
    dot.node(str(root.val))
    add_edges(dot, root)
    return dot
```

Operations On Binary Tree

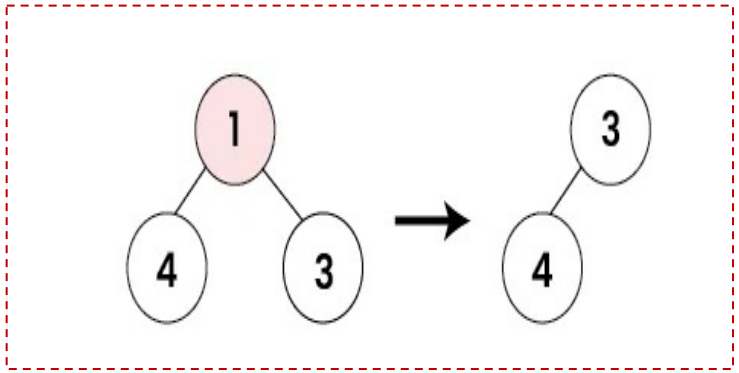
Delete a node in a Binary Tree



Leaf nodes can be deleted without any shifting of nodes



Node 2 is deleted and replaced with the deepest node 4

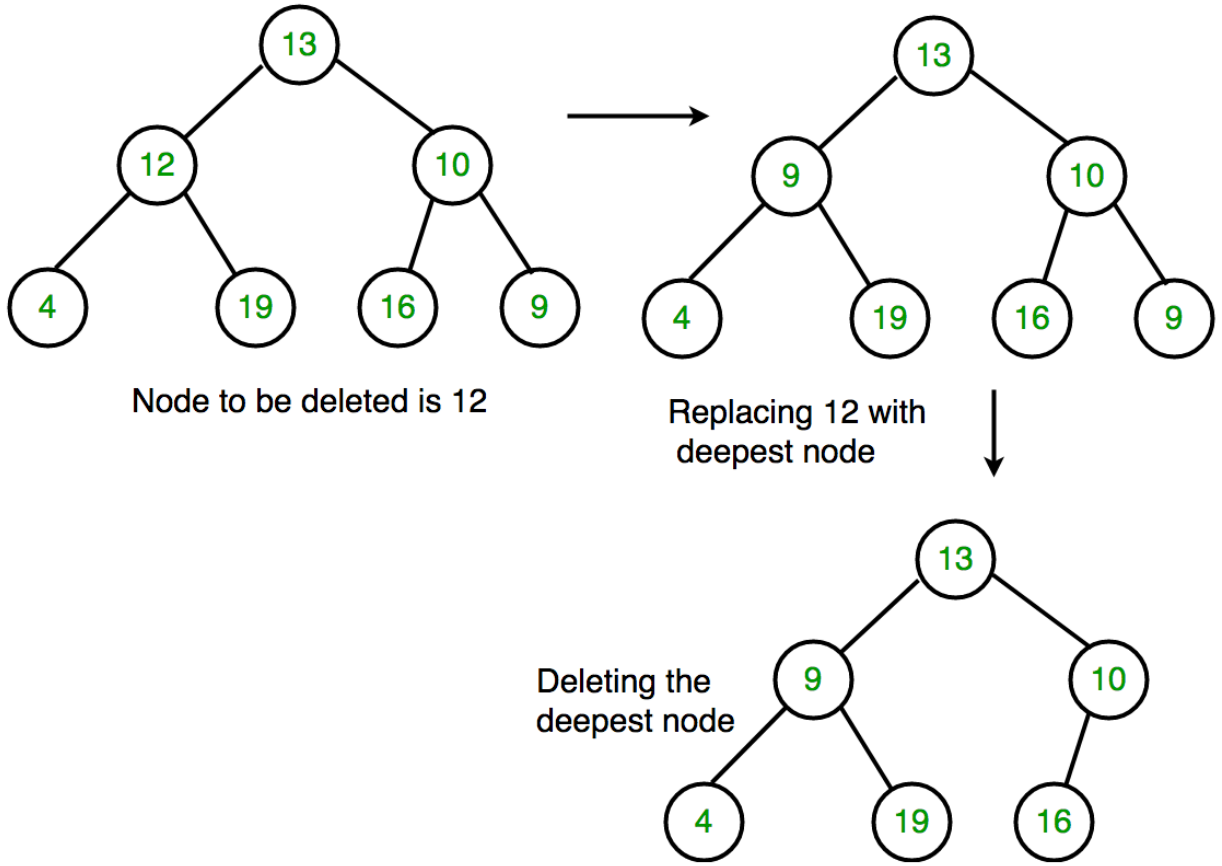


Node 1 is deleted and relaced with deepest node 3

We can delete any node in the binary tree and rearrange the nodes after deletion to again form a valid binary tree.

Operations On Binary Tree

Delete a node in a Binary Tree



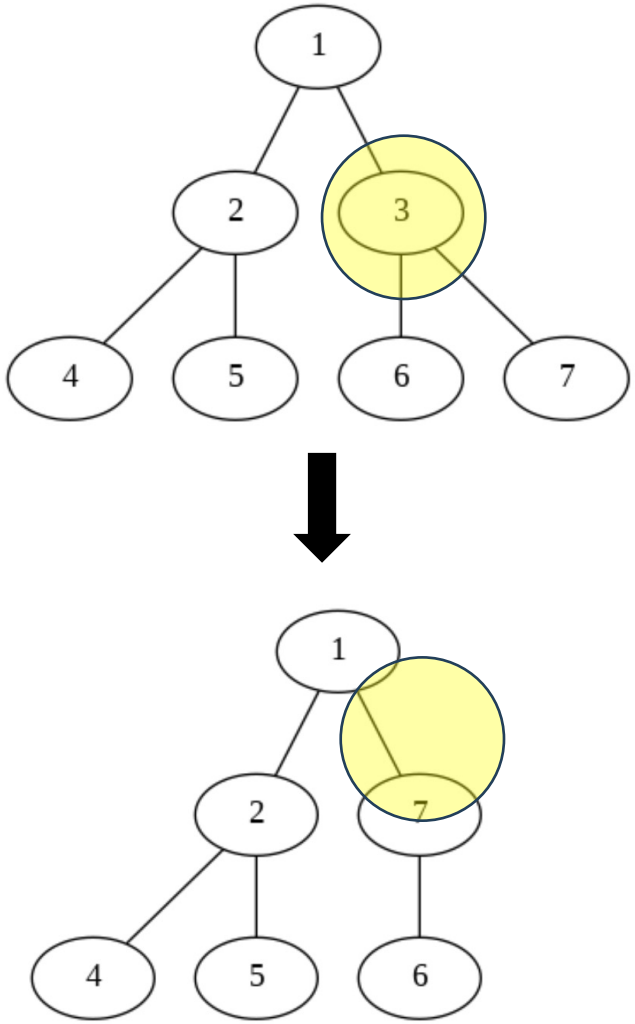
Algorithm to delete a node in a Binary Tree:

- **Find the Node to be Deleted**: Traverse the tree to find the node that needs to be deleted.
- **Replace the Node**: If the node to be deleted is not a leaf node, replace it with the deepest rightmost node in the tree.
- **Delete the Deepest Node**: Remove the deepest rightmost node from the tree.

AI VIET NAM
@aivietnam.edu.vn

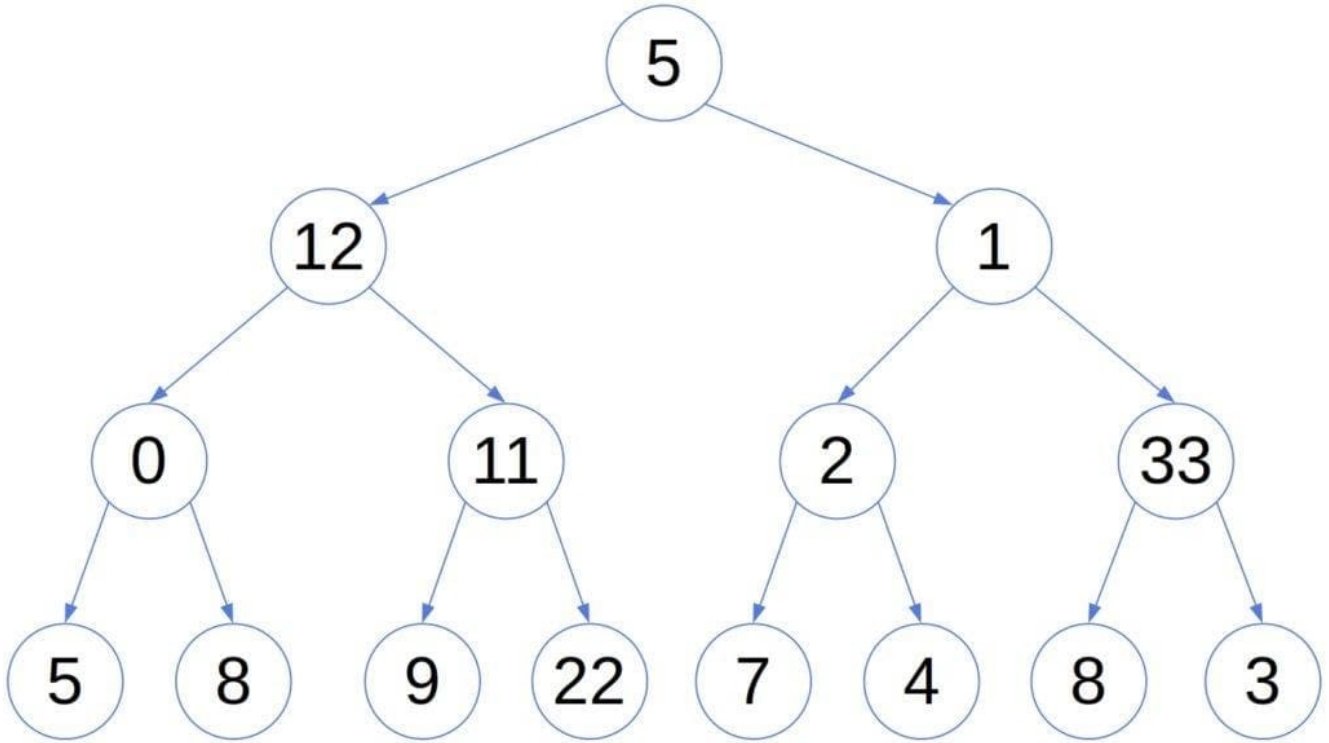
Delete a Node in Binary Tree

Delete node “3” from a Binary Tree




```python
if __name__ == '__main__':
    # Create the root of the tree
    root = TreeNode(1)
    root.left = TreeNode(2)
    root.right = TreeNode(3)
    root.left.left = TreeNode(4)
    root.left.right = TreeNode(5)
    root.right.left = TreeNode(6)
    root.right.right = TreeNode(7)

    dot = visualize_tree(root)
    dot.render('original_delete_tree', format='png', view=True)

    key = 3
    print(f"\nDeleting node with key {key}")
    root = delete_node(root, key)
    dot = visualize_tree(root)
    dot.render('after_delete_tree', format='png', view=True)
```

Binary Tree Limitations



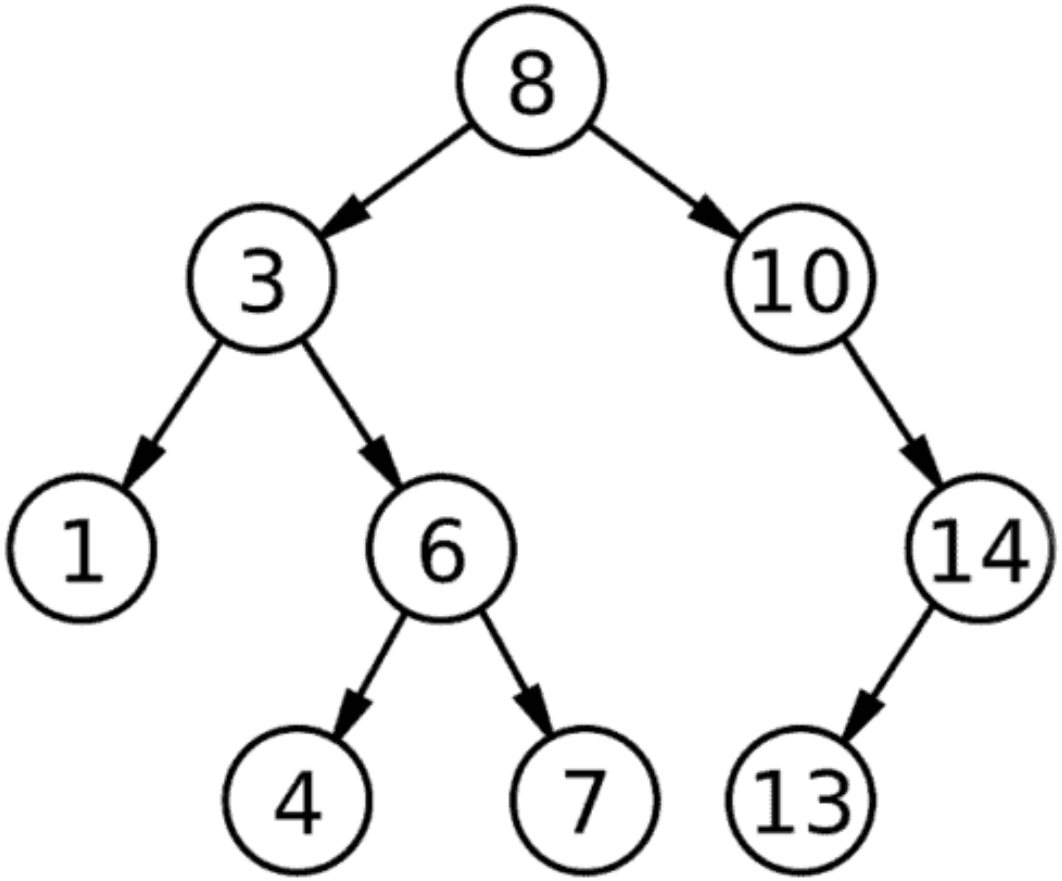
Although suitable for storing hierarchical data, **binary trees of this general form don't guarantee a fast lookup**. Let's take as the example the search for number 9 in the above tree.

Whichever node we visit, we don't know if we should traverse the left or the right sub-tree next. That's because the tree hierarchy doesn't follow the relation

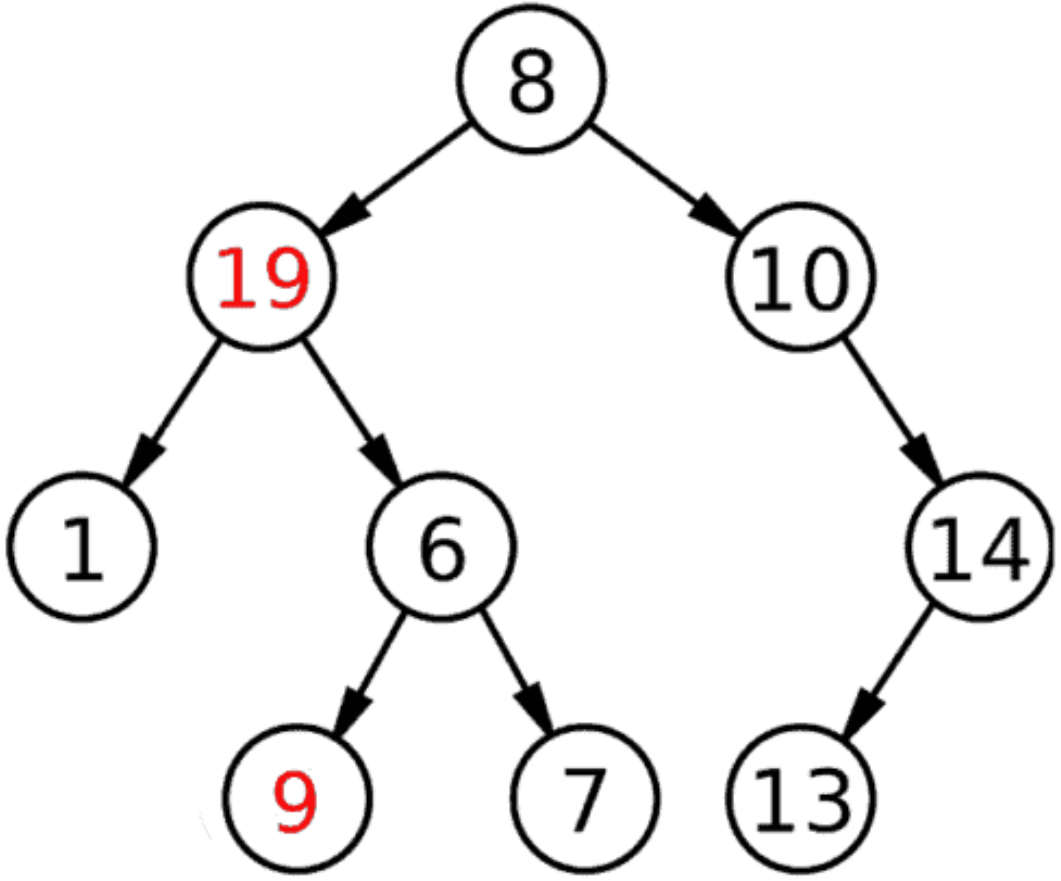
Outline

- What is a Tree
- What is a Binary Tree
- Algorithms on Trees
- What is a Binary Search Tree
- What is a K-D Tree
- Summary

Binary Search Tree



(a)



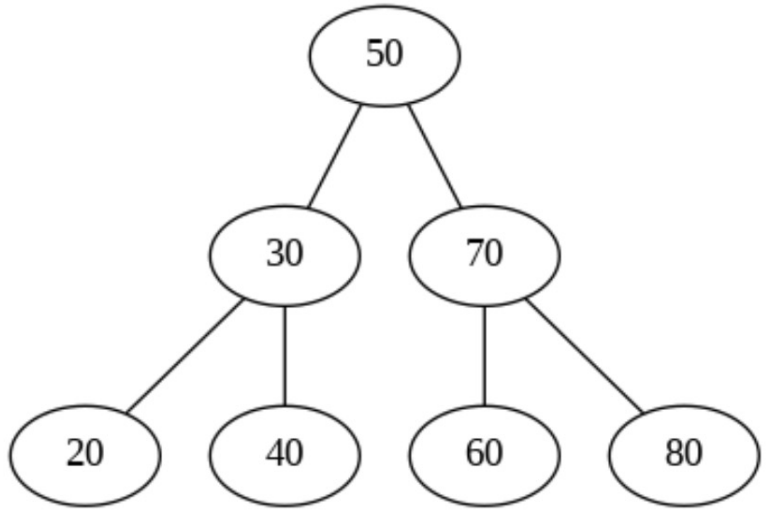
(b)

Which one is the binary search tree? and Why?

Binary Search Tree Implementation

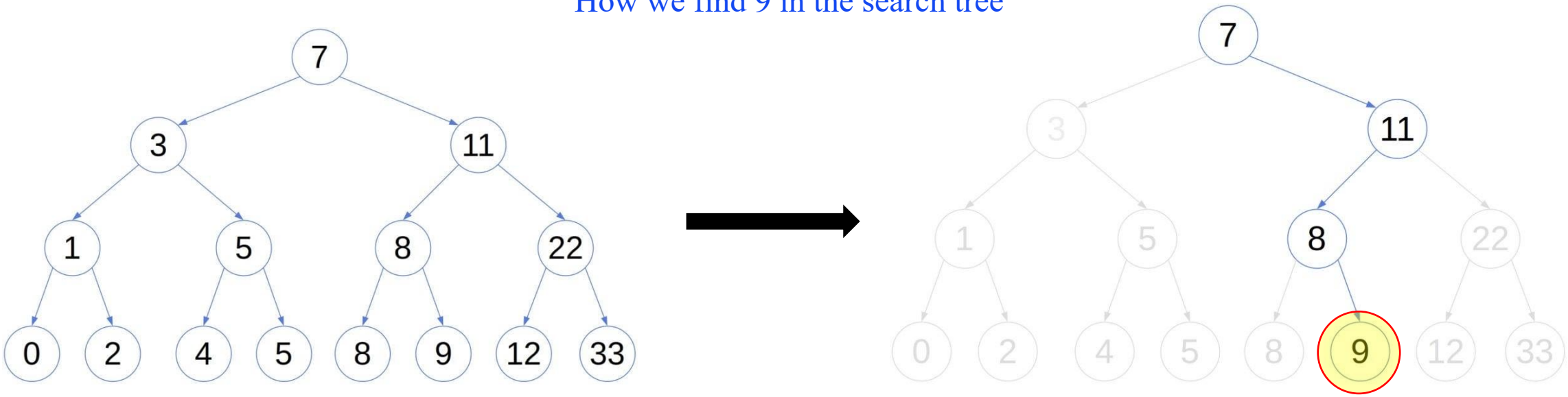
```python
class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert(self.root, key)

    def _insert(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert(node.right, key)
```

```python
if __name__ == '__main__':
    bst = BST()

    # Insert elements
    elements = [50, 30, 20, 40, 70, 60, 80]
    for element in elements:
        bst.insert(element)
```



Binary Search Tree

How we find 9 in the search tree



For each node **x** in a BST, all the nodes in the left sub-tree of **x** contain the values that are strictly lower than **x**. Further, all the nodes in the **x**'s right sub-tree are >= **x**

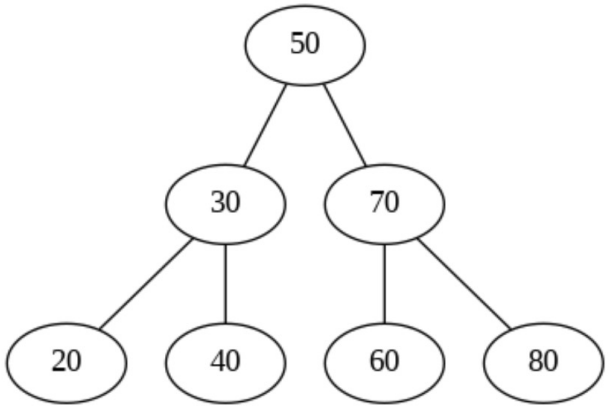
Binary Search Tree

Search algorithm in BST

```
algorithm lookup(tree, key):
    if key = tree.key:
        return true

    if key < tree.key:
        return lookup(tree.leftChild(),key)

    if key > tree.key:
        return lookup(tree.rightChild(), key)
```

```python
def search(self, key):
    return self._search(self.root, key)

def _search(self, node, key):
    if node is None or node.val == key:
        return node

    if key < node.val:
        return self._search(node.left, key)

    return self._search(node.right, key)
```

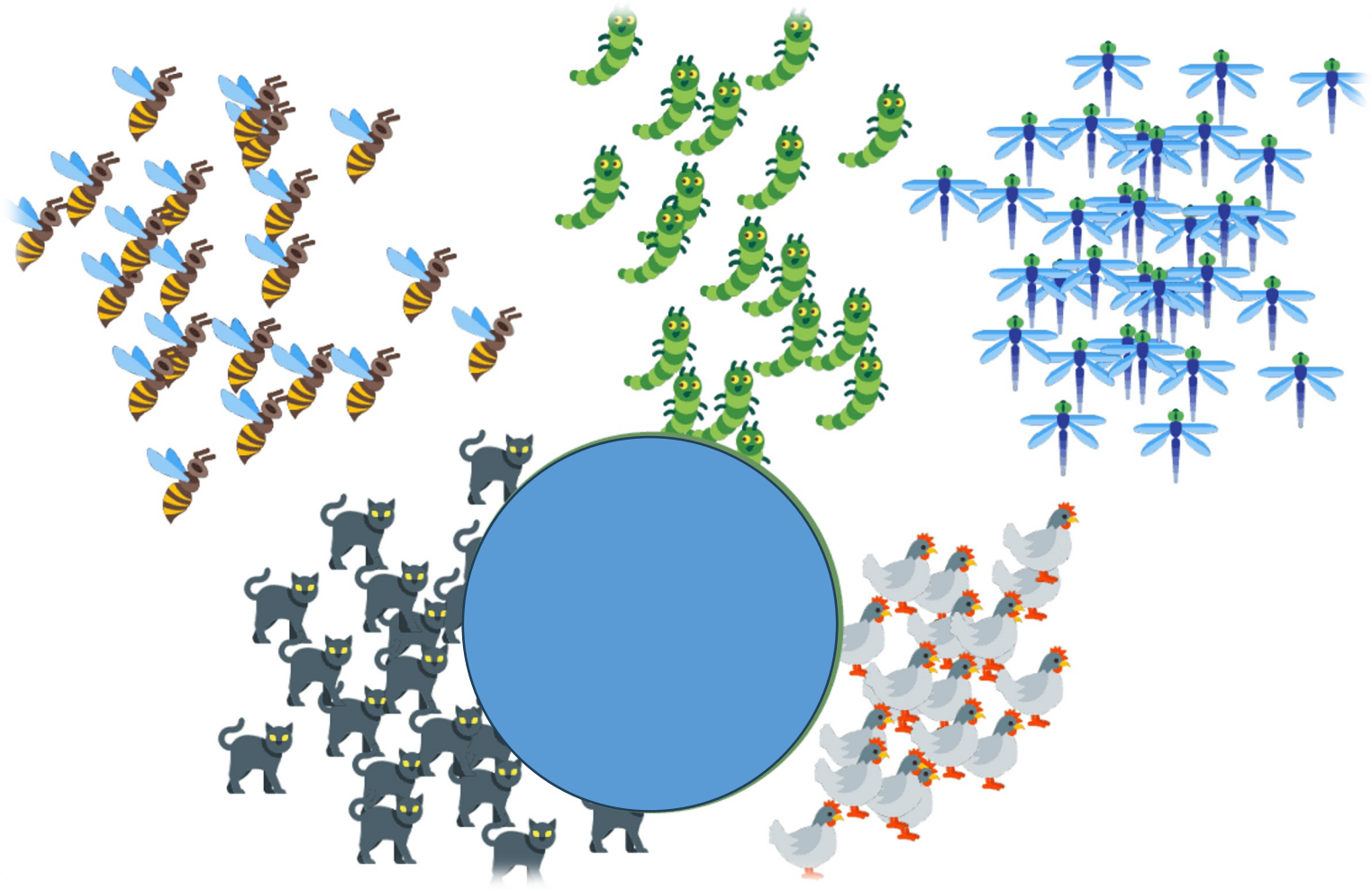
```python
if __name__ == '__main__':
    bst = BST()

    # Insert elements
    elements = [50, 30, 20, 40, 70, 60, 80]
    for element in elements:
        bst.insert(element)


     # Visualize the BST
    dot = visualize_tree(bst.root)
    dot.render('bst', format='png', view=True)

    # Search for elements
    key = 40
    result = bst.search(key)
    if result:
        print(f"Element {key} found in the BST")
    else:
        print(f"Element {key} not found in the BST")
```



Outline

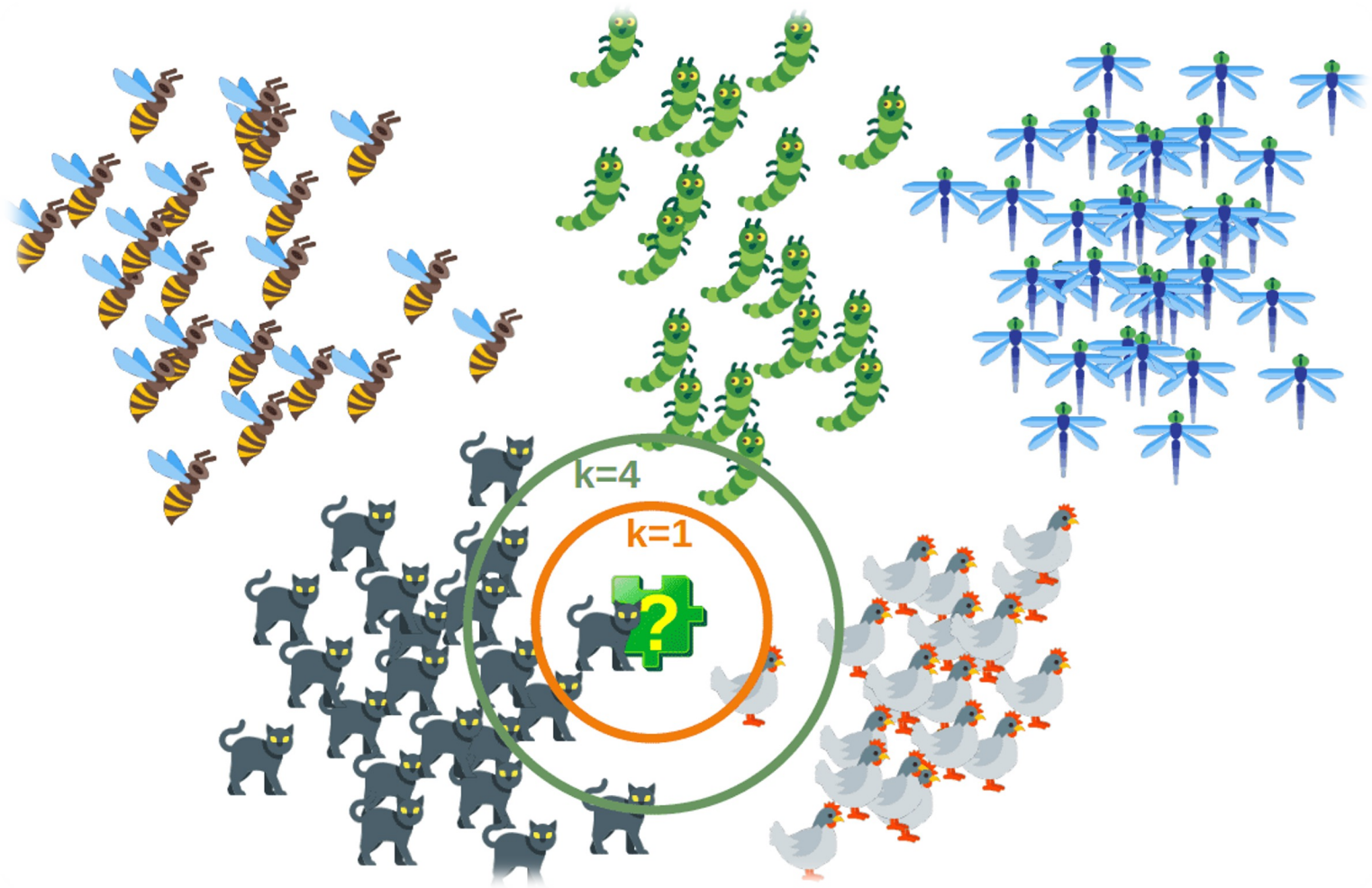
- What is a Tree
- What is a Binary Tree
- Algorithms on Trees
- What is a Binary Search Tree
- What is a K-D Tree
- Summary

Classification Problem



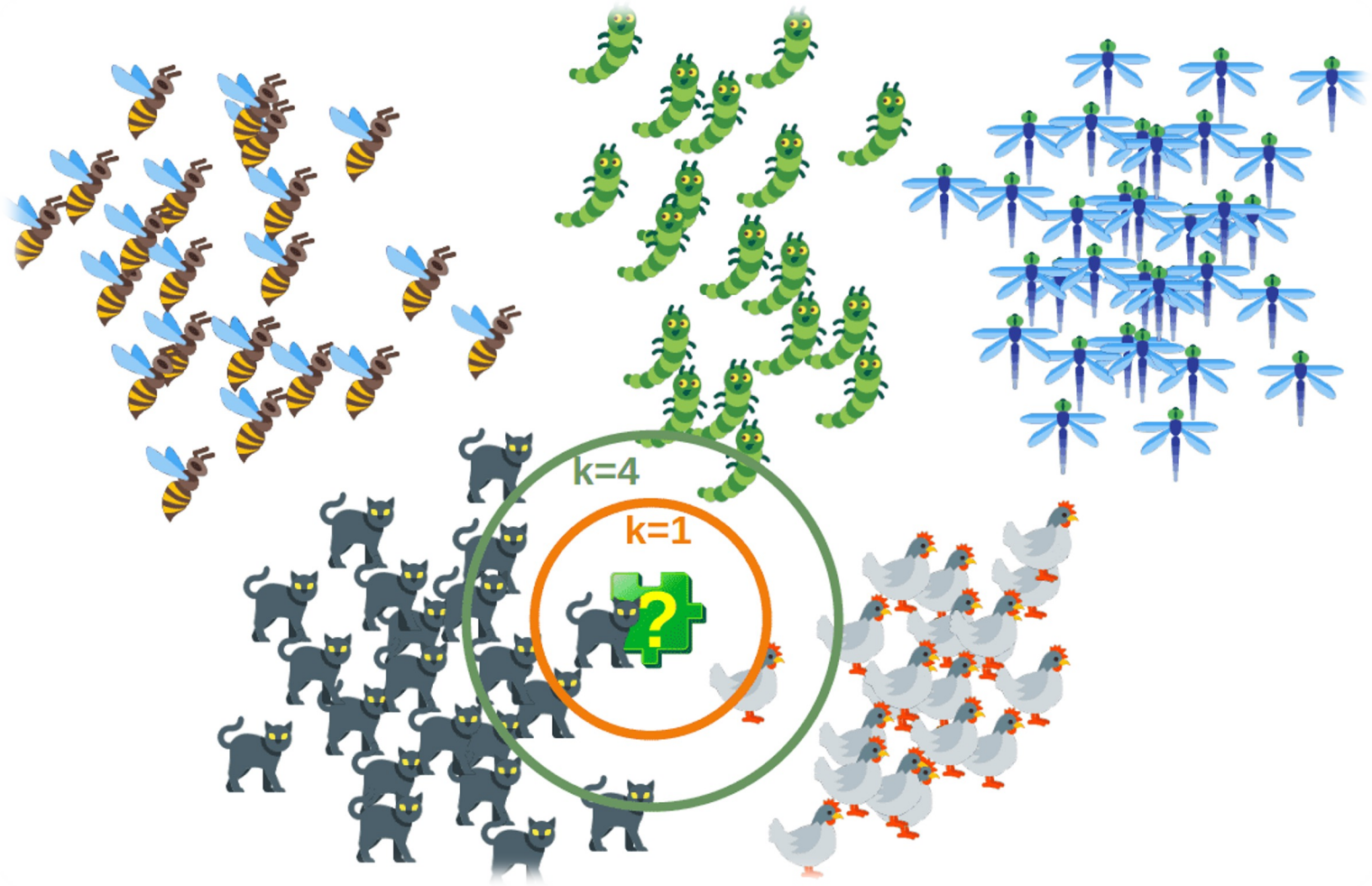
Given dataset with known categories

Classification Problem



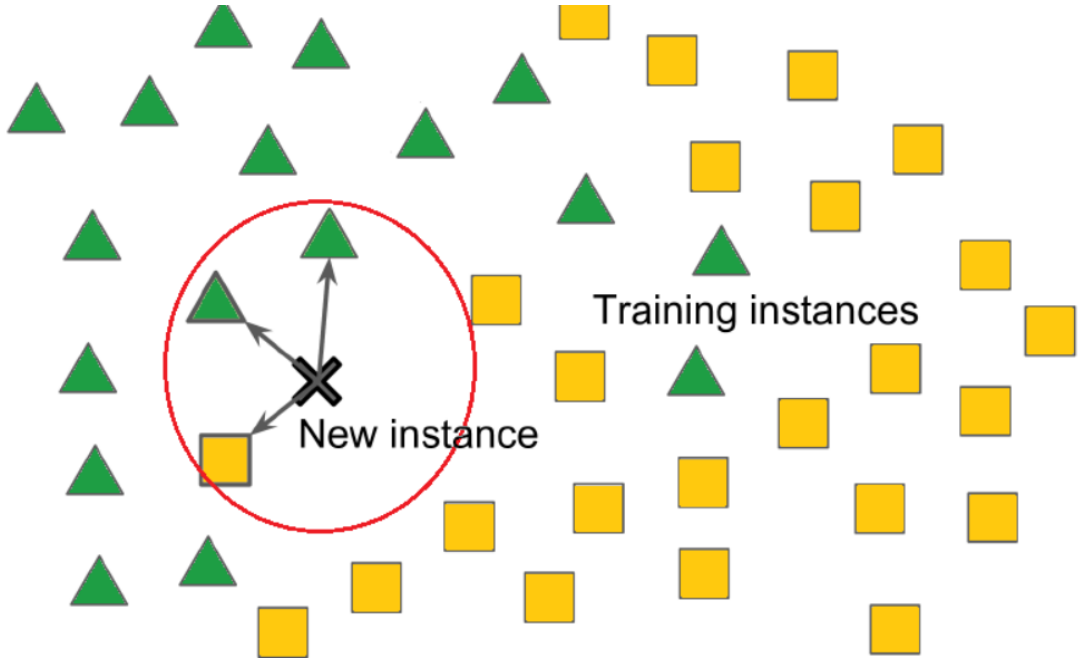
What is a category of a new image?

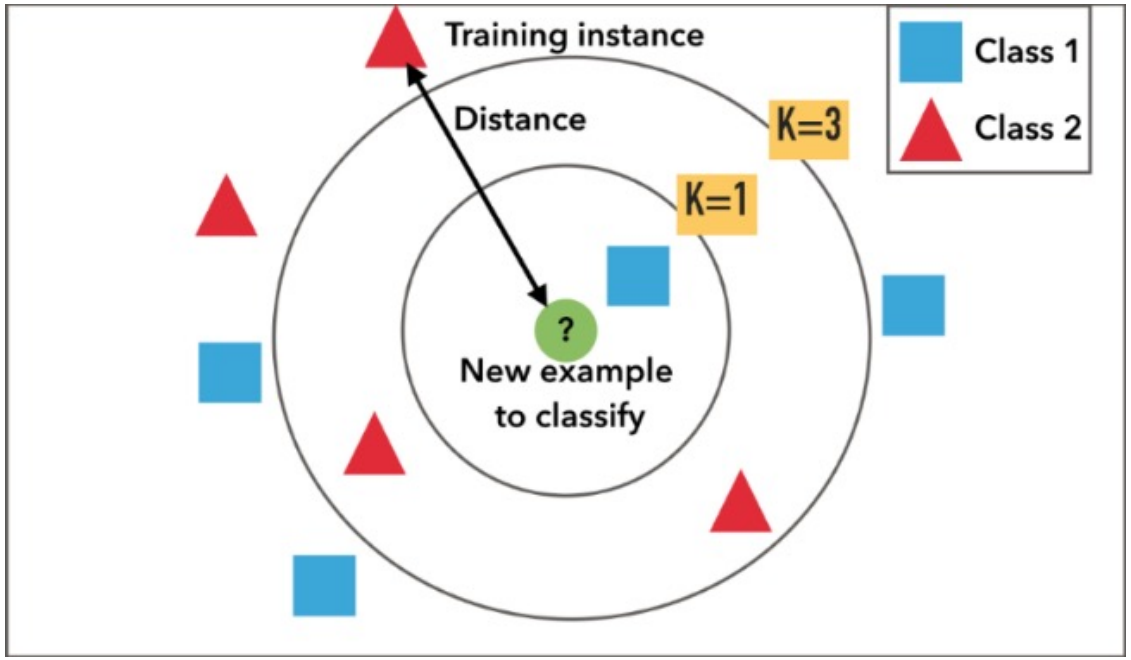
Classification Problem



What is a category of a new image? → **Look for K nearest neighbors**

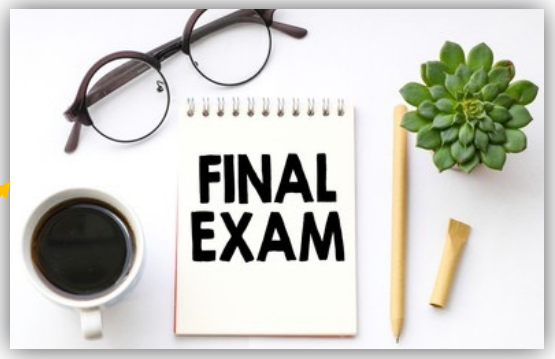
Classification Problem





What is a category of a new image? → **Look for K-nearest neighbors**

Exam Motivation



Study Hard

Laziness

Two students are going to take the final exam

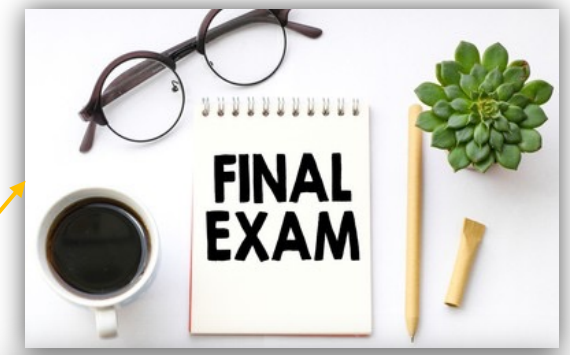
Exam Motivation

Study hard

Two students are get good results

Exam Motivation

Study hard

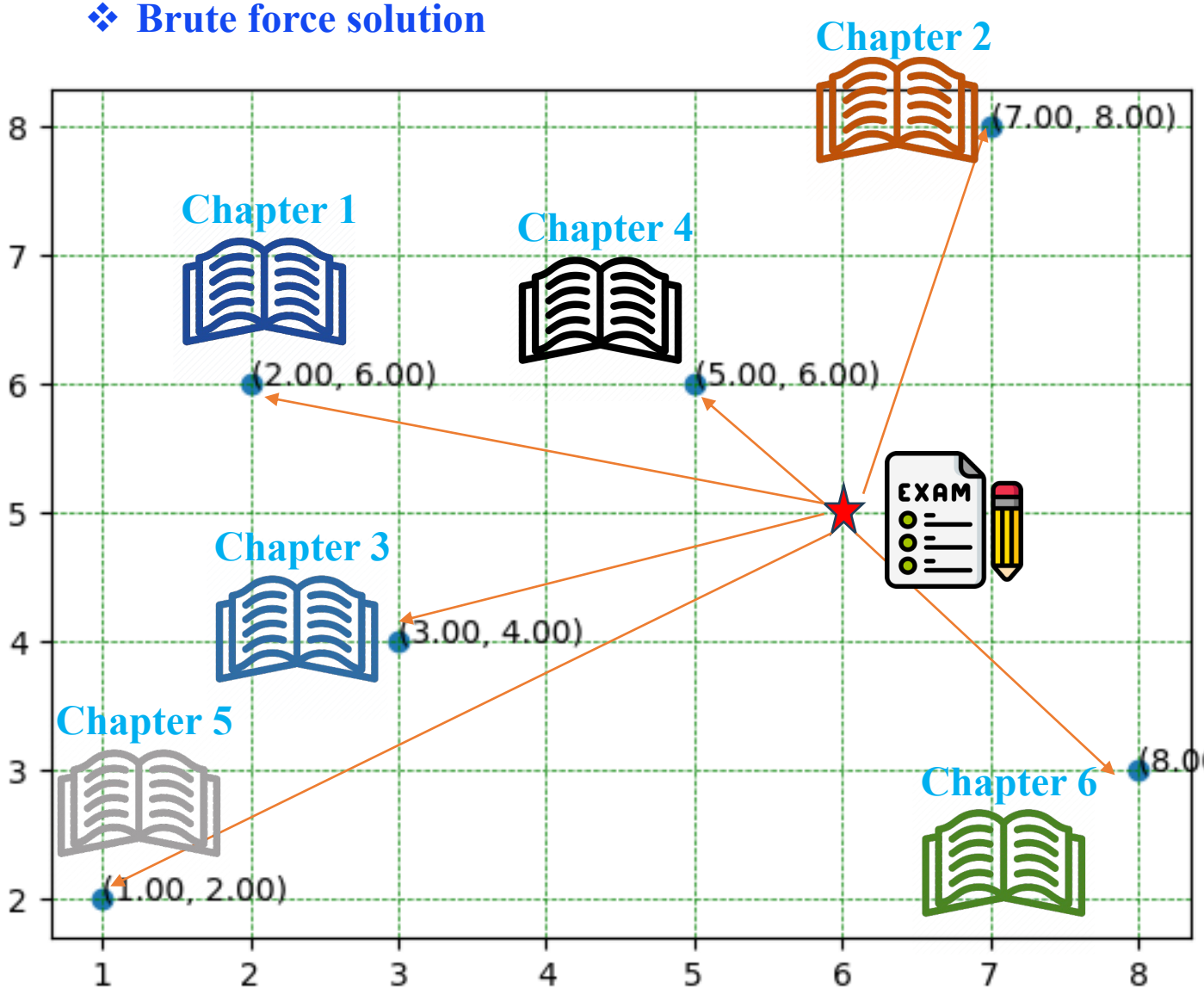


Laziness

What is happening Here?

Classification Problem

❖ **Brute force solution**



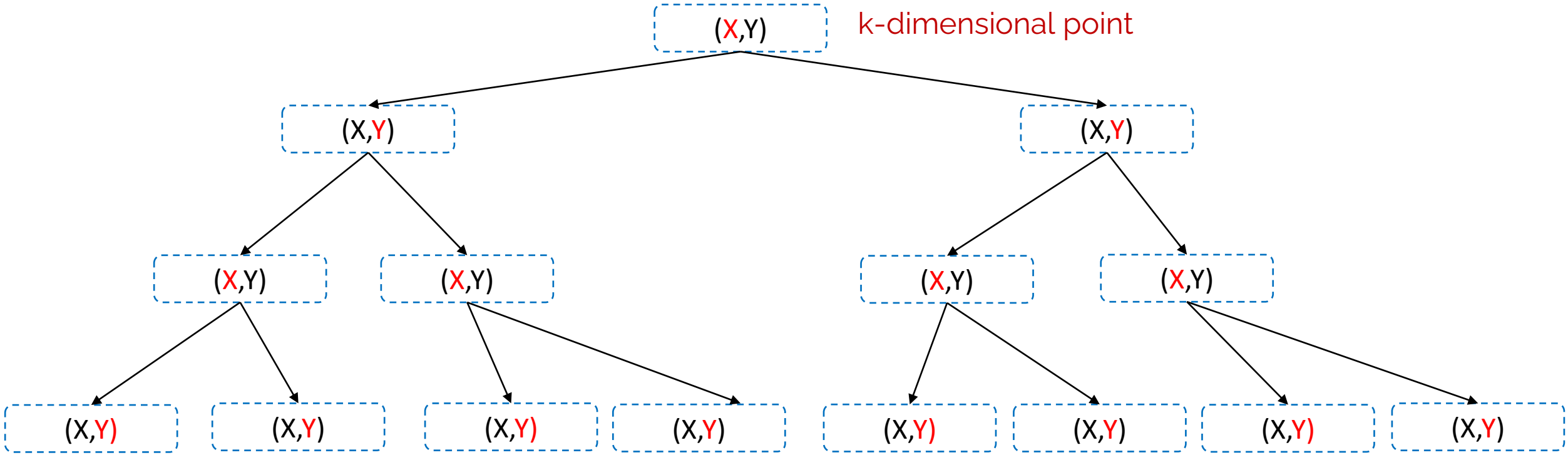

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 5, 7, 8]
y = [2, 6, 4, 6, 8, 3]

for xy in zip(x, y):
    plt.annotate('(%.2f, %.2f)' % xy, xy=xy)

plt.scatter(x, y)
plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)

plt.show()
```

★ **New datapoint**

1. Find distance of each point in the dataset
2. The point with lowest distance would be nears

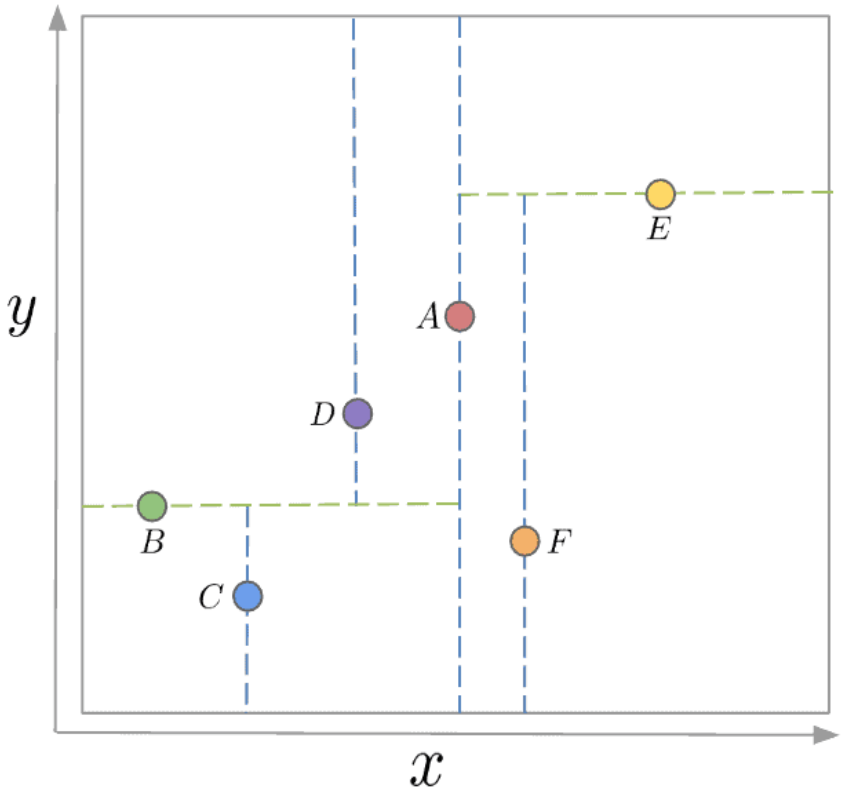
K-D Tree Solution

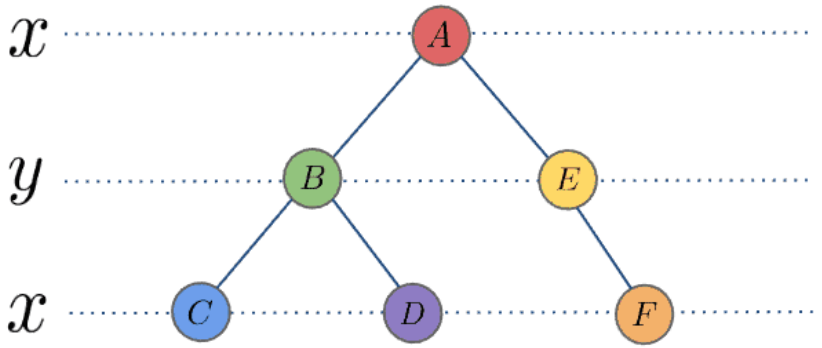
A K-D Tree is a **binary tree** in which each node represents a k-dimensional point**. Every non-leaf node in the tree acts as a hyperplane, dividing the space into two partitions.** This hyperplane is perpendicular to the chosen axis, which is associated with one of the K dimensions.



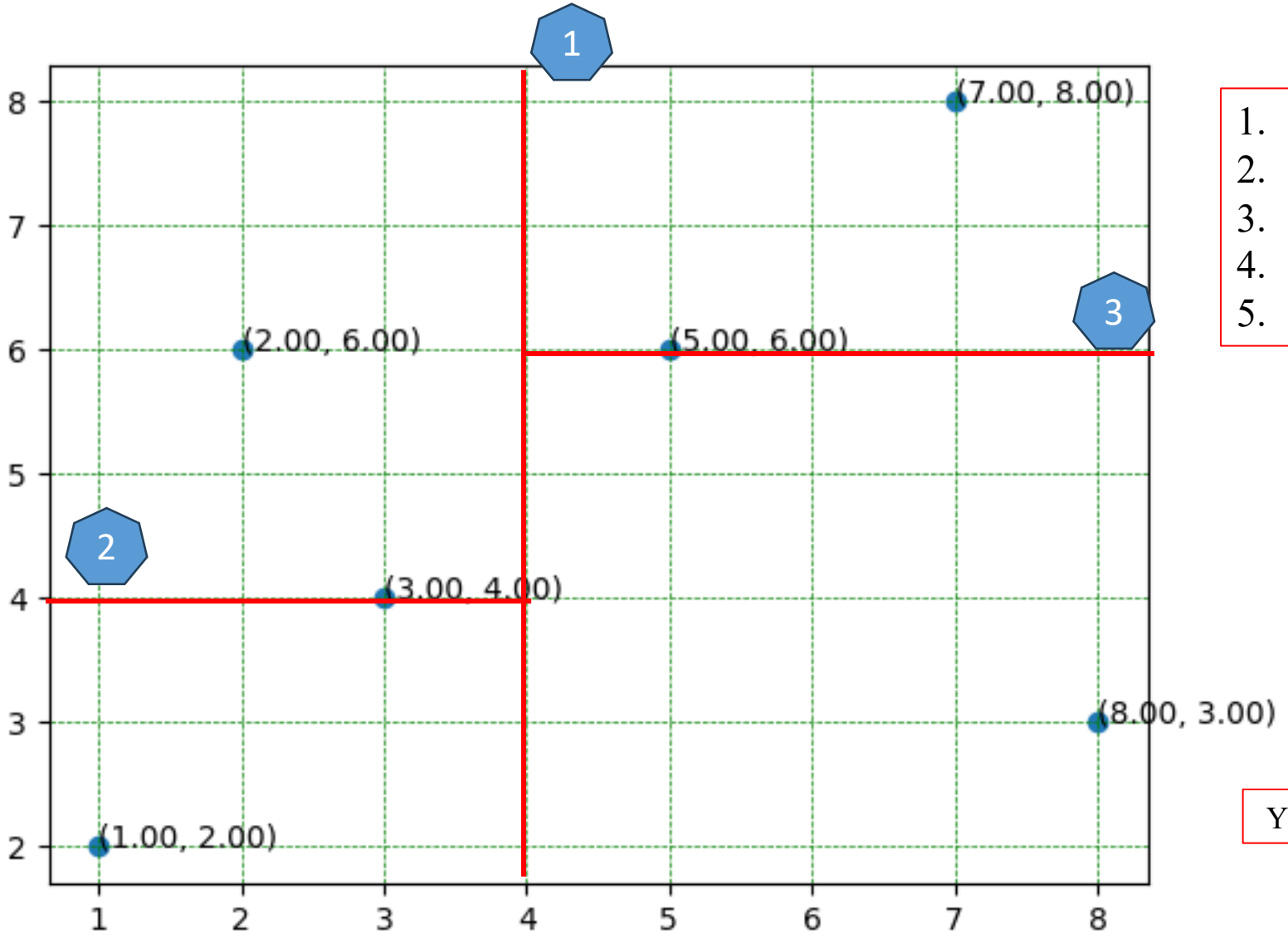
K-D Tree Solution

There are different strategies for choosing an axis when dividing, but the most common one would be to cycle through each of the K dimensions repeatedly and select a midpoint along it to divide the space. For instance, in the case of 2-dimensional points with x and y axes, we first split along the x-axis, then the y-axis, and then the x-axis again, continuing in this manner until all points are accounted for:

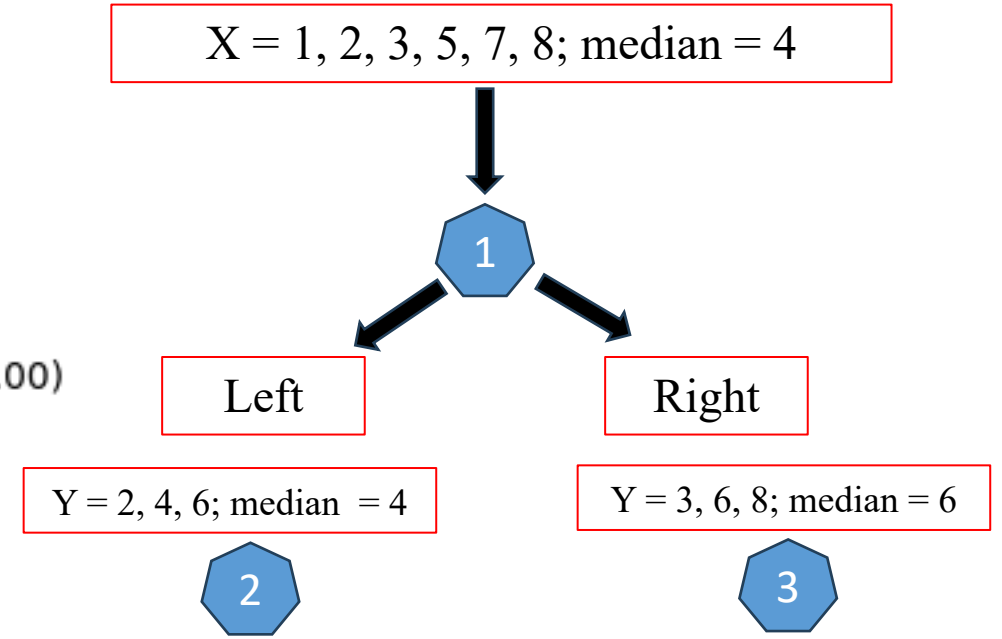




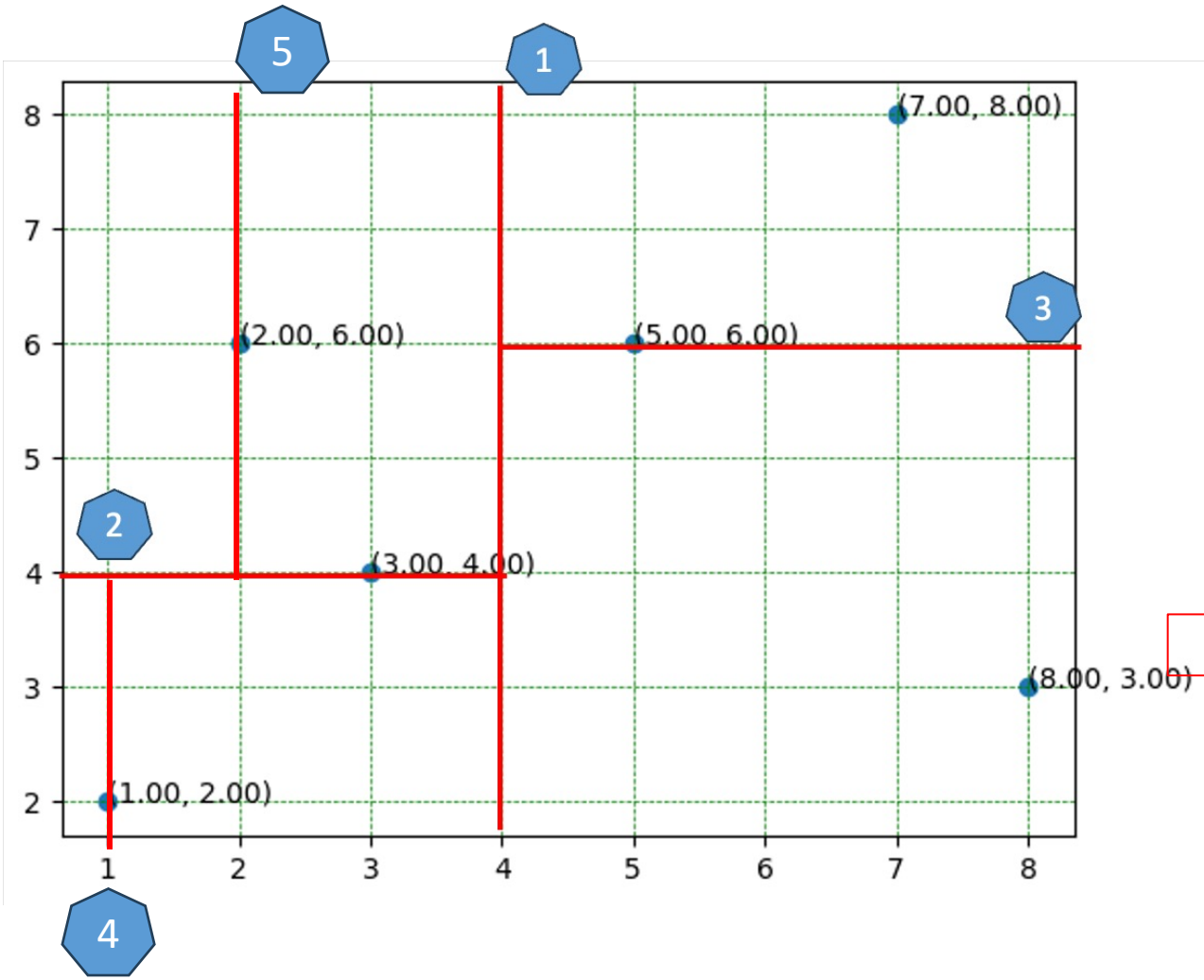
K-D Tree Solution

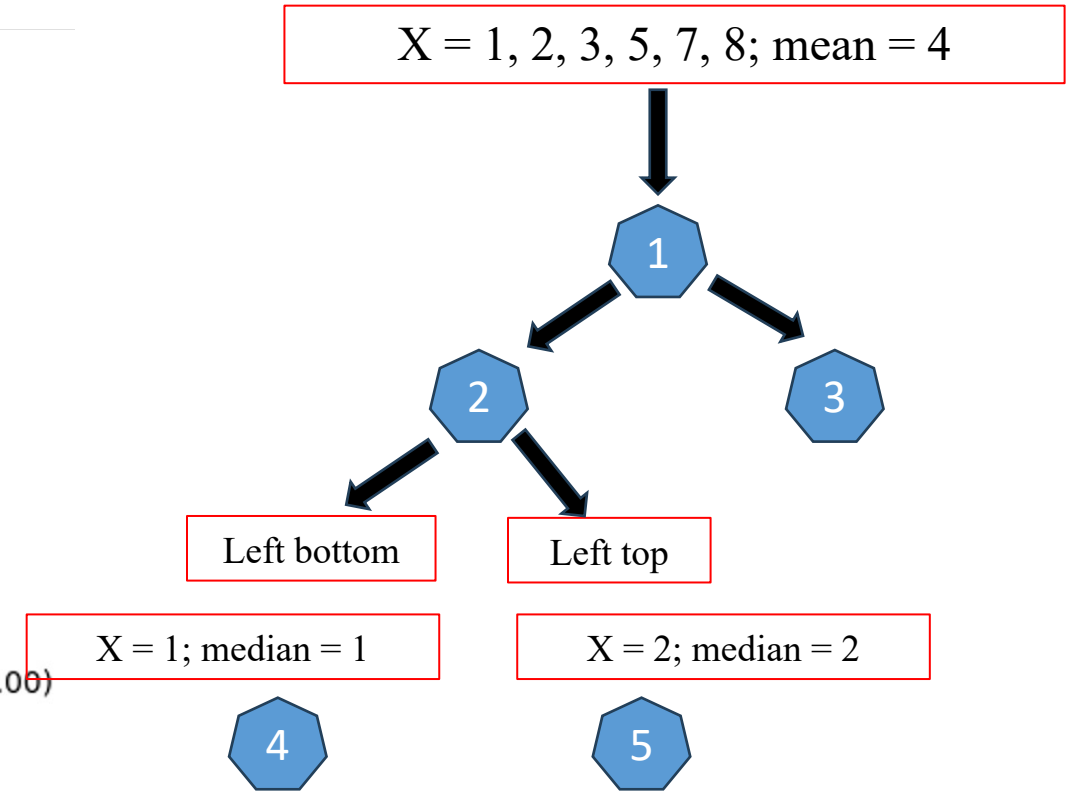


1. Pick any one feature at random
2. Find median
3. Split dataset in approximate equal halves
4. Pick next feature and repeat step #2,3
5. Continue until all data points are partitioned

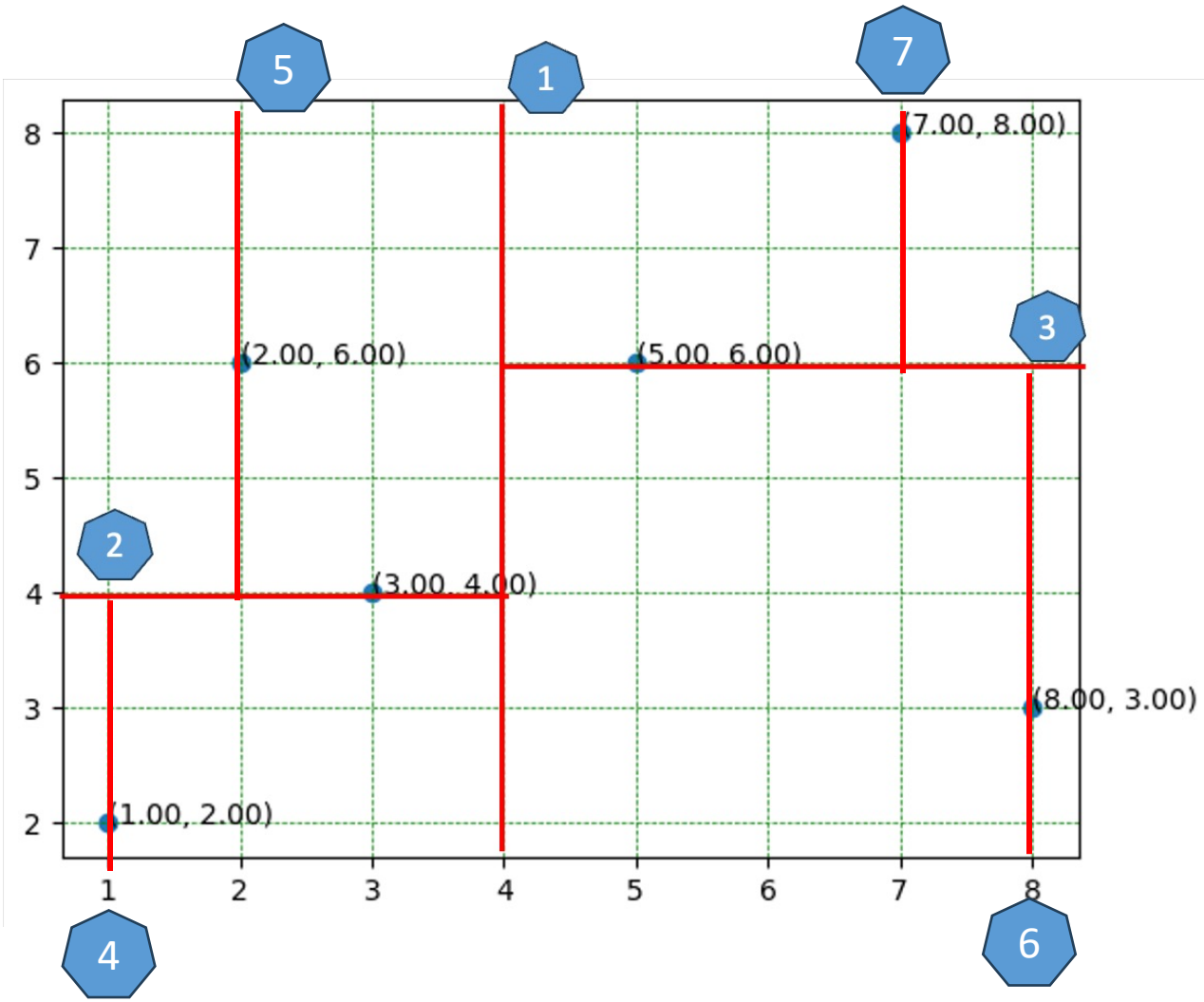


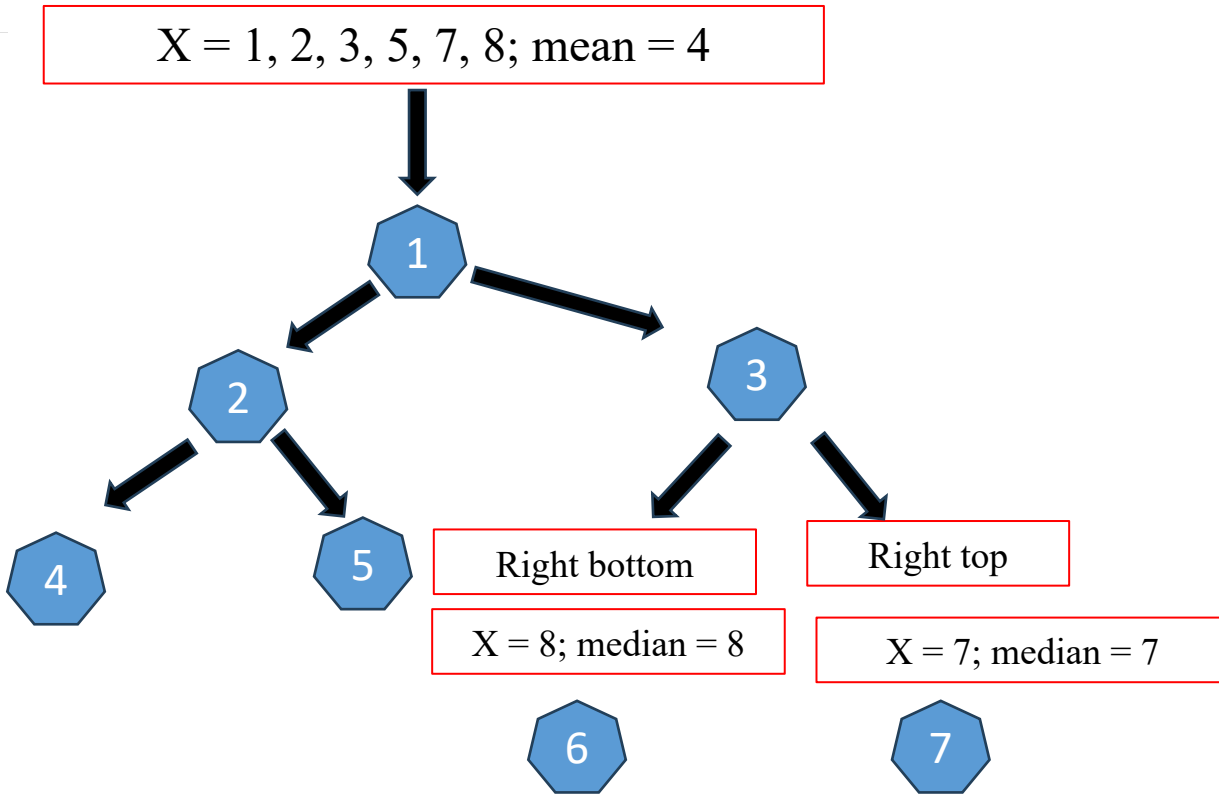
K-D Tree Solution



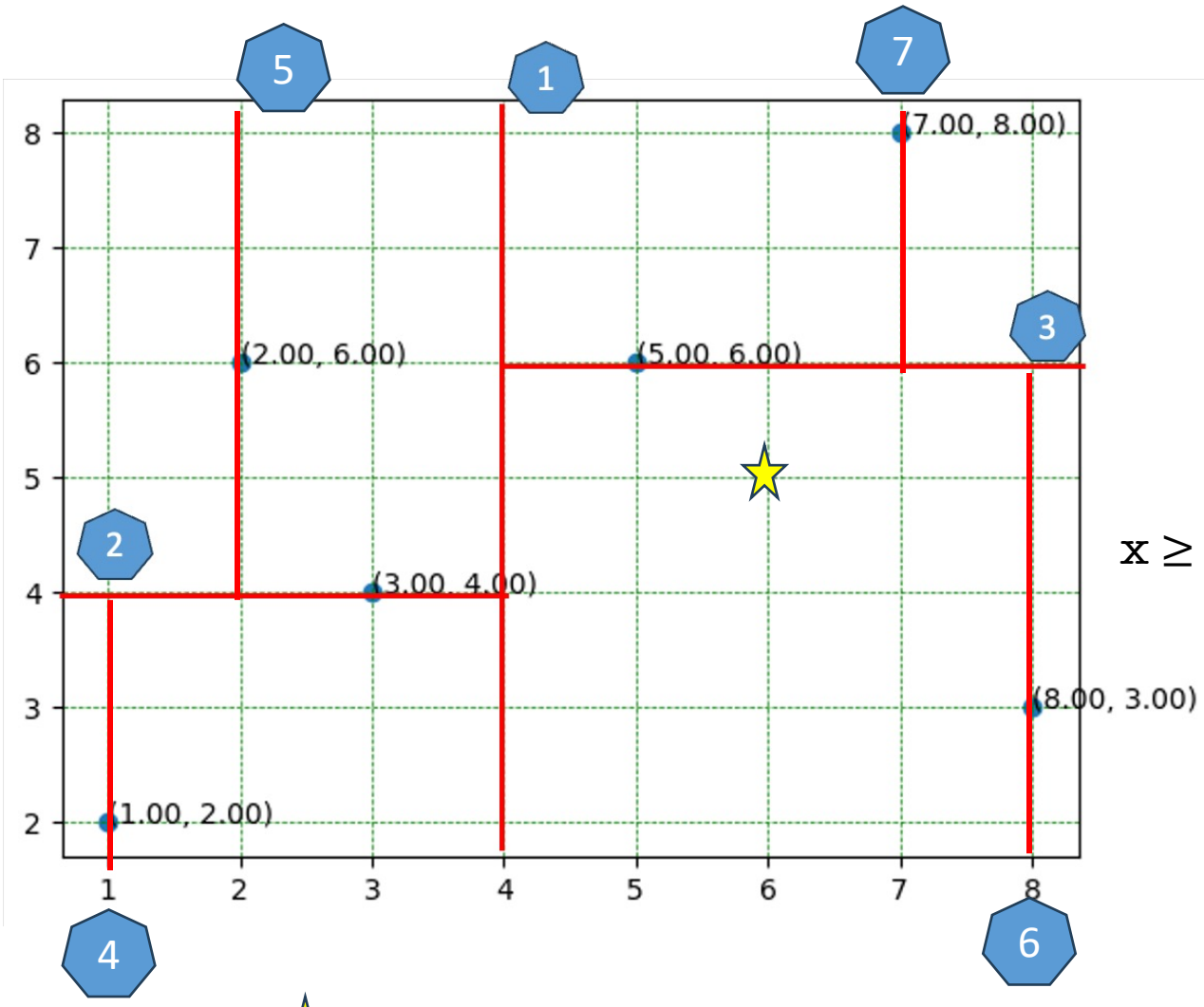


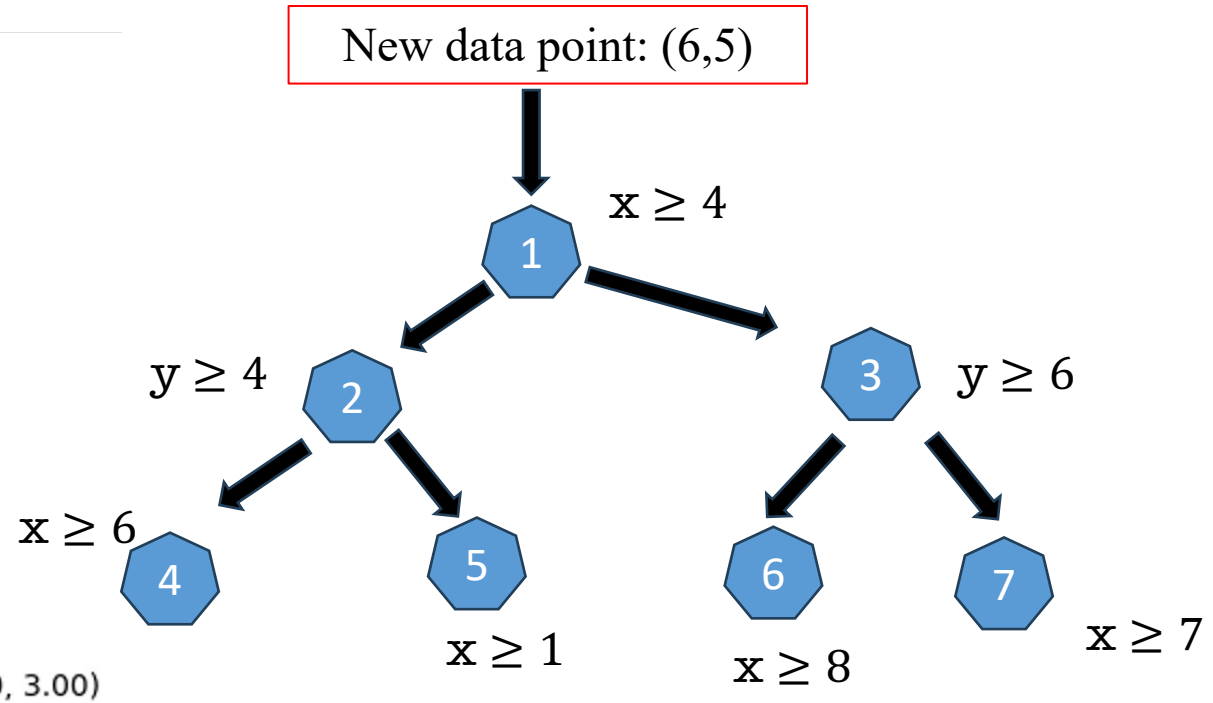
K-D Tree Solution





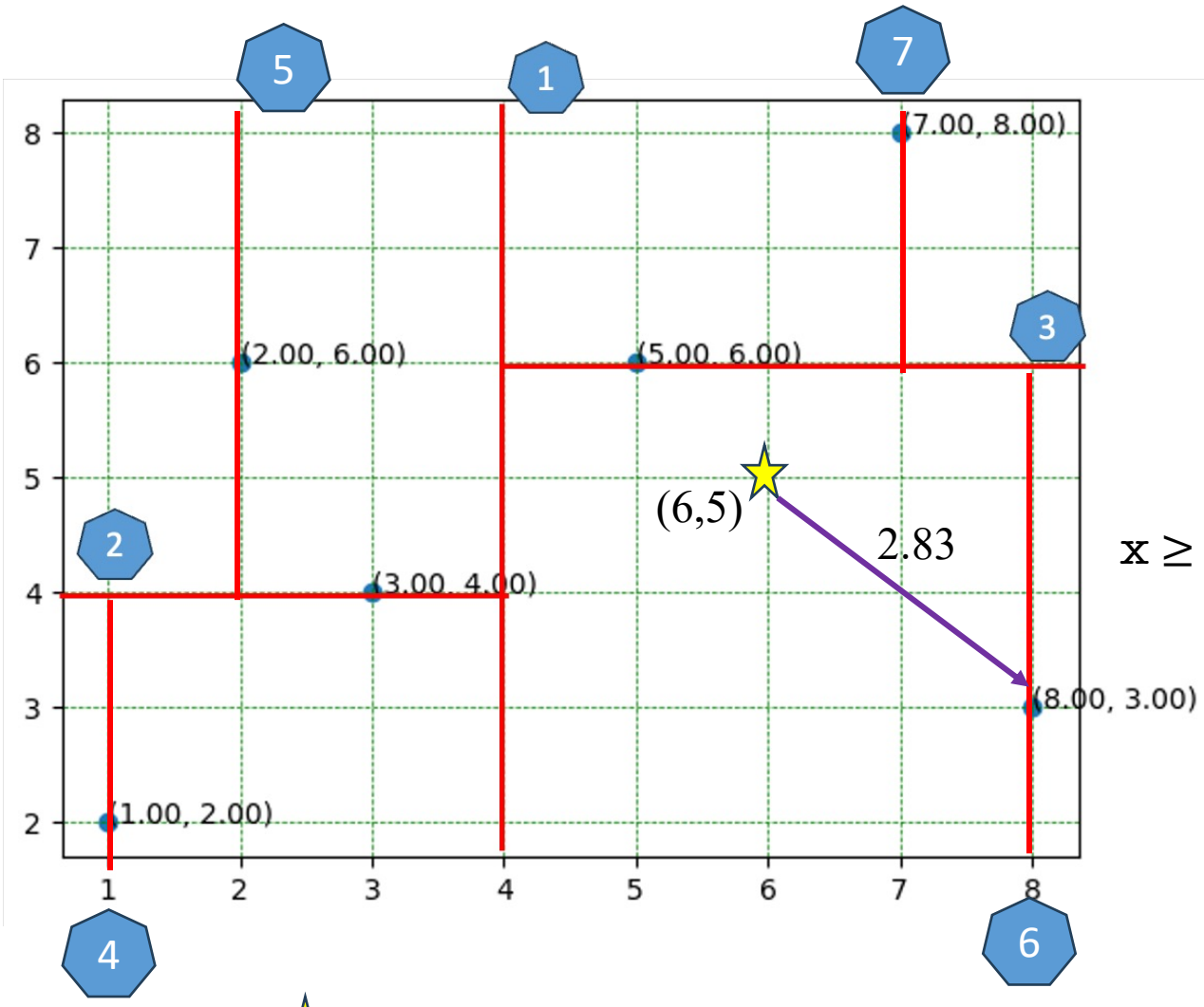
K-D Tree Solution

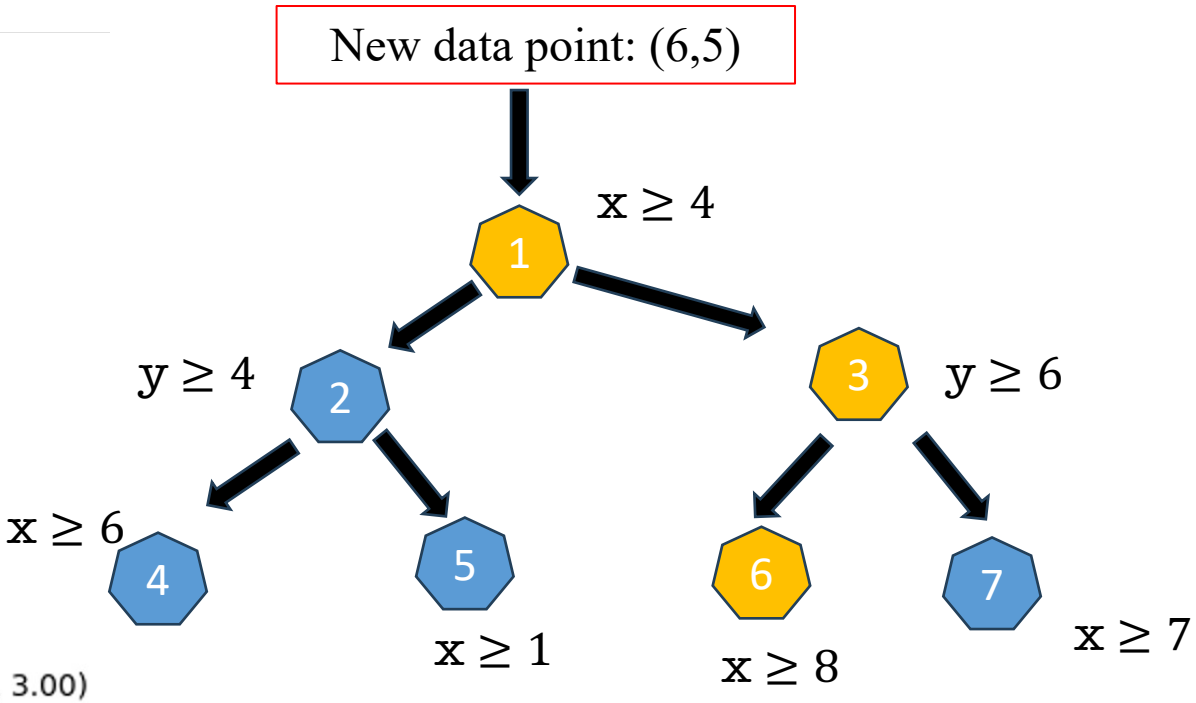




AI VIET NAM
@aivietnam.edu.vn

K-D Tree Solution



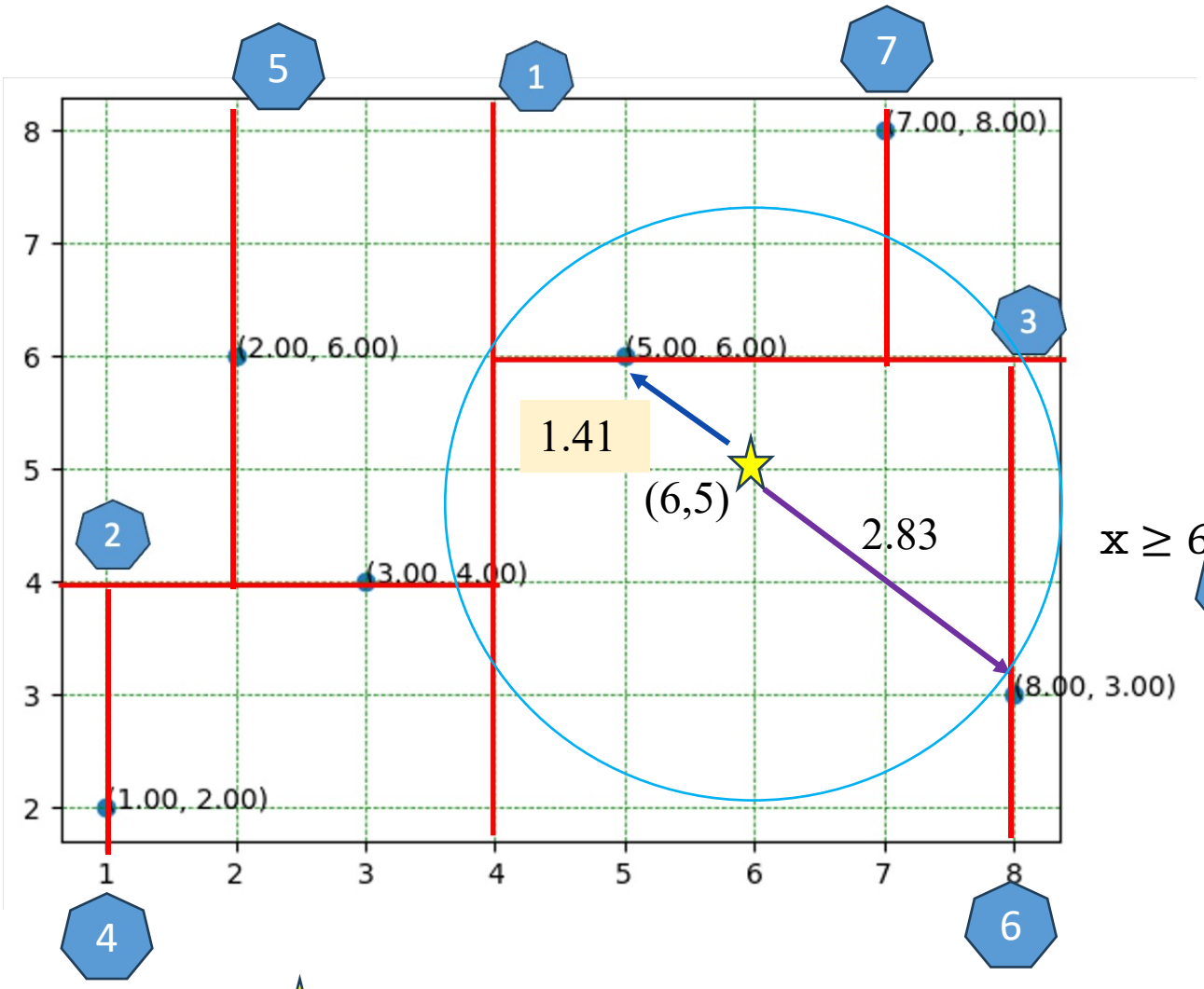


Minimum distance = 2.83

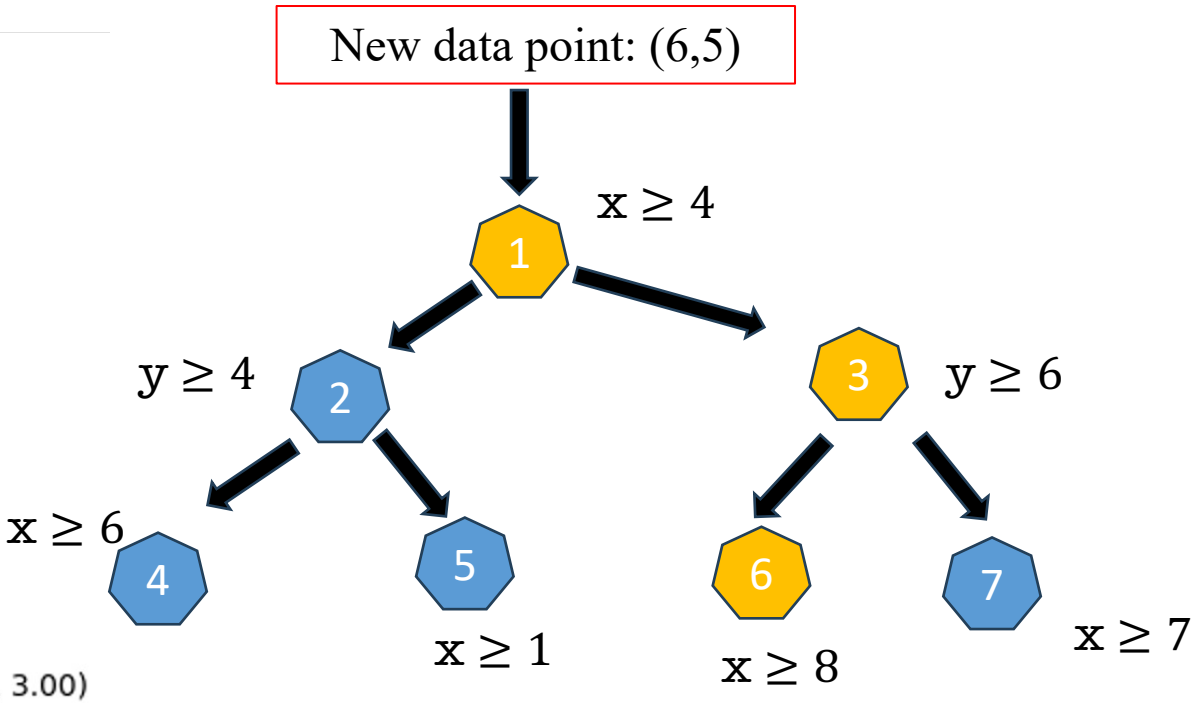
★ **New datapoint**

AI VIET NAM
@aivietnam.edu.vn

K-D Tree Solution



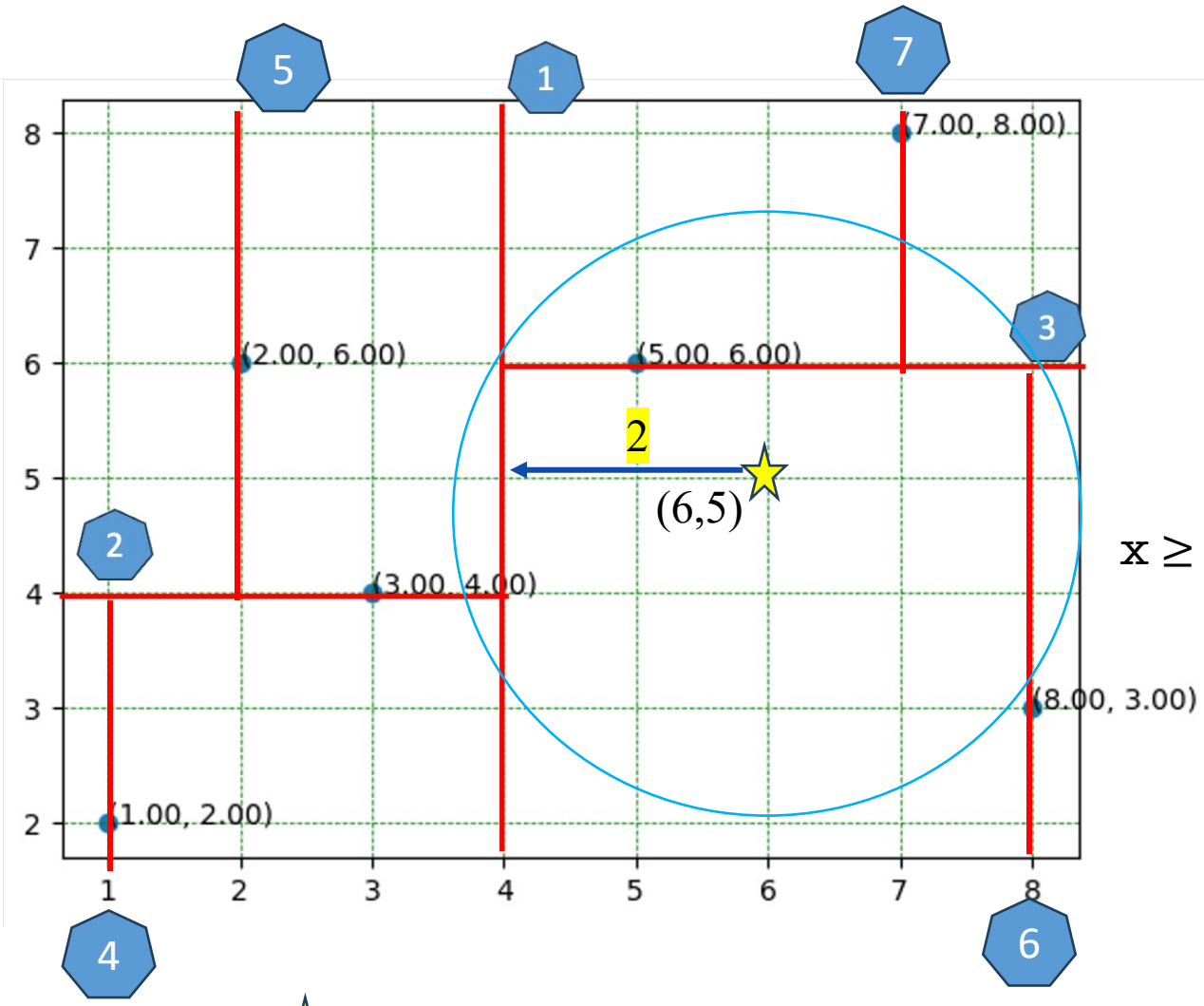
New datapoint



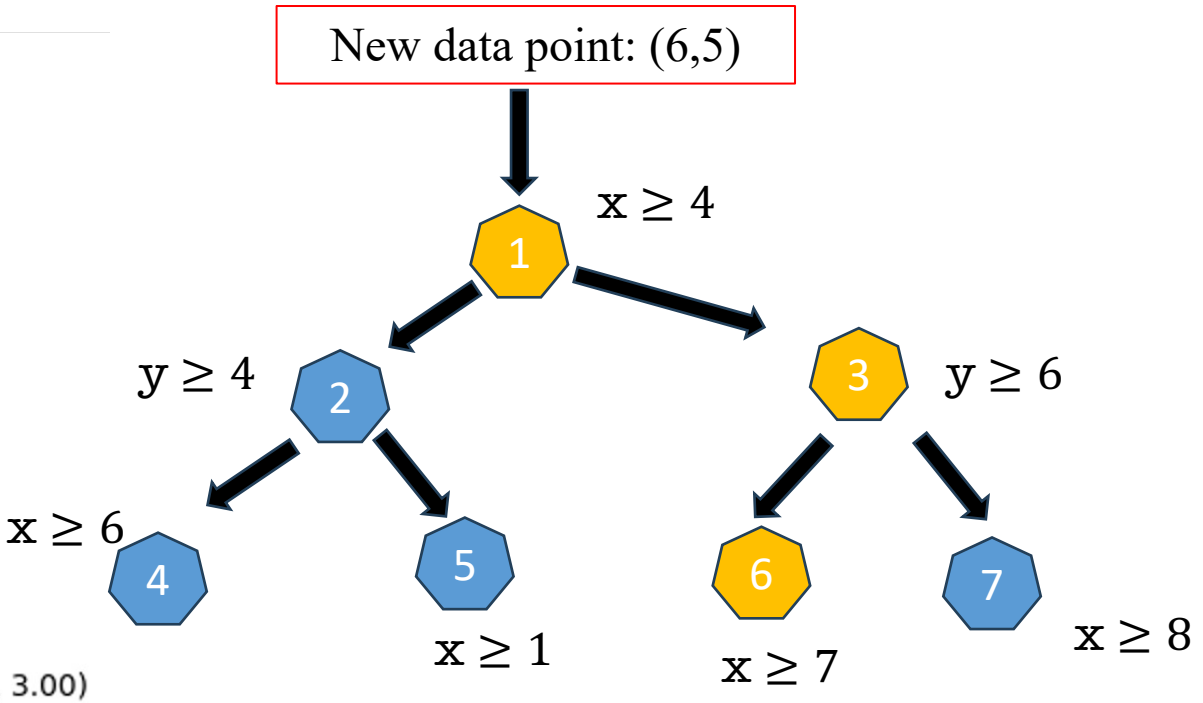
Minimum distance = 2.83

Minimum distance = 1.41

K-D Tree Solution



New datapoint



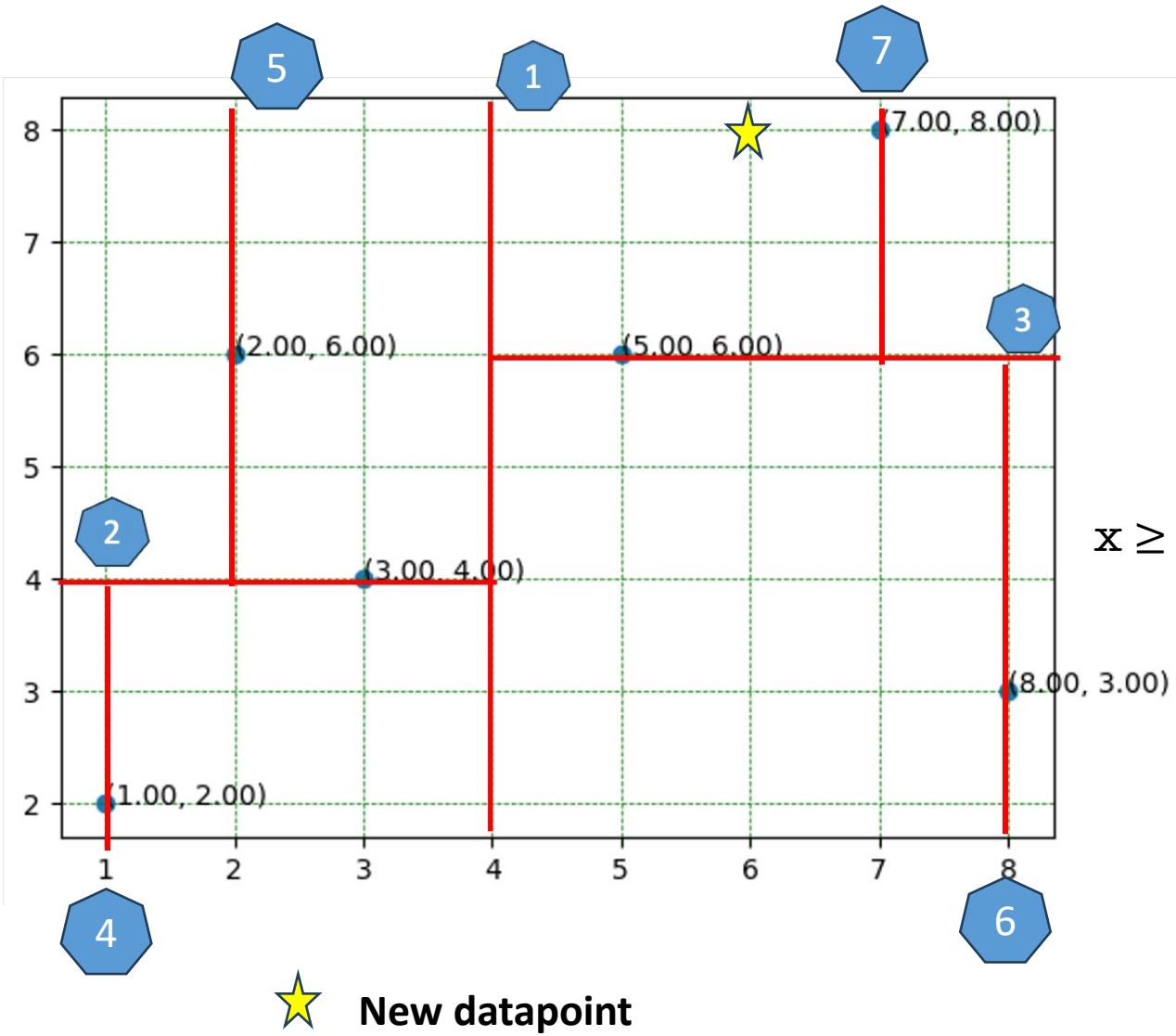
Minimum distance = 2.83

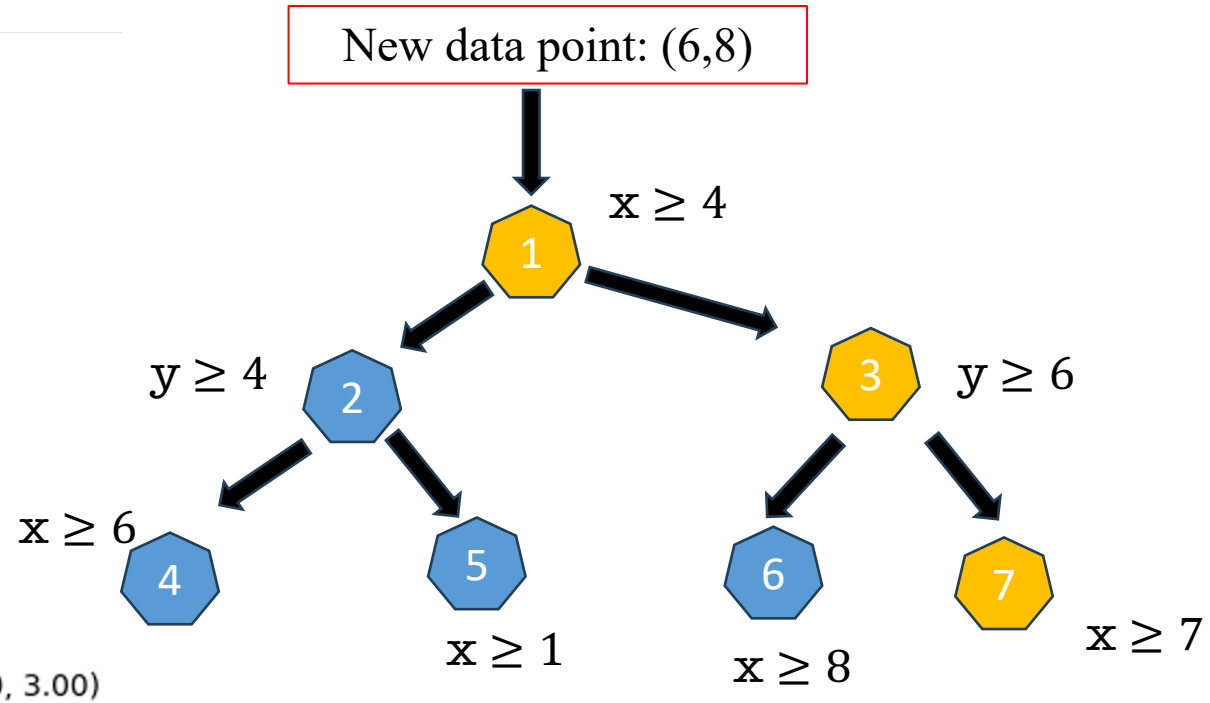
Minimum distance = 1.41

(5,6) is the nearest point

AI VIET NAM
@aivietnam.edu.vn

K-D Tree Solution





What is the nearest neighbor?

K-D Tree Implementation Solution

Algorithm 1: K-D Tree Construction

Function *kdtree(pointList, depth)*:

Input: a list of points, *pointList*, and an integer, *depth* ;

Output: a k-d tree rooted at the median point of *pointList* ;

```
/* Select axis based on depth so that axis cycles
   through all valid values                          */
```

let $axis := depth \mod k$;

```
/* Sort point list and choose median as pivot element
   */
```

select median by *axis* from *pointList*;

```
/* Create node and construct subtree                 */
```

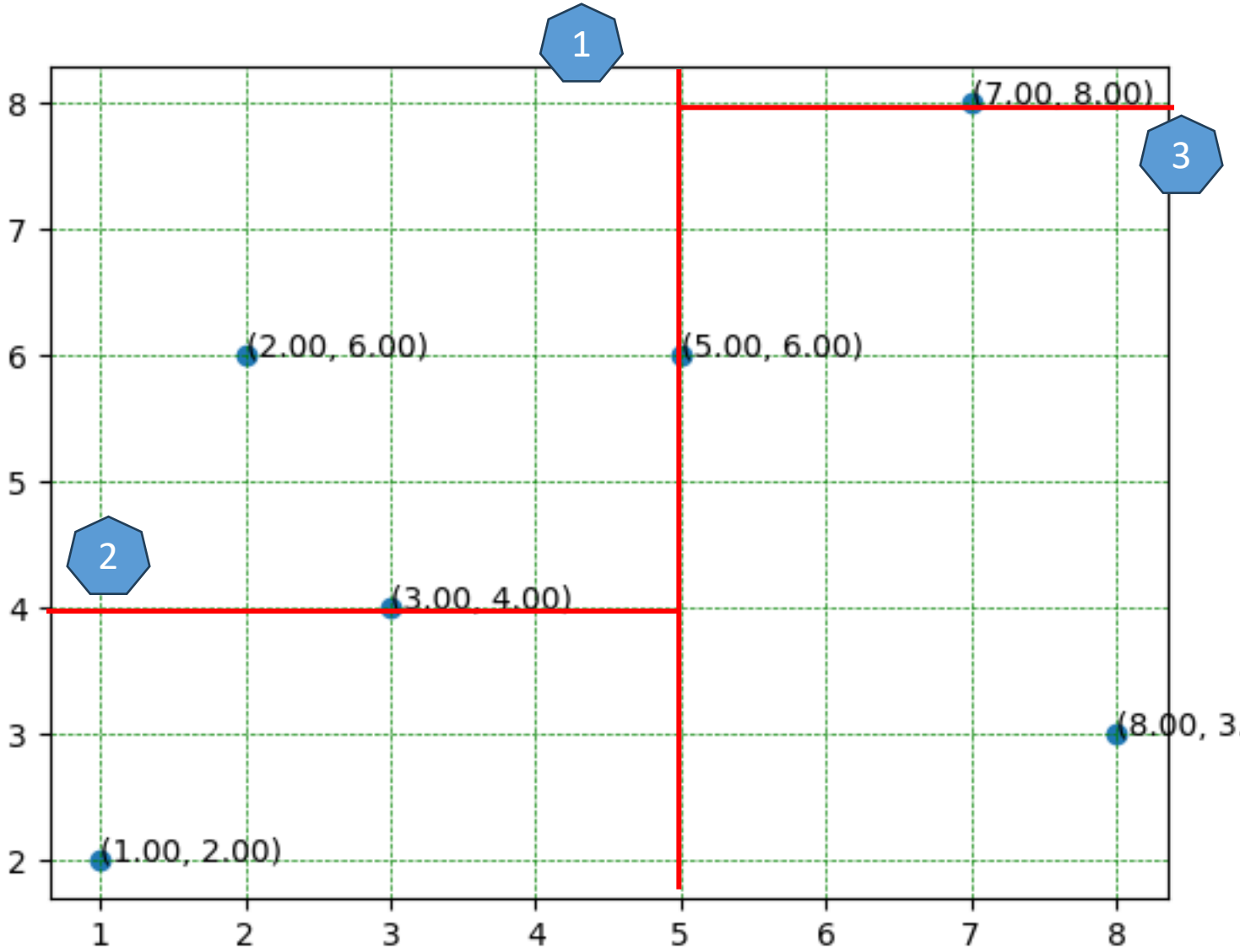
let *node.location* := *median*;

let *node.leftChild* := kdtree(points in *pointList* before *median*, $depth + 1$);

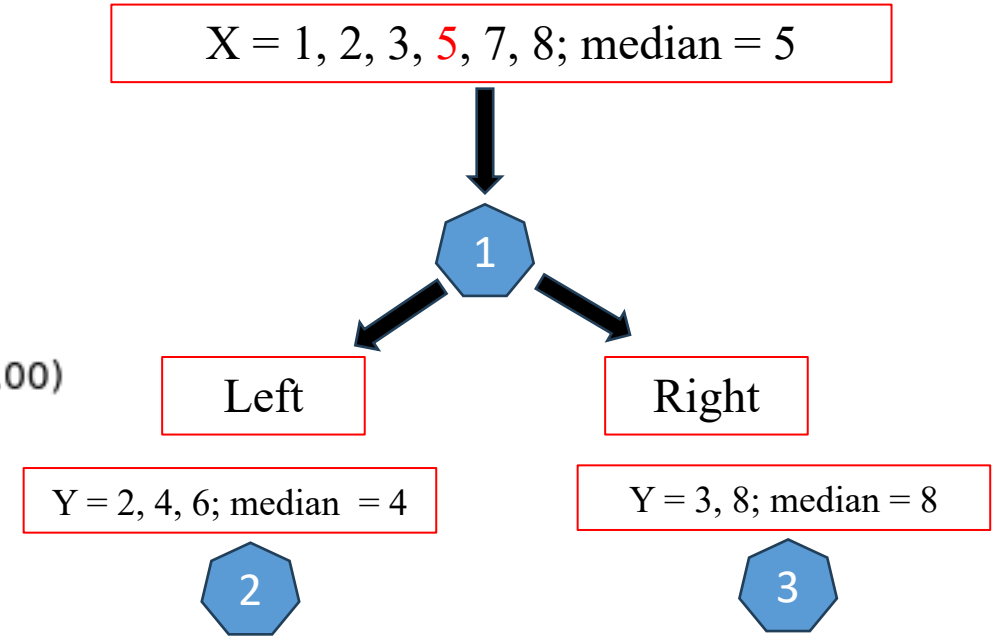
let *node.rightChild* := kdtree(points in *pointList* after *median*, $depth + 1$);

return *node*;

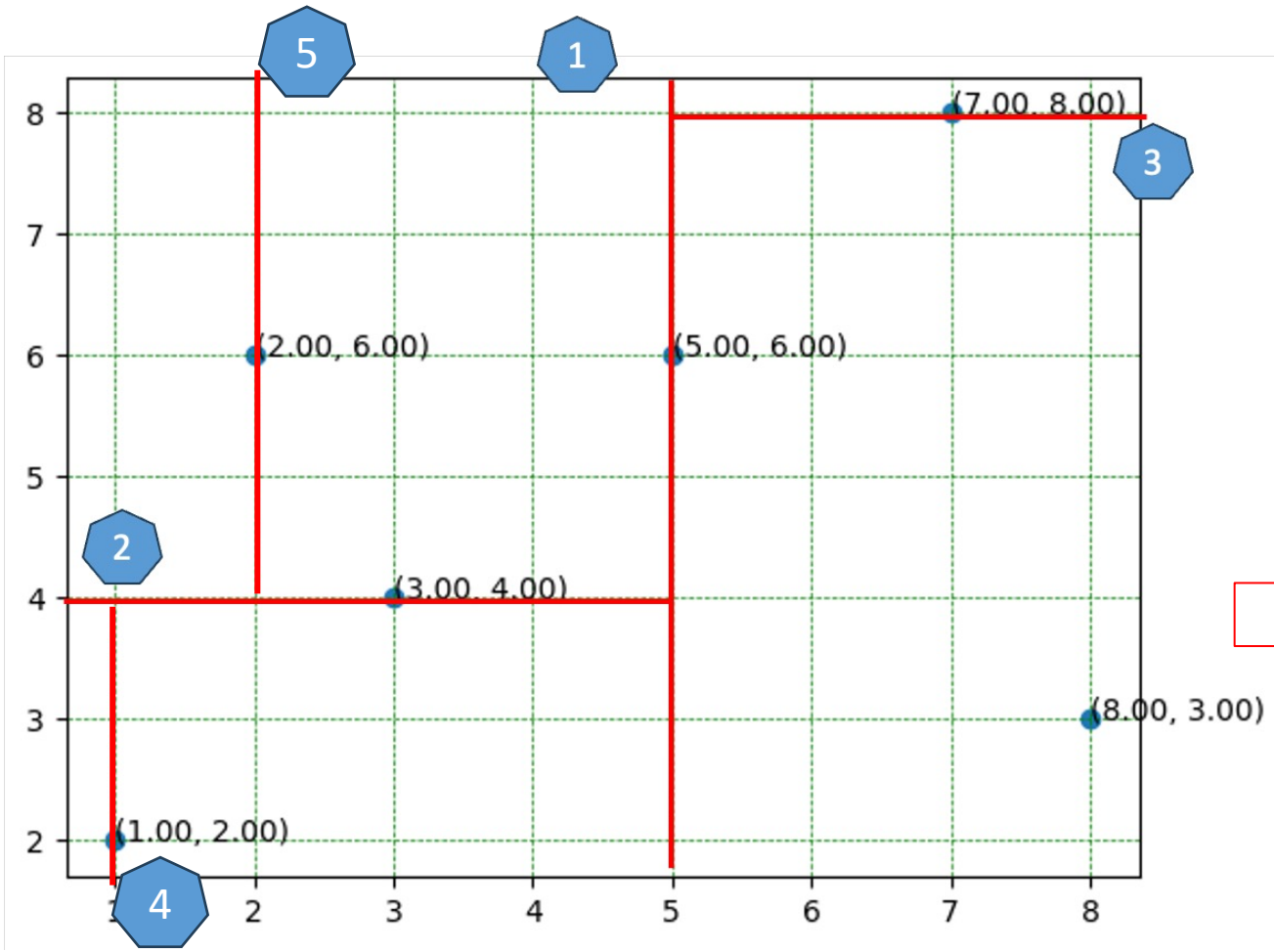
K-D Tree Implementation Solution

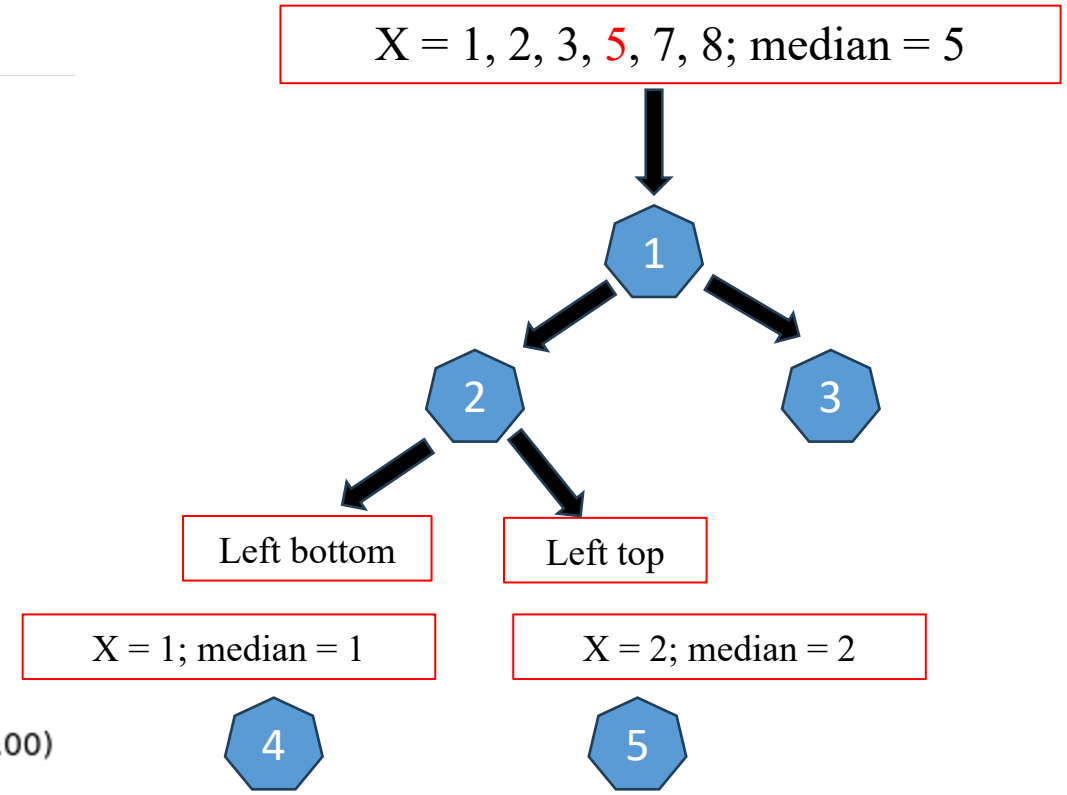


1. Pick any one feature at random
2. Find median
3. Split dataset in approximate equal halves
4. Pick next feature and repeat step #2,3
5. Continue until all data points are partitioned

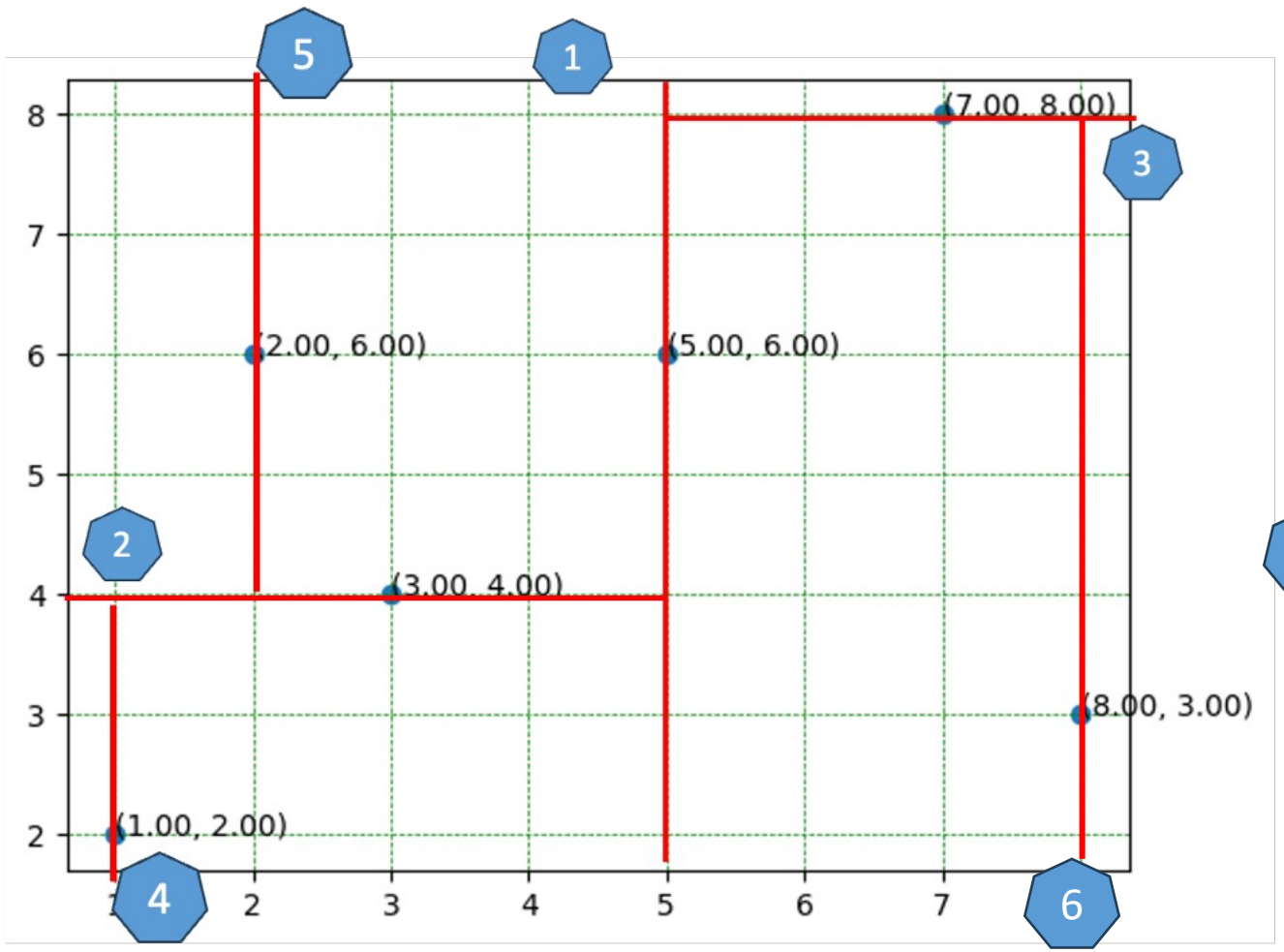


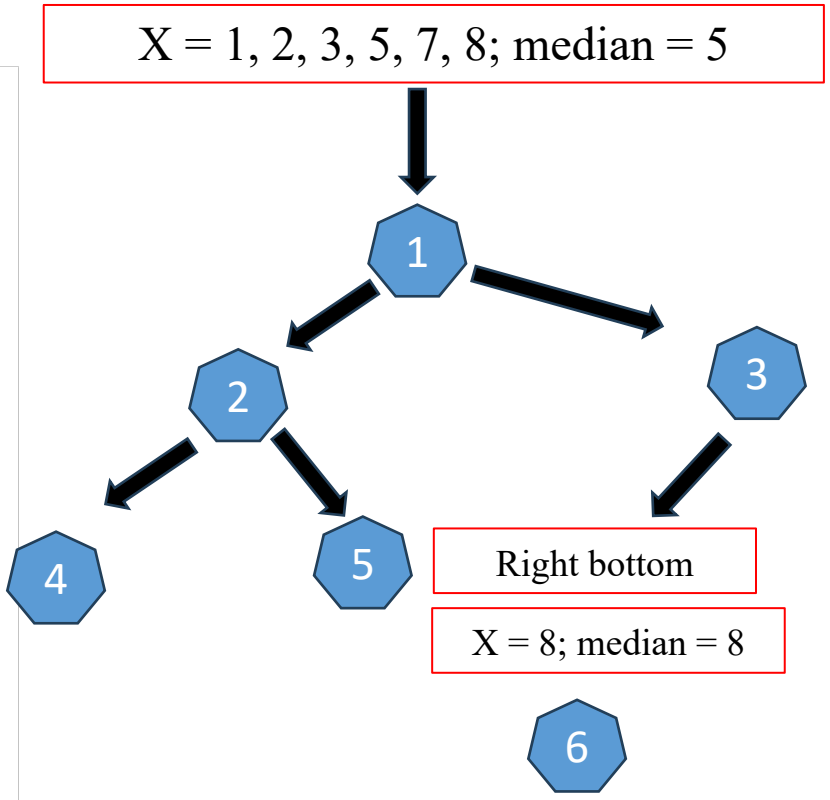
K-D Tree Implementation Solution



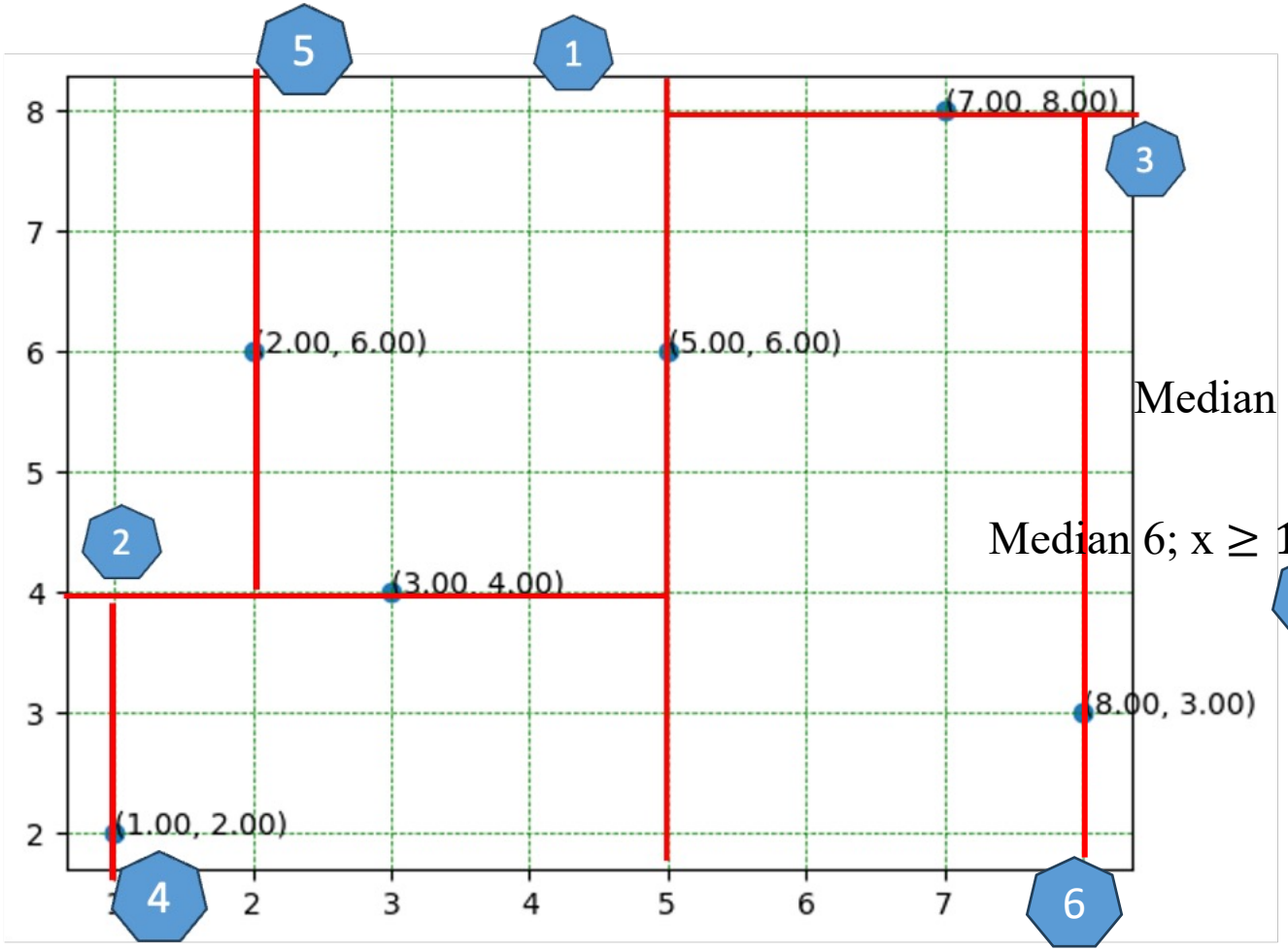


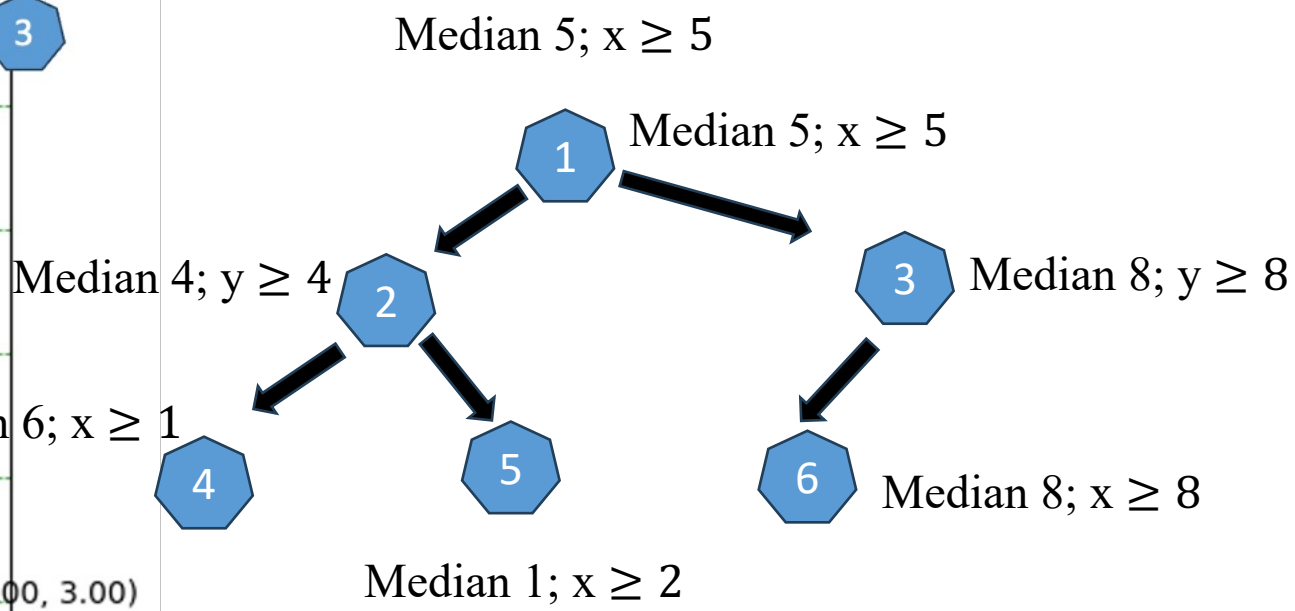
K-D Tree Implementation Solution





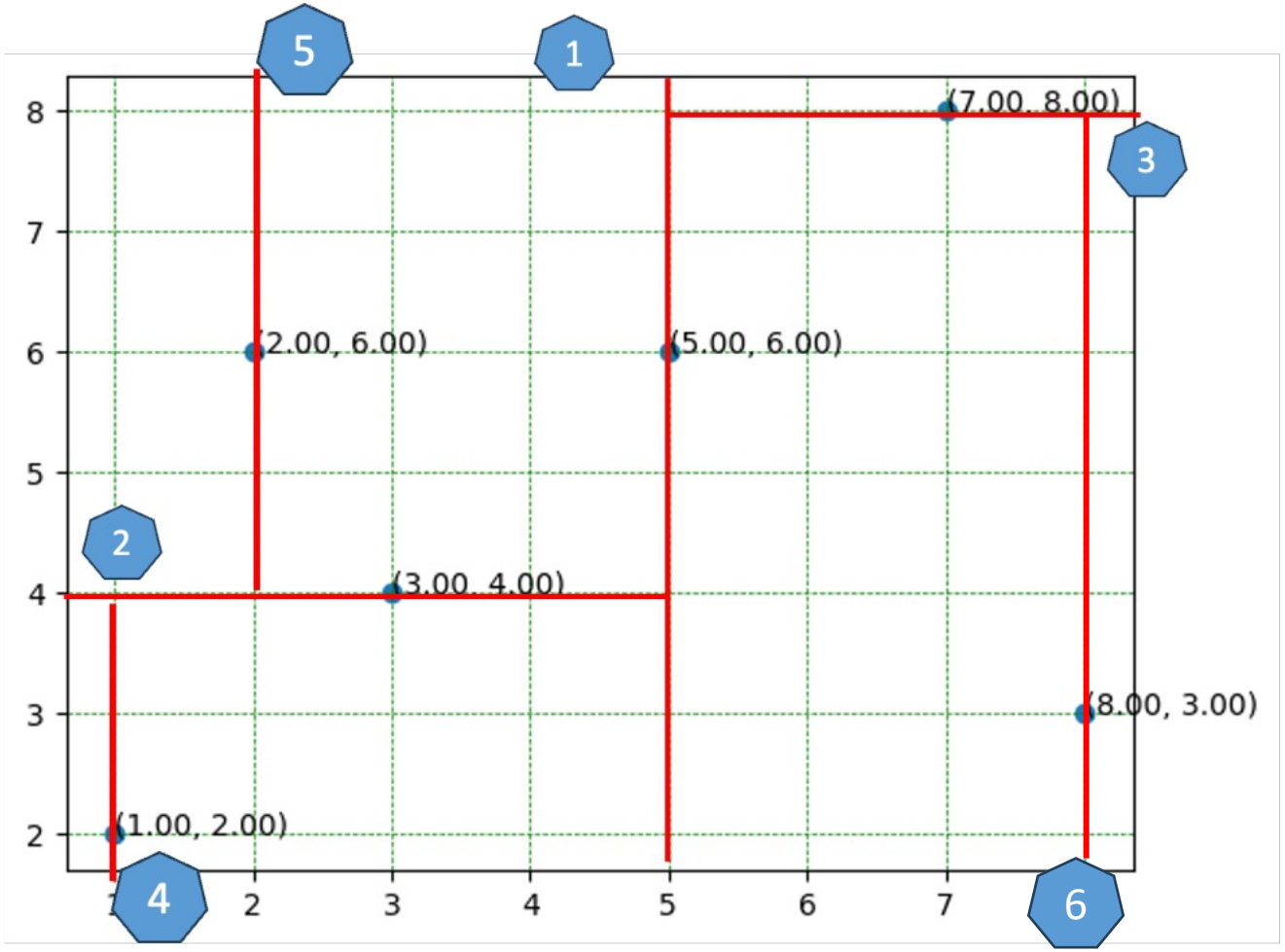
K-D Tree Implementation Solution

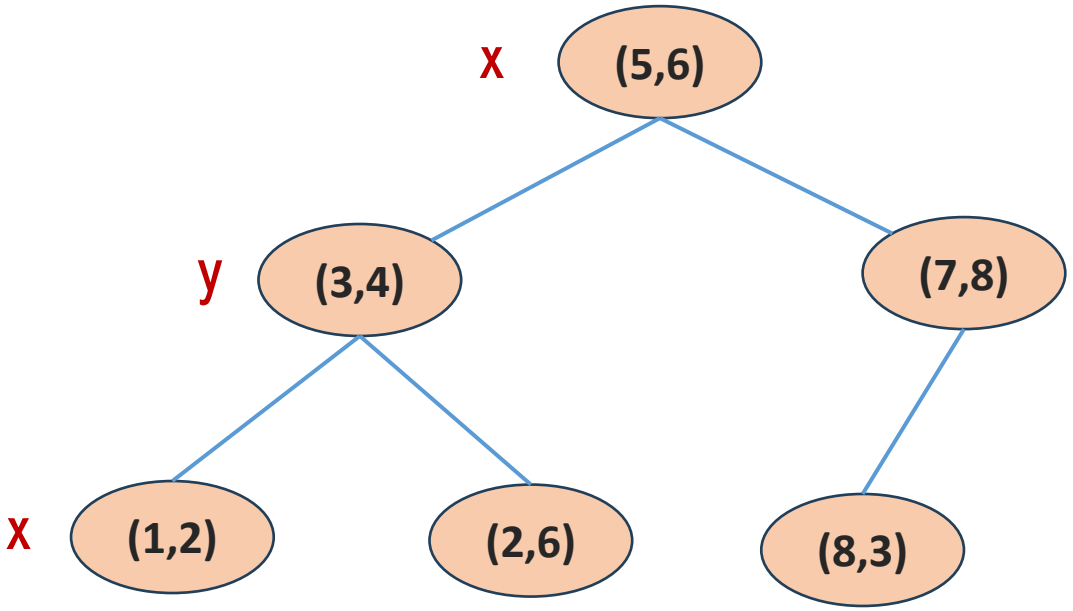




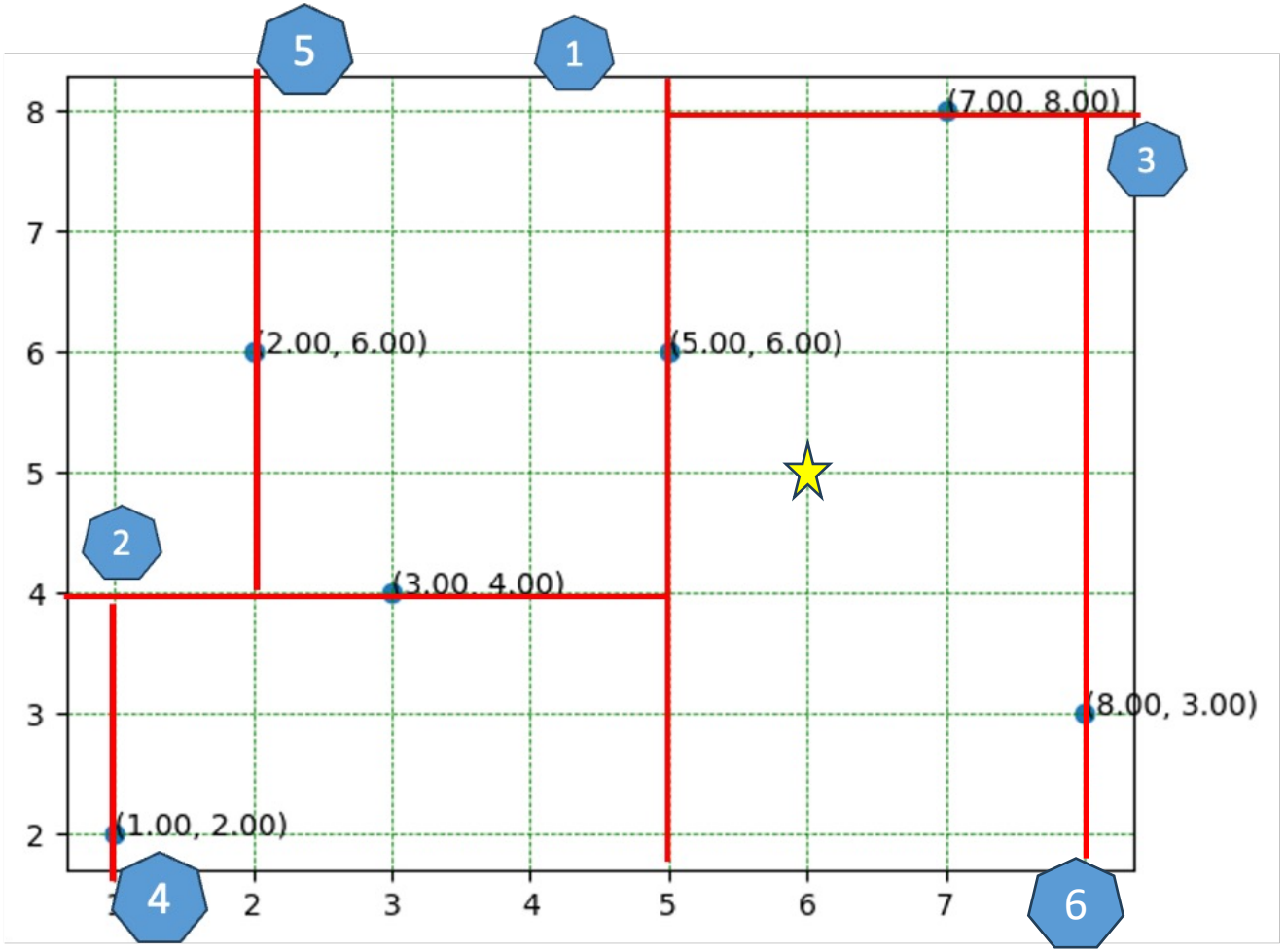
★ **New datapoint**

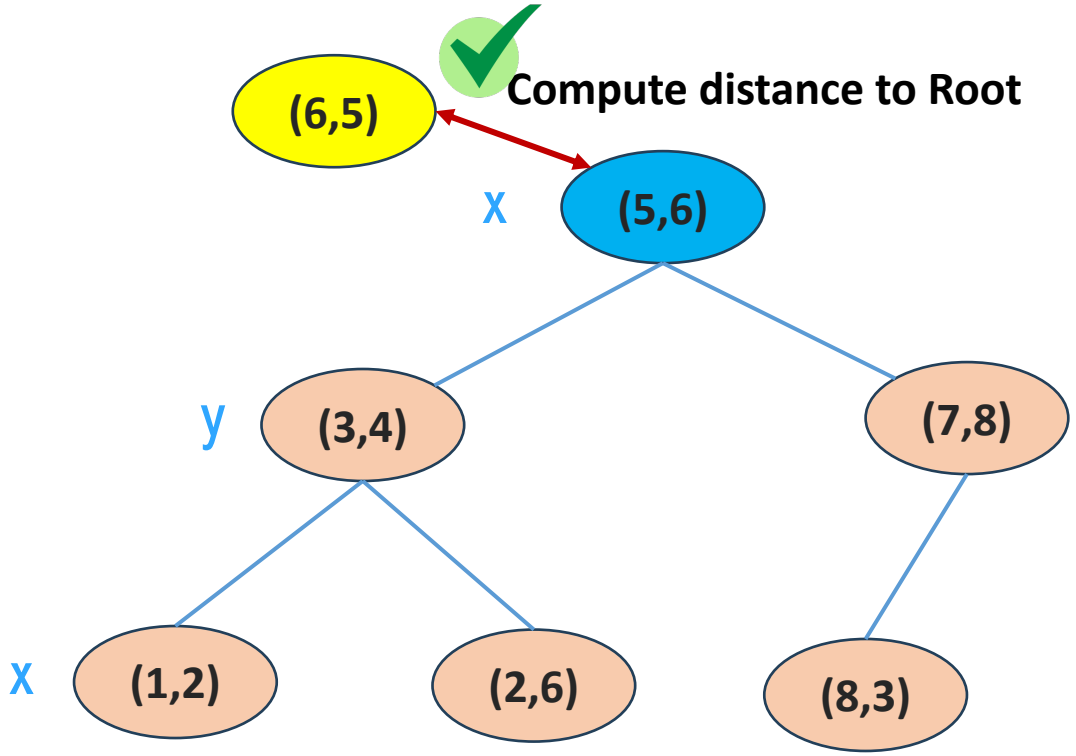
K-D Tree Implementation Solution





K-D Tree Implementation Solution

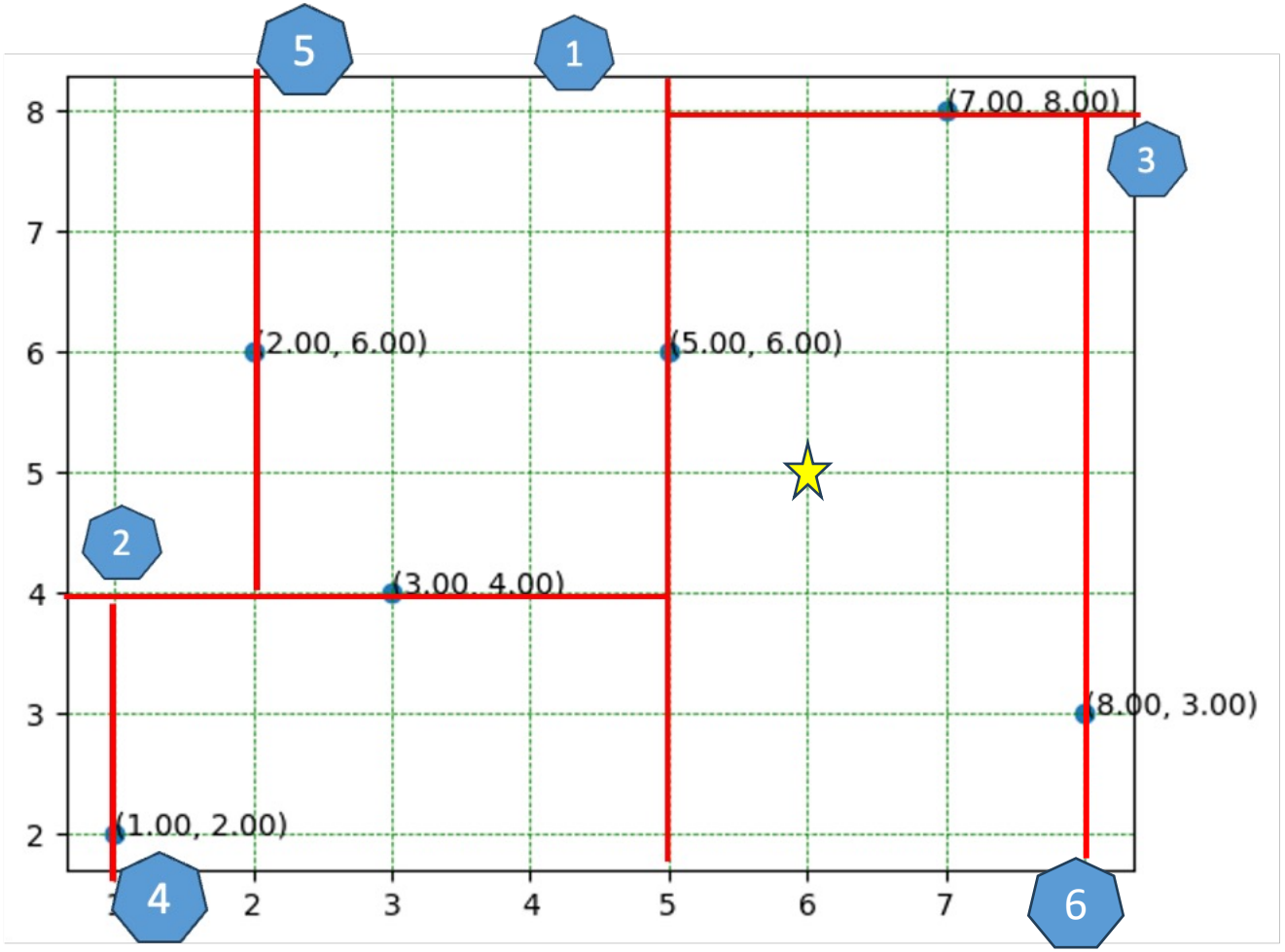


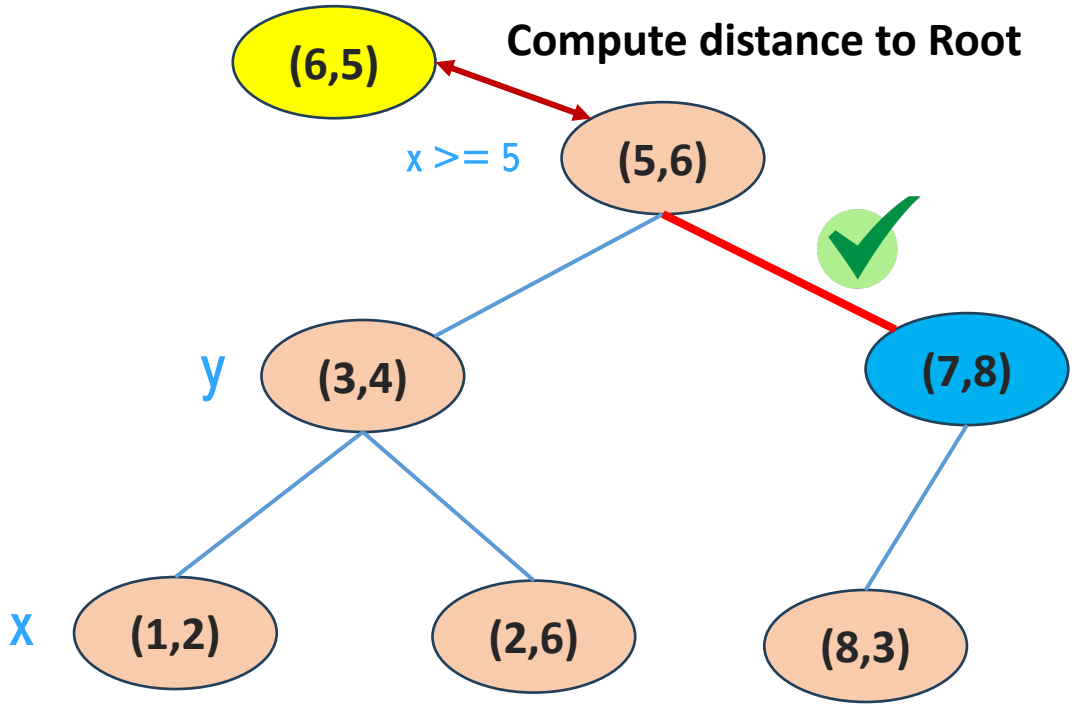


Best distance = 1.41
Best label = (5,6)

New datapoint

K-D Tree Implementation Solution





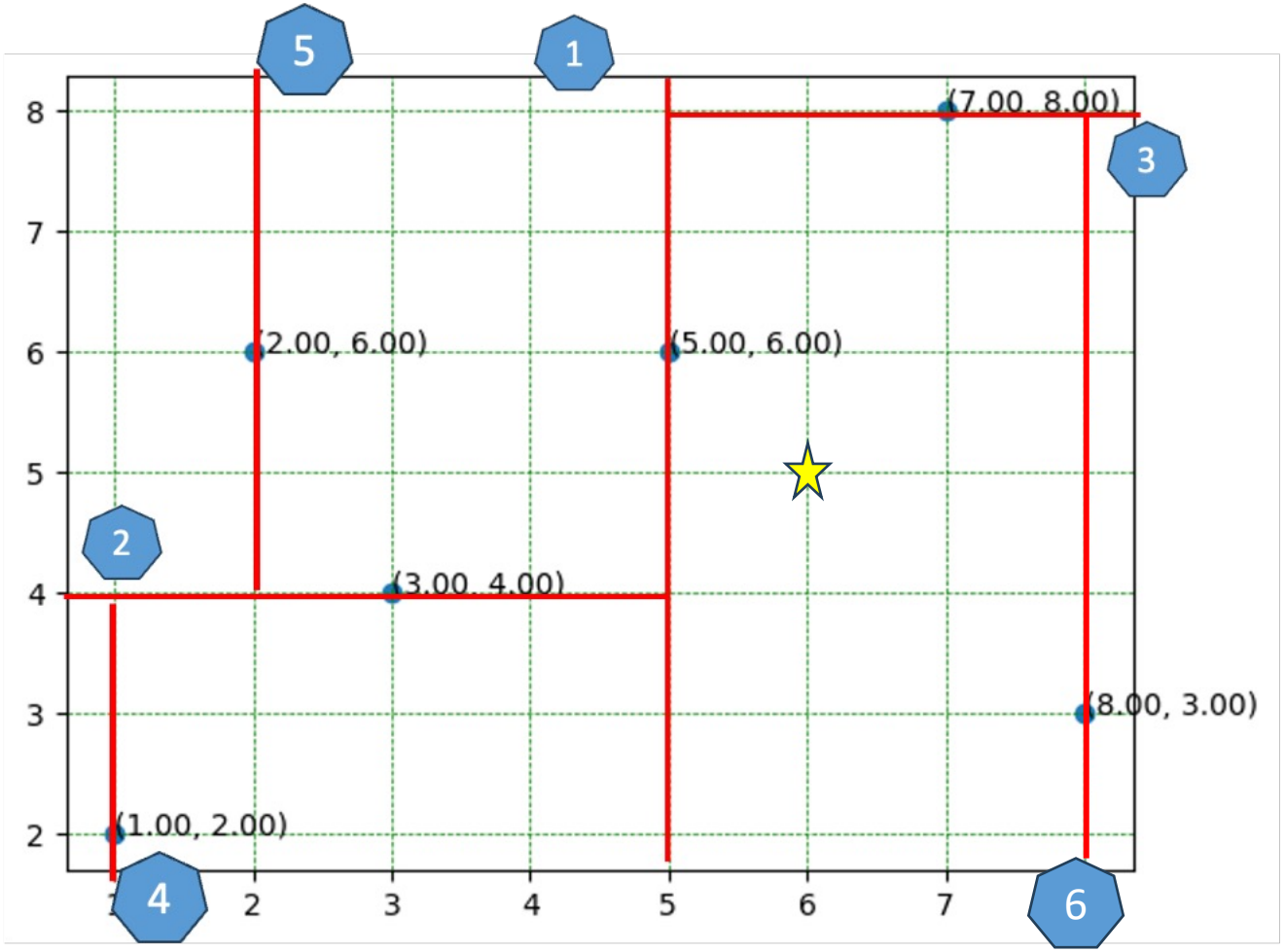
New datapoint

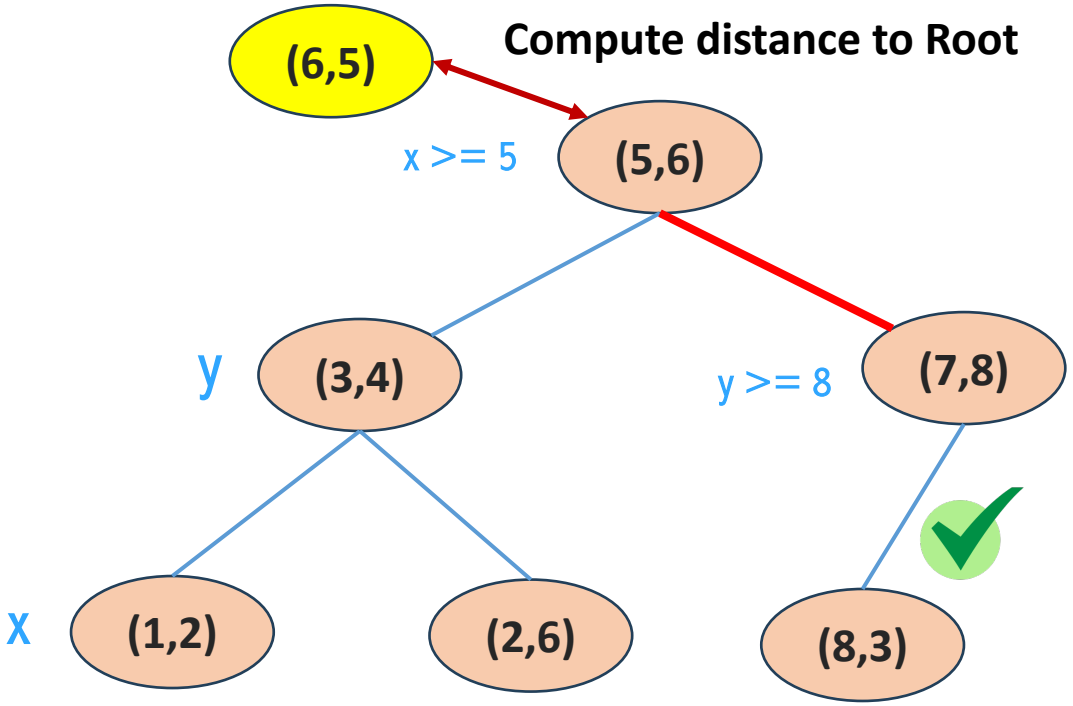
Best distance = 1.41
Best label = (5,6)

Curent distance = 3.16
Curent label = (5,6)

Not update the best distance

K-D Tree Implementation Solution





(6,5)

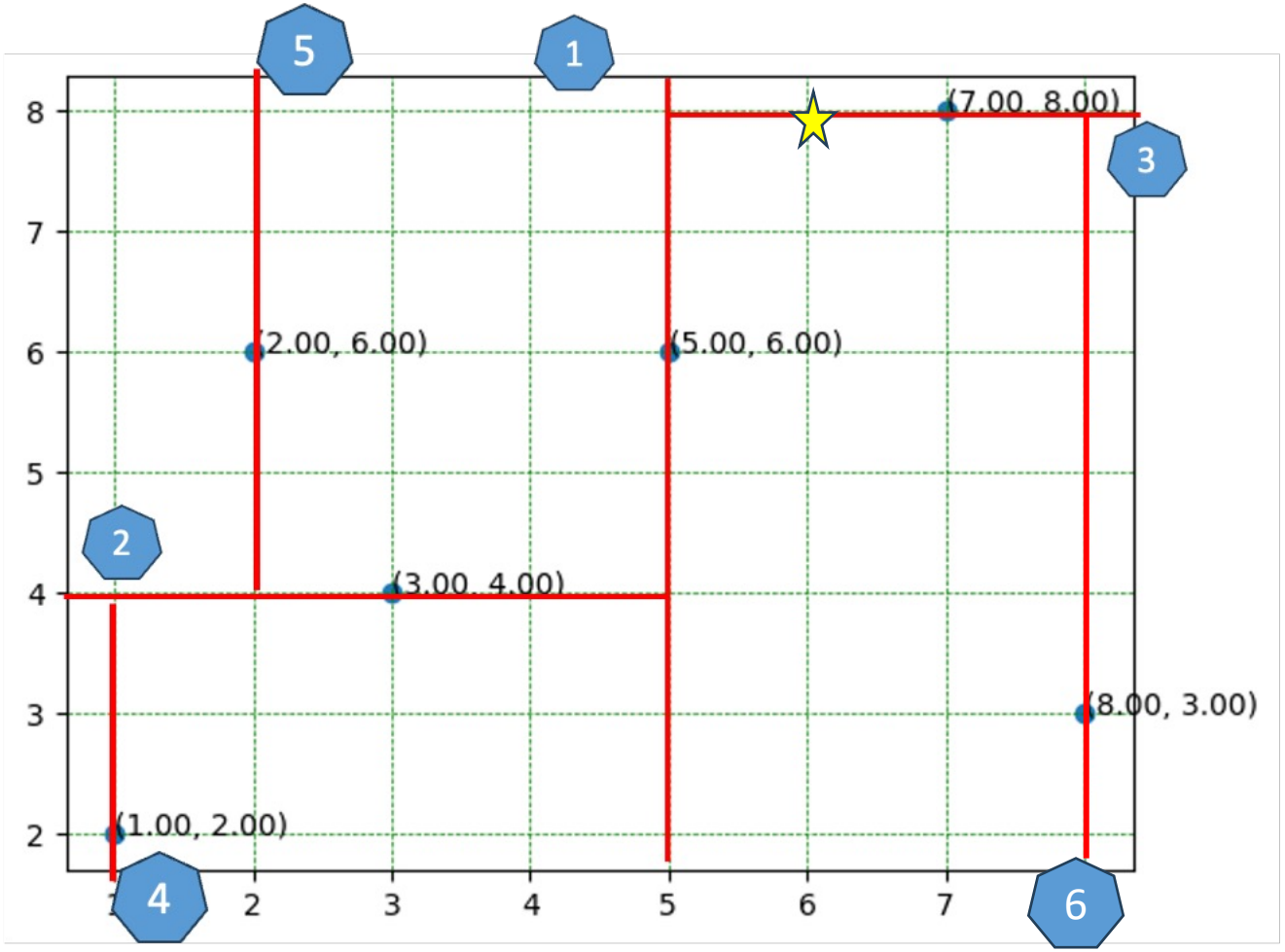
The closest points is (5,6)

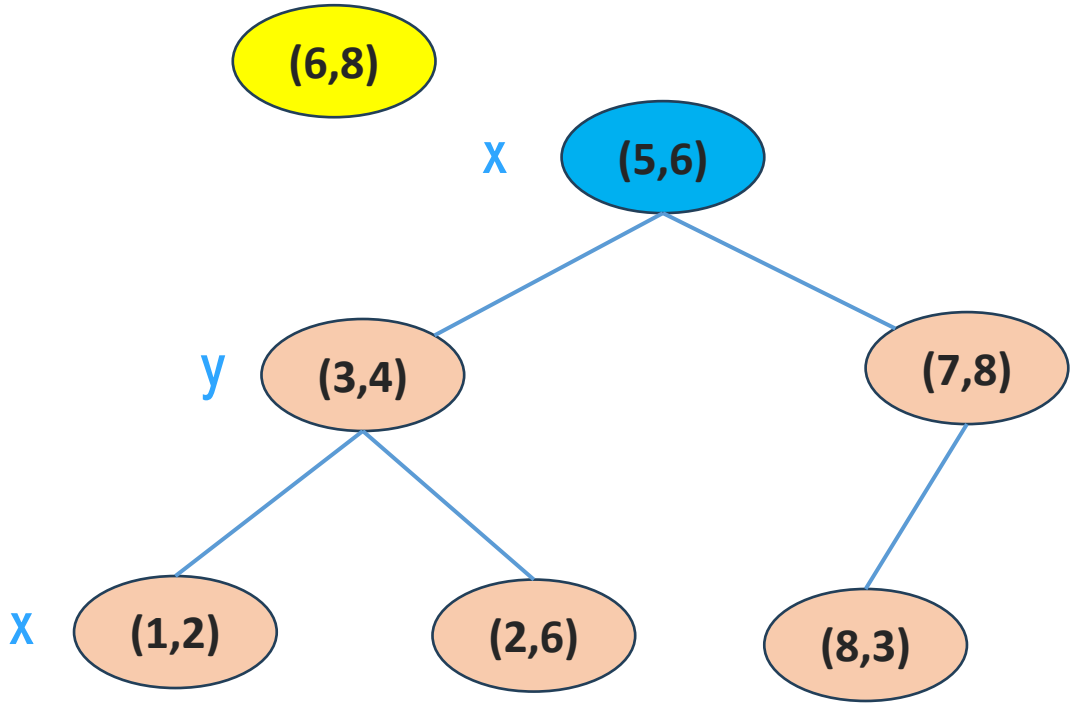
Best distance = 1.41
Best label = (5,6)

Curent distance = 2.82
Curent label = (8,3)

Not update the best distance

K-D Tree Implementation Solution





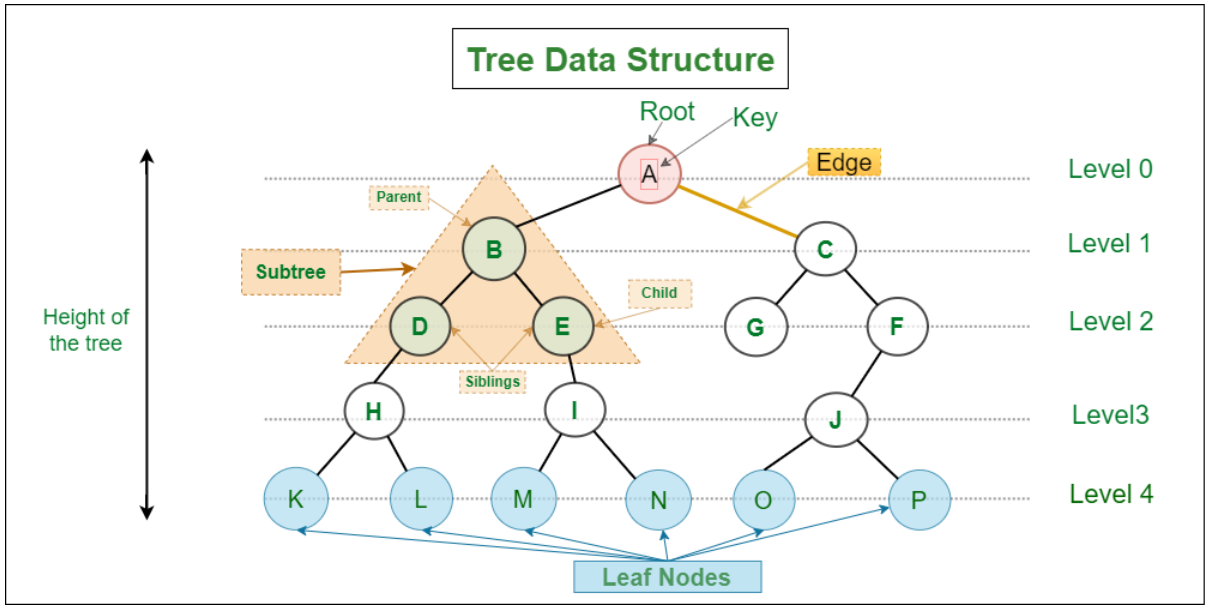
★ **New datapoint**

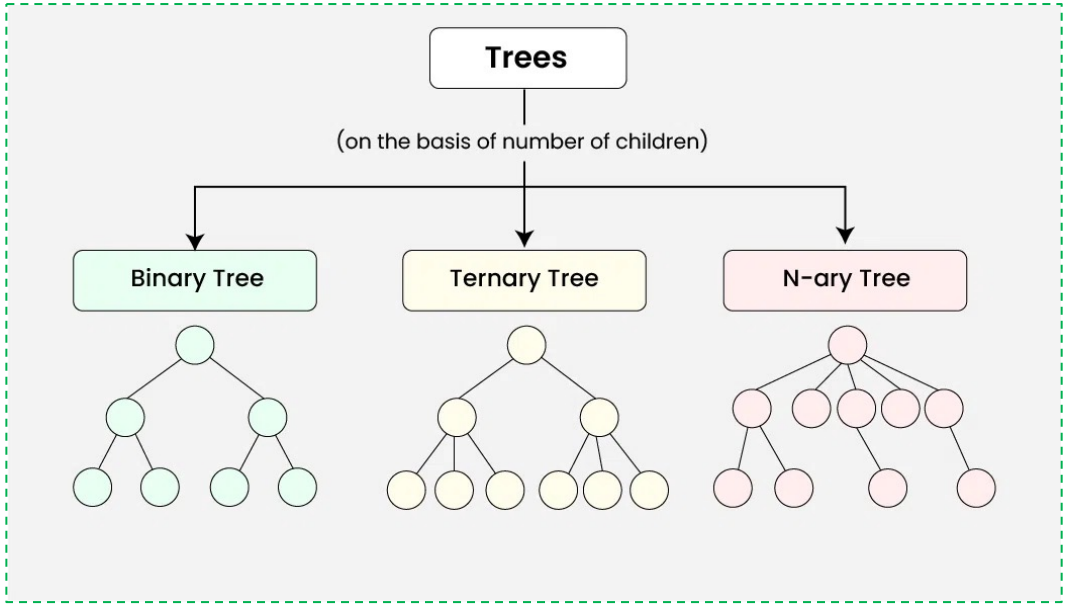
What is the nearest neighbor?

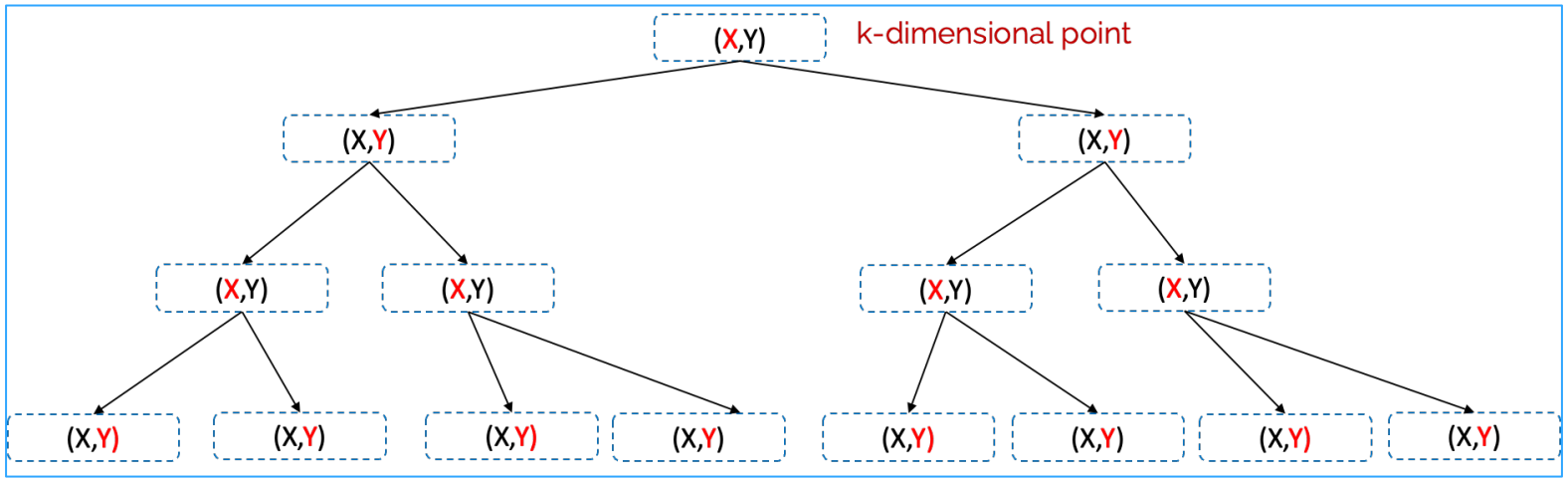
Best distance = 1.41
Best label = (5,6)

AI VIET NAM
@aivietnam.edu.vn

Summary







References

Problem Solving with Algorithms and Data Structures

Release 3.0

Brad Miller, David Ranum

September 22, 2013